

ספר פרויקט

רמת השרון **תיכון אלון לאמנויות ולמדעים** **אלון** ^{תיכון}

- שם בית הספר - תיכון אלון רמה"ש
- שם העבודה - DaVinci Mail
- שם התלמיד - גיא משה טארנטו
- ת.ז. התלמיד - 331719252
- שם המנחה - עידן פלדברג
- שם החלופה - "הגנת סייבר"
- תאריך ההגשה - 21.5.25



DaVinci Mail

תוכן עניינים

- 2.....★ מבוא.
- 6.....★ תיאור תחום הידע.
- 9.....★ מבנה / ארכיטקטורה.
- 47.....★ מימוש הפרויקט.
- 73.....★ מדריך למשתמש.
- 88.....★ סיכום אישי / רפלקציה.
- 90.....★ ביבליוגרפיה.
- 91.....★ נספחים - קוד הפרויקט.

מבוא

ייזום

- **תיאור ראשוני של המערכת;** המערכת אותה בניתי במהלך כתיבת הפרויקט, המוצר המוגמר הינו שירות מייל לכל דבר ועניין. השירות ממוקד לשימוש על בסיס רשת פנימית (LAN). באמצעות השירות, כמו בכל שירות מייל ניתן לפתוח חשבון משתמש עם מייל ייחודי, לשלוח הודעות, לצרף קבצים, לחסום משתמשים, לתייג הודעות באמצעות תגיות שונות, לעשות סינונים שונים ועוד...
- תחילה הייתה לי התלבטות גדולה בנוגע לנושא הפרויקט, התלבטתי בין נושאים שונים ומגוונים. באחד מהימים שחשבתי על נושא לפרויקט, במהלך חיפוש רעיונות בגוגל קיבלתי מייל, ואז חשבתי אולי לעשות כפרויקט שירות מייל.
- התחלתי לחשוב על כלל הדברים שאצטרך לממש לשם כתיבת שירות מייל רציני ומכובד. לאחר סיכום הדברים וסיום תהליך החשיבה, מסקנותיי היו כי אצטרך לממש קוד בנושאים רבים, מגוונים ומורכבים לשם יצירת שירות המייל וכי אלמד הרבה דרך עשייתו ולכן בחרתי בפרויקט. הבנתי שדרך עשייתו אוכל לחזק את הידע שלי בתחומים שונים בתכנות ואלמד ידע נוסף ונרחב. האתגרים שאני צופה לעצמי במהלך עשיית הפרויקט הם ההמשך לסיבות בחירתו - הקוד יהיה מורכב וישלב תחומים רבים, אצטרך להתמודד עם מצבים שונים ויוצאי דופן, לחשוב על תקלות אפשריות, למנוע פריצות אבטחה ולשמור על אבטחת מידע, לבנות ממשק גרפי קל ופשוט, לבנות שרת מרובה משתמשים, יצירת פרוטוקול תקשורת. לא מעט דברים...
- כמו כן, השנה הינה שנה עמוסה, אני ועוד כמה חברים נרשמנו לתחרות כספות פיזיקה, לתחרות עלינו לבנות פרויקט שהינו בסדר גודל של הפרויקט הנ"ל. עקב כך, שילוב עשיית שני הפרויקטים הללו והעומס הלימודי הרגיל לא עומד להיות פשוט.
- **הגדרת הלקוח;** המערכת, שירות המייל, מיועד לכל אדם אשר חפץ להשתמש בשירות מייל יעיל, מוצפן, קל ונוח לשימוש, לכן יש לי קהל לקוחות נרחב - כל אדם פוטנציאלי. אך בשל כך שהמערכת עובדת על גבי הרשת הפנימית בלבד, שירות המייל יכול להיות מצוין בשימוש פוטנציאלי לחברות, מוסדות לימודיים וגופים בעלי רשתות פנימיות בסדר גודל דומה.
- **הגדרת יעדים/מטרות;** היעדים המרכזיים שיהיו לי במהלך הפיתוח - מימוש תקשורת בין השרת ללקוחות, כתיבת ממשק קל ונוח לשימוש, מניעת התקפות, אבטחת מידע, הוספת פיצ'רים מעניינים והעשרת אופקיי. מימוש כלל היעדים יובילו למטרה המרכזית - שירות מייל יעיל, מוצפן, קל ונוח לשימוש.
- **בעיות תועלות וחשכונות;** המערכת לא עונה על שום בעיה שאין לה כיום פיתרון חינומי ונגיש. המערכת הינה שירות מייל יעיל, מוצפן, קל ונוח לשימוש ואלו התועלות שסביר לצפות ממנה כמו מכל שירות מייל. השירותים אותם המערכת תיתן היא היכולת לפתוח חשבון, לתקשר עם כל שאר משתמשי השירות באופן מאובטח - באמצעות שליחת מיילים עם יכולת הוספת קבצים (עד גודל מסוים), סינונים שונים (labels, מיילים חשוכים וכדומה), רשימות תפוצה, בדיקת שפה נאותה, חסימת משתמשים, שינוי סיסמא קיימת (Reset/forgot password) ועוד...
- **סקירת פתרונות קיימים;** כיום קיימים שירותי מייל שונים ומגוונים אשר מספקים שירות מייל נוח וקל לשימוש לאנשים פרטיים ולחברות, המוכר ביותר הינו של Gmail של חברת Google. ישנם גם את Outlook, Yahoo ועוד נוספים...
- שאיפתי בפרויקט הייתה לכך ששירות המייל שלי יהיה דומה כמה שיותר ביכולותיו לזה של Google.
- **סקירת טכנולוגית הפרויקט וקיום סייגים בהגדרתה;** הטכנולוגיה בפרויקט איננה חדשה ואיננה בלתי מוכרת, הפרויקט הינו שרת מייל וכפי שציינתי בפסקה הקודמת ישנם לא מעט שירותים מקבילים. אין שום סייגים בהגדרת המערכת, כל משתמש יכול להשתמש בשירות ללא שום מגבלה או חסם, אלא אם יעבור על תקנון השימוש שנמצא בחלון ההגדרות. ה"סייג" היחיד שניתן לציין הינו שהפרויקט עובד על גבי הרשת הפנימית (LAN), אך אלו הן דרישות הפרויקט.
- **תיחום הפרויקט, התחומים והתחומים בהם המערכת לא מטפלת;** הפרויקט מתעסק בתחום הרשתות באמצעות הקמת שרת מרובה לקוחות, כתיבת קוד לקוח, העברת מידע באמצעות פרוטוקול TCP בעזרת ספריית Socket של python, וקיום פרוטוקול תקשורת המבוסס על העברת מילונים עם תבנית קבועה ותוספות. זאת ועוד, הפרויקט מתעסק בתחום אבטחת מידע

ע"י שימוש בגיבוב ססמאות בעת שמירתן במסד נתוני השירות וע"י שימוש ב-RSA, AES ושינוי סדר הביטים במידע הנשלח על מנת להעביר כל הודעה מהשרת ללקוח ולהפך. בנוסף, תחום הגנת הסייבר קיים גם כן, כתבתי בקוד הפרויקט הגנה מובנית למתקפת מסוג DOS ו-DDOS, ע"י השהייה רנדומלית מובנית בתקשורת בין הלקוח לשרת והגבלת כמות הלקוחות שהשרת יכול לשרת ברגע נתון. יתר על כן, ישנו שימוש בפרמטרים בפנייה ל-database על מנת למנוע SQL injection. שני תחומים נוספים המובעים בפרויקט הם תחום מסדי הנתונים אשר מהווים את ליבת שמירת המידע בפרויקט ותהליכונים (threading) הכולל שימוש ב-lock בעת גישה למסד הנתונים בצד השרת. יתרה מכך, הפרויקט עוסק בתחום ממשק המשתמש (gui) שעליו עמלתי רבות, באמצעות ספריית customtkinter. הפרויקט איננו עוסק בפיתוח WEB, למשל בפרוטוקול HTML או בשפת JavaScript, ישנם שירותי מייל מבוססי אפליקציה (כמו שלי), שירותים מבוססי אתר ושירותים מבוססי אתר ואפליקציה.

פירוט תיאור המערכת (אפיון)

- **תיאור מפורט המערכת, מה היא אמורה לעשות? המערכת אותה כתבתי הינה מערכת שירות מיילים חנימית, הפתוחה לשימוש כל אדם או עסק החפץ בה על בסיס הרשת הפנימית (Lan). המערכת הינה מאובטחת מקצה לקצה, בעלת מנגנון בדיקה לשפה נאותה (ניתן להיחסם אם משתמשים בשפה לא נאותה יותר מכחמש פעמים - כמצוין בדף המיועד לחוקי השירות המופיע באפליקציה), סינון מיילים לפי קטגוריות (Labels, מיילים חשובים, חיפוש תוכן ספציפי וכו'), שימוש ברשימות תפוצה, העברת קבצים שונים ותמונות באמצעות מיילים אותם כל משתמש יכול לשלוח לכל משתמש שירות אשר יחפץ (מנגד משתמש שירות יכול לחסום כל משתמש אחר מלשלוח לו מיילים). זאת ועוד, בעת פתיחת האפליקציה ניתן להתחבר, להירשם (הגדרת סיסמא ומענה על שתי שאלות אבטחה) או לבצע איפוס סיסמא באמצעות שתי שאלות האבטחה עליהן המשתמש השיב בעת הרשמתו לשירות. למערכת יש ממשק נוח וקל לשימוש, מטרתה ועשייתה המקורית הינה לספק שירות מיילים נוח, מאובטח וקל לשימוש.**
- **פירוט היכולות שהיא תעניק לכל סוג משתמש במערכת; במערכת שלי ישנו סוג משתמש אחד ויחיד - משתמש רגיל. להלן פירוט כלל היכולות הזמינות לכלל המשתמשים;**
 - ☐ הרשמה
 - ☐ התחברות
 - ☐ שינוי סיסמא
 - ☐ שליחת הודעות (כולל קבצי TXT ותמונות)
 - ☐ צפייה בהיסטוריית המיילים אשר קיבלתה.
 - ☐ שלוש רשימות תפוצה שונות
 - ☐ חמישה labels שונים לשמירת המיילים (מייל יכולה להישמר תחת כמה labels שונים, בנוסף המשתמש יכול לשנות את שמות ה-labels כרצונו).
 - ☐ סינון מיילים
 - ☐ פילטור הודעות
 - ☐ חיפוש תוכן בהודעות
 - ☐ העברת הודעות (Forward)
 - ☐ הצגת תוכן ההודעה והקבצים
 - ☐ התנתקות
 - ☐ שינוי צבע התצוגה הראשי
- **פיצ'רים נוספים שהוספתי -**
 - ☐ קבלת מייל מהחשבון הרשמי של התוכנה (אשר לא ניתן ליצור חשבון כמוהו ולא ניתן לחסום אותו - מובנה בקוד) ביום ההולדת של המשתמש.
 - ☐ חסימת משתמשים לא רצויים (ההודעות שנשלחות מהם יתקבלו וישמרו במסד נתוני השרת למקרה של הורדת החסימה)

□ מניעת דיבור לא הולם - משתמש שיכתוב במיילים שלו מילים לא הולמות (מתוך אוסף מיילים המובנה בקוד) חמש פעמים ומעלה יקבל חסימה (ban) מהשירות. לשם ההומור משתמש שנחסם יקבל הודעה כי על מנת להחזיר את החשבון עליו לשלם בסה"כ 100 דולר 😊.

□ חסימה בזמן אמת - משתמש שיעבור על כללי השירות בפעם החמישית ינותק מהאפליקציה ויקבל חסימה מהשירות בזמן אמת, על מנת שלא יוכל להמשיך בכך.

● פירוט הבדיקות (קופסא שחורה) שמתוכננות לפרויקט;

□ **בדיקה 1;** בדיקת תקינות מנגנון הכנת הודעה לשליחה וקבלתה - המרה לייצוג JSON, הצפנה, דחיסה ובדיקת תקינות מנגנון קבלת הודעה וביצוע אי עיבוד - חילוץ, פענוח והמרה מ-JSON לטיפוס python. יש בצד השולח (שרת או לקוח) להדפיס את המידע לפני שלבי העיבוד השונים, בין שלבי העיבוד השונים ואת המידע לאחר שנדחס - שלב העיבוד האחרון. לאחר מכן יש להדפיס את המידע בצד המקבל (שרת או לקוח) לפני החילוץ, שלב אי-העיבוד הראשון ובין שלבי אי-העיבוד הנוספים (שקורים בסדר הפוך משלבי העיבוד). לבסוף יש לראות כי ההודעה עצמה מ"שפת" הפרוטוקול שנשלחה הגיע לצד השני כפי שנשלחה וללא שינוי כזה או אחר.

□ **בדיקה 2;** בדיקה שההגנות אכן עובדות, יש להסניף באמצעות wireshark את המידע ולדמות מתקפת Man in the middle. יש לראות כי כל אדם/בוט שינסה לקרוא את המידע המועבר בין השרת ללקוח לא יבין יותר מידי למעט מהפרטים הקשורים לשכבות השונות (למשל הפרטים הקשורים לפרוטוקול TCP) אותם אין עליי לשבש בשל כך שההודעה לא תגיע ליעדה. כמו כן, יש לנסות לבצע SQL injection בעת ניסיון התחברות ע"י קלטים אופייניים למתקפה כמו "random_passwd or 1=1" ולראות כי התחברות לא מאושרת. זאת ועוד, יש לחשוב האם משתמש יוכל להזין בקלטים מסוימים תווים מסוימים בעל פוטנציאל לפגיעה בתפקוד השירות, למשל כתובת מייל המכילה "/", רווח וכדומה...

□ **בדיקה 3;** תפקוד מסד הנתונים - ראשית יש לבדוק האם מסדי הנתונים מתעדכנים בהתאם לנתונים המוזנים אליהם ושנית יש לבדוק כי כל הפעולות הכרוכות בשימוש במסדי הנתונים פועלות כראוי לפי הנתונים השונים.

□ **בדיקה 4;** האם המידע האישי של הלקוח (למשל סינונים, חסימות, העדפות וכו') מתעדכן ונשמר במסד הנתונים בעת **התנתקות**. יש לבדוק כמה פעמים ועם חשבונות שונים האם לאחר התנתקות המשתמש אין שינוי לא תקין במידע האישי שלו והאם **בהתחברות** מחדש כל המידע מוצג ומעודכן כראוי בצד הלקוח, בדיקה זו איננה רק לגבי מסד הנתונים עצמו, אלא גם הצגה ב-gui ועוד...

□ **בדיקה 5;** פרטיות, יש לראות האם בעת כניסה מחודשת ללא סגירת התוכנה, המידע של המשתמש הקודם שהיה מחובר (הודעות וכו') לא מוצג למשתמש האחרון שהתחבר.

□ **בדיקה 6;** מהם מקרי הקצה הרלוונטיים לשירות? ושגיאות כאלו ואחרות העלולות לצוץ? בדיקה זו איננה ספציפית, היא כללית ונעשתה לאורך כל עשיית הפרויקט, אומנם יש להתמקד ולשים דגש על שני מקרים עקרוניים, התנתקות פתאומית של אחד מצדדי התקשורת או בעיה במסד הנתונים. יש לחשוב ולנסות לחזות כמה שיותר מקרי קצה מראש כדי להימנע מלפתור ולהתמודד עימם לאחר שיצוצו בזמן אמת.

□ **בדיקה 7;** gui - האם ממשק המשתמש ברור, נוח, קל ויעיל לשימוש לשם חווית משתמש מיטבית? יש לתת לגורמים שהם לא אני (ליוצר הממשק הוא תמיד יהיה ברור) כמו משפחה וחברים להתנסות בשימוש בשירות. בנוסף, בדיקת התאמת גודל מסך השירות לגודל מסך המחשב - ישנם שני מצבים למסך השירות, רגיל וקטן. יישמתי בקוד בדיקה לכך שאם מסך המחשב קטן מידי מסך התוכנה יעבור התאמות ויוצג כקטן יותר (יום אחד עבדתי על הפרוייקט במחשב נייד עם מסך קטן יחסית, וראיתי שלא ניתן להשתמש בשירות כראוי, לכן ביצעתי התאמות למצב של מסך קטן).

● תכנון וניהול לו"ז זמנים לפיתוח המערכת;

משימה	פירוט	זמן מוארך	זמן ביצוע בפועל
בחירת נושא והגשת הצעה	בחירת נושא הפרויקט, תכנון והגשת הצעה למורי המגמה בבית הספר.	שבועיים	שבועיים
כתיבת בסיס הקוד לפרויקט	כתיבת הקוד הבסיסי לפרויקט, שרת - לקוח, מסד נתונים בסיס, gui בסיסי ועוד...	שלושה שבועות	שבועיים וחצי
הפיכת הקוד מבסיסי לפרויקט סופי	כאשר בונים בית קודם כל מתכננים אותו, לאחר מכן בונים את השלד, קווי המים והחשמל ולאחר שלב זה מתחילים בהוספת ה"פיצ'רים", מטבח, מקלחות וכו'. כך גם אני אפעל, תחילה אבצע תכנון מוקדם, לאחר מכן אבנה את שלד הפרויקט ולאחר מכן אתחיל בהוספת הפיצ'רים השונים.	עד יום הגשת ספר פרויקט למורי המגמה, יתכן והוסיף שיפורים מינוריים עד ליום הבחינה, כפי שאושר ע"י מורי המגמה.	כמה ימים לפני מועד הגשת ספר הפרויקט המערכת הייתה מוכנה.
כתיבת ספר הפרויקט	תוך כדי העמקת העבודה על הפרויקט אכתוב את הספר עצמו.	חודש	שלושה וחצי שבועות.

- **ניהול הסיכונים בפרויקט והדרכים להתמודדות איתם**; בביצוע הפרויקט ותכנונו היו סיכונים, העיקרי והמרכזי בהם הוא שחלק מהקוד לא יעבוד, במקרה שכזה אחשוב לעומק על הבעיה, אתייעץ עם חבר או מורה ואחקור באינטרנט עד למציאת פתרון. סיכון נוסף הוא שאולי לא אספיק לממש את כל המטרות ששמתי לעצמי (בהתאם לדרישות הפרויקט, ומעבר - דרישות שלי לעצמי), ציינתי שהשנה הייתה עמוסה מאוד, לכן תיעדפתי קודם כל ביצוע של הדברים ההכרחיים ולאחר מכן את התוספות. זאת ועוד, מכיוון שישנם חומרים רבים שנאלצנו ללמוד לעומק לבד, או שנלמדו באופן בסיסי בבית הספר, הייתה לנו בעיה לחשוב על אילו פרויקטים ניתן לעשות עם הידע הקיים לפני תחילת עשיית הפרויקט. עקב כך, גם עשיית הפרויקט עצמו לקחה זמן משמעותי. זמן משמעותי זה גרר את עשיית הפרויקט אל תוך תקופת הבגרויות וגרם ללוח זמנים צפוף ביותר בעשייתו.

תיאור תחום הידע

פירוט יכולות בצד שרת:

שם היכולת	מהות היכולת	אוסף היכולות/פעולות הנדרשות למימוש היכולת	אובייקטים נחוצים
הרשמה למערכת (Register)	רישום של משתמש חדש ; הגדרת מייל וסיסמא ומענה על שתי שאלות אבטחה אישיות.	ממשק משתמש- מסך הרשמה, קליטת נתונים, בדיקת תקינות, ביצוע גיבוב, דחיסה והצפנה, שליחה לשרת המרכזי, פענוח וחילוץ, בדיקת תקינות מול מסד הנתונים בשרת, תקין - אחסון המידע במסד הנתונים, ייצור תשובה, הצפנה ודחיסה חוזרת, שליחת התשובה מהשרת, קבלת תשובה מהשרת, חילוץ ופענוח והצגת התשובה. למשתמש.	ממשק משתמש, הצפנה/פענוח, תקשורת בין שרת ללקוח, מסד נתונים, גיבוב.
חיבור משתמש למערכת (Login)	חיבור משתמש ע"פ מייל וסיסמא והעברתו למסך הראשי.	ממשק משתמש- מסך כניסה, קליטת נתונים, בדיקת תקינות, דחיסה והצפנה, שליחת הנתונים לשרת, פענוח וחילוץ, הצלבה מול הנתונים במסד הנתונים בשרת (פעולת verify מול הערך המגובב), הצפנת ודחיסת תשובה, קבלה בלקוח, חילוץ ופיענוח ותוצאה בהתאם.	ממשק משתמש, דחיסה/חילוץ, הצפנה/פענוח, תקשורת בין שרת ללקוח, מסד נתונים, גיבוב.
שמירת הודעה במסד הנתונים (receive message)	הוספה של הודעה למסד הנתונים של השרת.	ממשק משתמש - מסך שליחת הודעה, קליטת נתונים, בדיקת תקינות, דחיסה והצפנה, שליחת הנתונים לשרת, בדיקת נתונים, תקין - שמירה במסד נתונים, שמירה במסד הנתונים, דחיסה והצפנה של הודעת סטטוס ההודעה מהשרת ללקוח, פענוח וחילוץ, הצגה למשתמש - ממשק משתמש במסך הראשי.	ממשק משתמש, דחיסה/חילוץ, הצפנה/פענוח, תקשורת בין שרת ללקוח, מסד נתונים.
שליחת הודעות חדשות ללקוח (retrieve messages)	שליחת הודעות אשר מסומנות כלא נקראו ללקוח.	ממשק משתמש - כפתור רענון, הצפנה ודחיסה של הבקשה, פענוח וחילוץ בצד השרת, בדיקה במסד להודעות חדשות, דחיסה והצפנה של תגובה, פענוח וחילוץ בצד הלקוח וממשק משתמש - הצגת תגובה או תוספת של ההודעות החדשות.	ממשק משתמש, דחיסה/חילוץ, הצפנה/פענוח, תקשורת בין שרת ללקוח, מסד נתונים.
שינוי סיסמא (forgot password)	שינוי הסיסמא באמצעות מענה נכון על שאלות האבטחה האישיות שנבחרו בהרשמה לחשבון.	ממשק - כפתור מתאים, שליחת בקשה דחוסה ומוצפנת לקבלת התשובות והשאלות האישיות, פענוח וחילוץ הבקשה, שליפת נתונים ממסד הנתונים בשרת, דחיסה והצפנה, קבלת הנתונים בצד השרת, חילוץ ופענוח, קליטת נתוני תשובות הלקוח, הצלבה מול הנתונים שהתקבלו מהשרת, לא מתאים - הודעה מתאימה, מתאים - דחיסה והצפנה של הודעה עם הסיסמא החדשה, פענוח וחילוץ בשרת, עדכון במסד הנתונים.	ממשק משתמש, דחיסה/חילוץ, הצפנה/פענוח, תקשורת בין שרת ללקוח, מסד נתונים.

גיא משה טארנטו - DaVinci Mail

התנתקות (logout), כולל שינוי מאפיינים יחודיים / פילטור הודעות לפי פרמטרים שונים.	עדכונים אודות ; חסימת משתמשים, שינוי צבע כפתורי התוכנה, פילטור הודעות לפי פרמטרים (חשוב, קריאה לאחר כך וכדומה..)	ממשק - כפתור מתאים או כפתור 'X', שליחת מידע דחוס ומוצפן אודות התנתקות, הוספת פילטרים, חסימות וכו', פענוח וחילוף המידע בשרת, עדכון נתונים במסד הנתונים בשרת במידת הצורך.	ממשק משתמש, דחיסה/ חילוף, הצפנה/פענוח, תקשורת בין שרת ללקוח, מסד נתונים.
חיבור לקוח למערכת	חיבור לקוח חדש לשרת.	קבלת חיבור לקוח והתחלת בתהליך ביצוע ה-Handshake של השירות ; שליחת הודעה האם ניתן לבצע חיבור (מספר הלקוחות המחוברים נמוך המקסימלי), לא ניתן - סגירת ה-socket, ניתן - דחיסת ושליחת המפתח הציבורי של השרת ללקוח, קבלת המפתח, חילוצו ושמירתו במחשב הלקוח, הצפנת מפתח ה-AES של הלקוח ומספר הביטים להזזה, דחיסה, שליחה לשרת, קבלה וחילוף.	הצפנה/פענוח, דחיסה/ חילוף, תקשורת בין שרת ללקוח,

פירוט יכולות בצד לקוח :

שם היכולת	מהות היכולת	אוסף היכולות/פעולות הנדרשות למימוש היכולת	אובייקטים נחוצים
הרשמה למערכת (Register)	רישום של משתמש חדש ; הגדרת מייל וסיסמא ומענה על שתי שאלות אישיות.	ממשק משתמש- מסך הרשמה, קליטת נתונים, בדיקת תקינות, דחיסה והצפנה, שליחה לשרת המרכזית, פענוח וחילוף, בדיקת תקינות מול מסד הנתונים בשרת, דחיסה והצפנה חוזרת, שליחת תשובה מהשרת, קבלת תשובה מהשרת, פענוחה והצגת התשובה למשתמש.	ממשק משתמש, הצפנה/פענוח, דחיסה/ חילוף, תקשורת בין שרת ללקוח, מסד נתונים.
חיבור משתמש למערכת (Login)	חיבור משתמש ע"פ מייל וסיסמא והעברתו למסך הראשי.	ממשק משתמש- מסך כניסה, קליטת נתונים, בדיקת תקינות, דחיסה והצפנה, שליחת הנתונים לשרת, חילוף ופענוח, הצלבה מול הנתונים במסד הנתונים בשרת, דחיסת והצפנת תשובה, קבלה בלקוח, חילוף ופיענוח ותוצאה בהתאם.	ממשק משתמש, הצפנה/פענוח, דחיסה/ חילוף, תקשורת בין שרת ללקוח, מסד נתונים.
קבלת מיילים חדשים, כלומר ביצוע רענון.	שליחת מיילים אשר מסומנים כלא נקראו ללקוח.	ממשק משתמש - אופציה refresh, דחיסה והצפנה של הבקשה, שליחת הבקשה, קבלה בצד השרת, חילוף ופענוח, בדיקה במסד הנתונים להודעות חדשות, דחיסה והצפנה של תגובה, חילוף ופענוח בצד הלקוח וממשק משתמש - הצגת תגובה או תוספת של ההודעות החדשות. במידה ויש הודעות חדשות, עדכונים במסד הנתונים בצד הלקוח.	ממשק משתמש, הצפנה/פענוח, דחיסה/ חילוף, תקשורת בין שרת ללקוח, מסד נתונים.

שליחת מייל למשתמש.ים	שליחת מייל למשתמש אחר או לכמה משתמשים אחרים ביחד (רשימת תפוצה).	ממשק משתמש - כפתור compose וחלונת שליחה, קליטת הקלט, בדיקת שפה נאותה ותקינות כתובת, במידה ותקין - דחיסה והצפנה של הבקשה, חילוץ ופענוח בצד השרת, בדיקה במסד להודעות חדשות, דחיסה והצפנה של תגובה, שליחת התגובה, קבלה בצד הלקוח, חילוץ ופענוח. ממשק משתמש - הצגת תגובה (אין מיילים חדשים) או תוספת של ההודעות החדשות.
שינוי סיסמא (forgot password)	שינוי הסיסמא באמצעות מענה נכון על השאלות האישיות שנבחרו בהרשמה לחשבון.	ממשק - כפתור מתאים במסך הפתיחה הפותח מסך לפעולה, שליחת בקשה דחוסה ומוצפנת לשרת לקבלת שאלות האבטחה האישיות עליהן הלקוח ענה בעת הרשמה, קבלת הבקשה בשרת, חילוץ ופענוח הבקשה, שליפת נתונים ממסד הנתונים בשרת, דחיסה והצפנה, קבלת הנתונים בצד השרת, חילוץ ופענוח, ממשק - הצגת השאלות ללקוח, קליטת נתוני תשובות הלקוח וסיסמא חדשה (בדיקה שהיא חזקה), דחיסה והצפנה, שליחה לשרת, קבלה בשרת, חילוץ ופענוח, הצלבה מול הנתונים בשרת, לא מתאים - ניסוח הודעה על כך, מתאים - עדכון הסיסמא במסד הנתונים וניסוח תשובה חיובית, דחיסה והצפנה, שליחת התשובה ללקוח, חילוץ ופענוח, הצגת התשובה למשתמש.
שינוי הגדרות ומאפיינים ייחודיים	שינוי צבע כפתורי התוכנה, הוספת משתמשים לחסימה, קריאת חוקי השירות ועוד...	ממשק - כפתור גישה במסך הראשי למסך ההגדרות, כפתורים לעדכון הנתונים (כמו חסימת משתמש, שינוי שם label) שיפתחו חלונות מתאימים לקליטת קלט, עדכון המשתנים / רשימות הרלוונטיות. בנוסף ממשק - כפתור כניסה למסך התקנון וכפתור חזרה ממנו למסך ההגדרות.
ביצוע פילטור וסינון מיילים והצגה לפי כך.	פילטור וסינון הודעות לפי פרמטרים למשל "חשוב", "קריאה לאחר כך", labels שולח ועוד...	ביצוע סינון/פילטור, ממשק - כפתורים ובחירת אופציות, מסד נתונים בצד הלקוח אשר יכיל את המיילים והשדות המתאימים לשם הסינון והפילטור, עדכון נתוני המסד בעקבות פעולות הלקוח. הצגת מסוננת/מפולטרת - ממשק - כפתורים מתאימים והצגת מיילים לפי בחירת הלקוח (בהתאם לסינון/קטגוריה / פרמטר וכו') במסך הראשי.
התנתקות (logout), כולל עדכון במסד נתוני השרת על שינוי מאפיינים ייחודיים וסינון/פילטור מיילים לפי פרמטרים שונים.	עדכונים אודות; חסימת משתמשים,	ממשק - כפתור מתאים, הוצאת נתונים ממשתנים, רשימות וממסד נתוני הלקוח (לאחר מכן ריקונו - פרטיות), אסיפת המידע ושליחתו כדחוס ומוצפן
ממשק משתמש, הצפנה/פענוח, דחיסה/ חילוץ, תקשורת בין שרת ללקוח, מסדי נתונים.	ממשק משתמש, הצפנה/פענוח, דחיסה/ חילוץ, תקשורת בין שרת ללקוח, מסדי נתונים.	ממשק משתמש, הצפנה/פענוח, דחיסה/ חילוץ, תקשורת בין שרת ללקוח, מסדי נתונים.

	שינוי צבע כפתורי התוכנה, סינון/פילטור מיילים.	אודות התנתקות, הוספת פילטרים, קבלה בשרת, חילוץ ופענוח המידע בשרת, עדכון נתונים במסד נתוני השרת במידת הצורך.
--	---	--

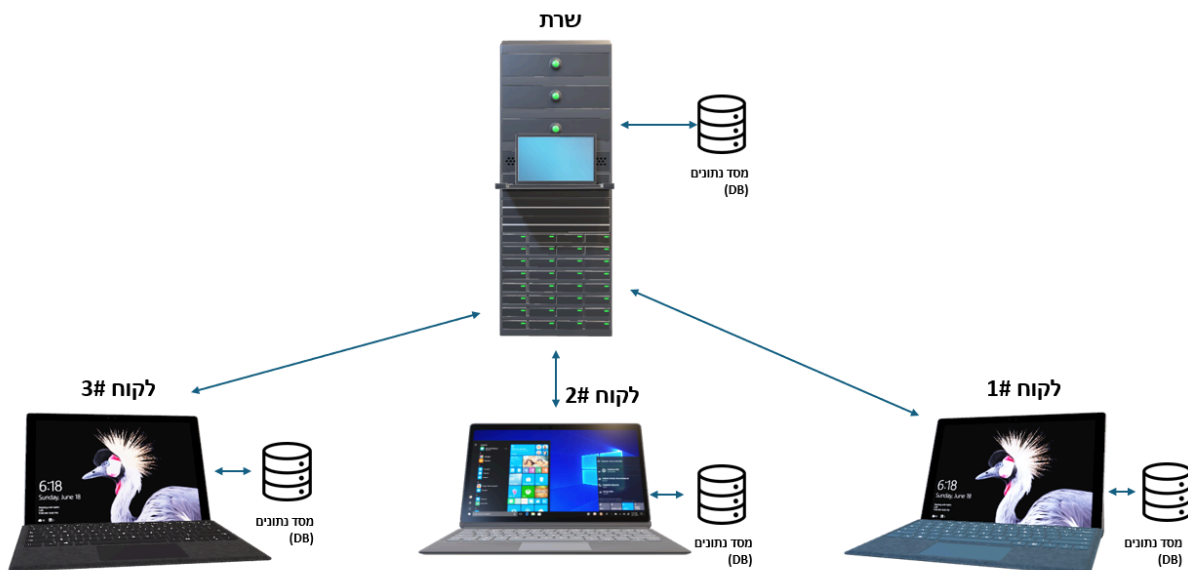
מבנה / ארכיטקטורה

תיאור הארכיטקטורה של המערכת

תיאור החומרה – רכיבים שונים והקשריהם; הפרויקט פועל על בסיס תקשורת מרובת משתתפים הממומשת בעזרת ספריית **Socket** ופרוטוקול **TCP**. לשם הפעלתו ניתן להשתמש במחשב אחד, אשר יריץ את קוד השרת ומספר פעמים את קוד הלקוח, בעצם על המחשב הנ"ל יהיה גם שרת וגם מספר לקוחות שיכולים לתקשר זה עם זה. אומנם, שימוש שכזה אינו אידיאלי ומומלץ לבצע שימוש בלפחות שלושה מחשבים או יותר; מחשב אחד אשר יריץ את קוד השרת ויהווה את שרת השירות, יכיל ויתקשר עם מסד נתונים ויאזין לקבלת לקוחות חדשים. בנוסף, מומלץ להשתמש בשני מחשבים או יותר שיריצו את קוד הלקוח, אשר יתחברו לשרת ומשתמשי המחשבים יוכלו להשתמש בשירות להנאתם. לשרת יש מסד נתונים נפרד ועצמאי משלו המכיל את נתוני כלל משתמשי השירות. כמו כן, לכל לקוח בעצמו יש מסד נתונים מקומי הפועל בעת הפעלת הקוד לצורך פעולת השירות. *לאחר התנתקות מסד הנתונים בצד הלקוח מרוקן מתוכנו.

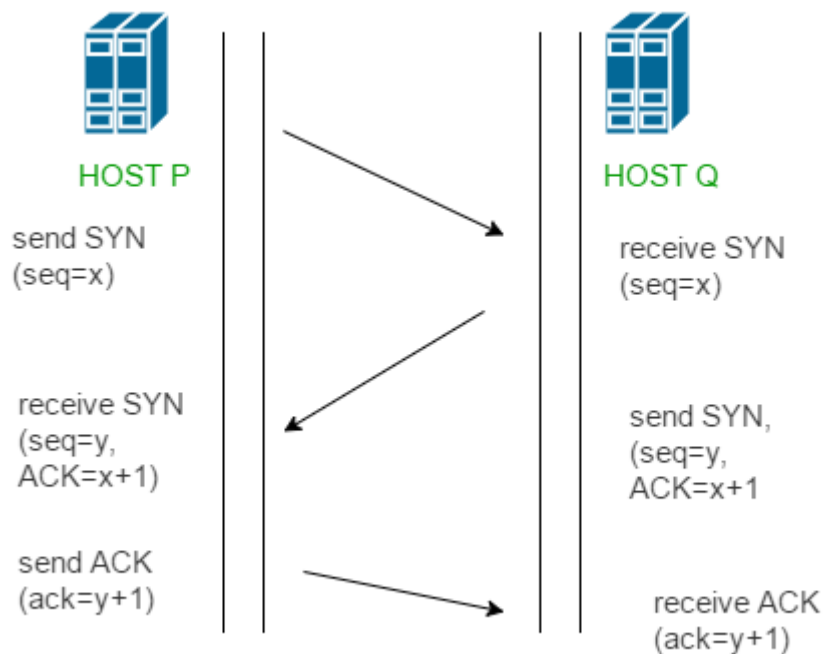
איור המציג קשרים אלו;

האיור הבא מתאר את היחסים בין רכיבי החומרה במערכת בשימוש אידיאלי ומומלץ, שלושה מחשבים או יותר; כלל החיצים הינם **דו - כיווניים** בשל כך שכל קשר **הינו דו - צדדי**; השרת מחליף נתונים עם כל לקוח, בנוסף השרת מבקש ומעדכן נתונים במסד נתוניו. גם כל לקוח מבקש ומעדכן נתונים במסד נתוניו המקומי.



תיאור הטכנולוגיה הרלוונטית

- **שפת תכנות** - הפרויקט נכתב במלואו בשפת הקוד Python, בעזרת הספריות המובנות אשר היא מציעה וספריות נוספות אשר נכתבו ע"י גורמים פרטיים. כמו כן, ישנו שימוש בתשתית בסיס נתונים טבלאית מסוג SQLite3.
- **מ"ה** - מערכות ההפעלה עליהן הפרויקט נועד לפעול הן windows10/11.
- **תקשורת** - התקשורת בפרויקט הינה בין הלקוח לשרת בלבד, השרת הינו מרובה משתתפים. התקשורת מבוצעת על בסיס ספריית Socket, פרוטוקול TCP, מוצפנת בשתי שכבות ודחוסה באמצעות ספריית gzip. על מנת להשתמש בפרוטוקול TCP בהתחברות הלקוח לשרת יש לבצע את תהליך ההתחברות של הפרוטוקול (Three-Way Handshake), מכיוון שפרוטוקול TCP הינו פרוטוקול מבוסס קישור. בחרתי להשתמש בפרוטוקול TCP מכיוון שהפרוטוקול מספק אמינות, וממקסם את הסיכויים לכך שההודעה תגיע ליעדה גם אם היו שיבושי תקשורת. למרות שזמן התגובה והביצוע בשימוש בפרוטוקול TCP גדול יותר ואיטי יותר מאשר בפרוטוקולים אחרים, בשירות מייל האמינות עולה על המהירות.
*איור הממחיש את תהליך ביסוס הקישור בין השרת ללקוח (Three way handshake):



- **תחומי עניין** - תחומי העניין המיושמים בפרויקט הם; רשתות - תקשורת מרובת משתתפים, אבטחת מידע, הגנת סייבר, ניהול והתקשרות עם מסדי נתונים, מחלקות, Threading, ו-gui, הכולל חווית משתמש קלה לתפעול, נוחה ומודרנית.

תיאור זרימת המידע במערכת

יכולות; התחברות הלקוח לשרת עצמו, הרשמה (register), התחברות משתמש (login), שינוי סיסמא (reset password), שליחת הודעות (send message), קבלת הודעות (retrieve/get message), התנתקות - כוללת גם עדכון סינונים וכדומה של המשתמש ויכולת חסימת משתמש (logout).

תרשימים; להלן תרשימים המציגים את כלל היכולות אשר מצוינות מעל, אלו כלל היכולות המערבות זרימת מידע במערכת השירות (בין שרת ללקוח). *חשוב לציין כי כל הודעה המתוארת בכל אחד מן התרשימים למעט בתרשים הראשון עוברת הצפנה ופענוח בעת מעבר מצד א' לצד ב' (מהלקוח לשרת ולהפך), בנוסף, נשלח בעבור הודעות אלו IV ייחודי באופן לא מוצפן (אין צורך). יתר על כן, לפני כל הודעה בכל תרשים נשלח באופן לא מוצפן (אין צורך) את אורך ההודעה ב-bytes העומדת להתקבל. זאת ועוד, אם ישנה תקלה במסד נתוני הלקוח, במסד נתוני השרת או בתקשורת בין השרת ללקוח, תוצג ללקוח הודעה מתאימה כי יש לסגור את התוכנה. לצורך ברירות והפשטת התרשימים תהליכים אלו לא מצוינים. *בתרשים הראשון מצויין לאיזה הודעה נעשית הצפנה.

- התחברות הלקוח לשרת (הפעלת האפליקציה);
תרשים המתאר זרימת המידע בין הלקוח לשרת והתהליכים השונים אשר קורים בעת התחברות הלקוח (לא המשתמש) לשרת (בנושאי אבטחה ורשתות).

מסד נתוני הלקוח (DB)

*סדר התהליך ללא הגעה למגבלת מספר הלקוחות המחוברים ממוספר לפי השלבים השונים

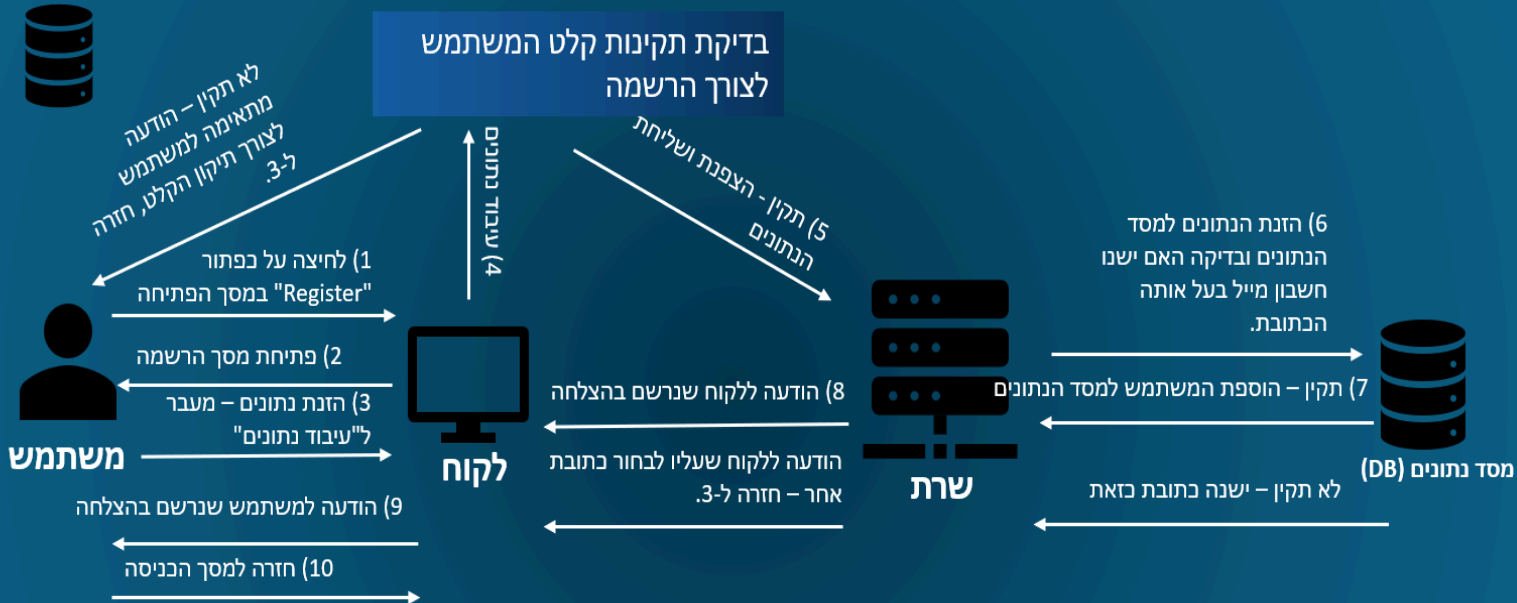


גיא משה טארנטו - DaVinci Mail

- הרשמה (register);
תרשים המתאר זרימת המידע בין הלקוח לשרת ועדכון הנתונים במסד נתוני השרת בעת ניסיון
הרשמה של משתמש למערכת.

מסד נתוני הלקוח (DB)

*סדר התהליך התקין ממוספר לפי השלבים השונים



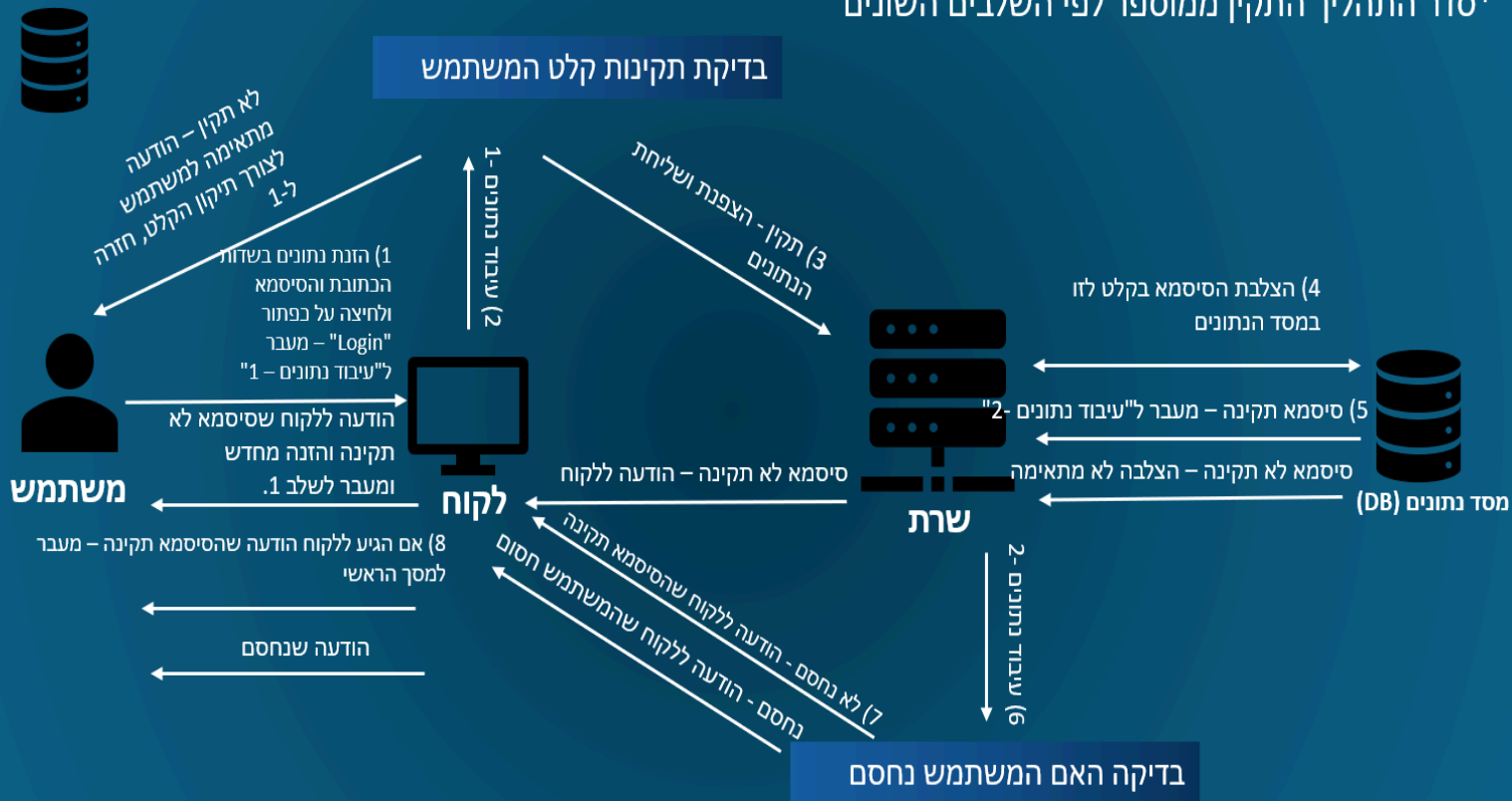
DaVinci Mail@

גיא משה טארנטו - DaVinci Mail

- התחברות משתמש (login);
תרשים המתאר זרימת המידע בין הלקוח לשרת, עדכון הנתונים במסד נתוני השרת והשינוי הגרפי בעת ניסיון התחברות משתמש לשרת.

מסד נתוני הלקוח (DB)

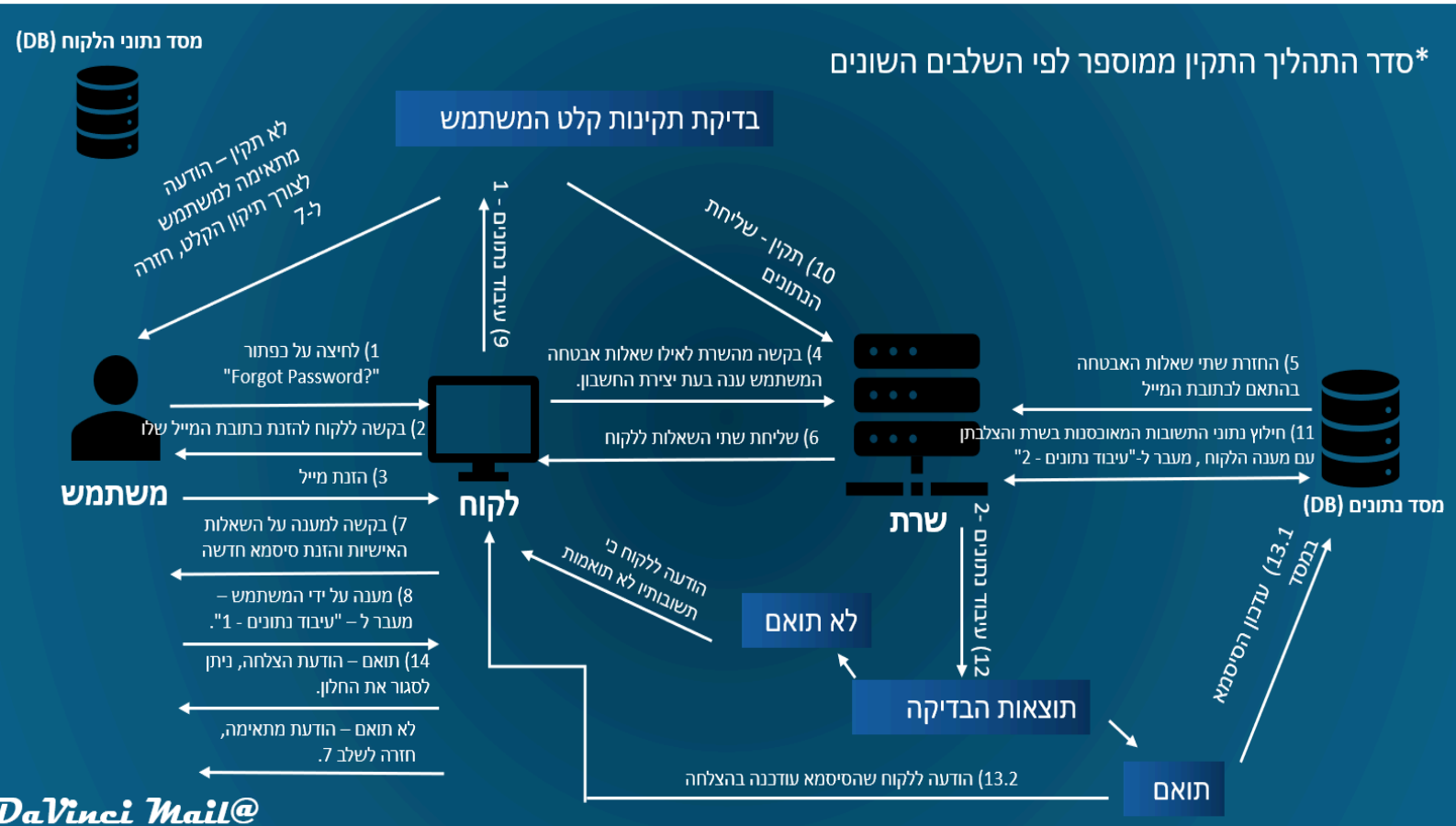
*סדר התהליך התקין ממוספר לפי השלבים השונים



DaVinci Mail@

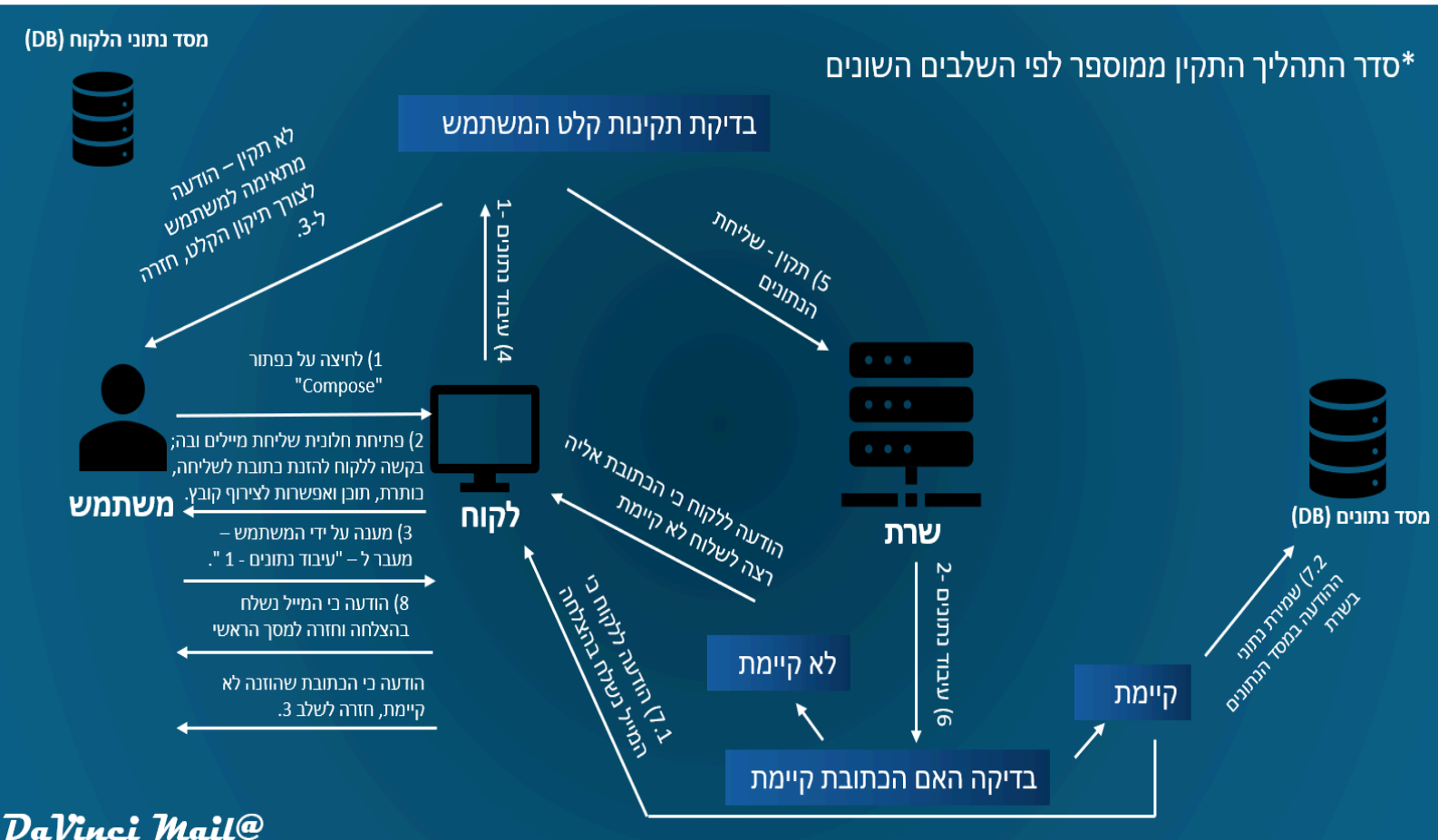
גיא משה טארנו - DaVinci Mail

- שינוי סיסמא (reset password);
תרשים המתאר זרימת המידע בין הלקוח לשרת, עדכון הנתונים במסד נתוני השרת והשינויים הגרפים בעת ניסיון שינוי סיסמא של משתמש קיים במערכת.



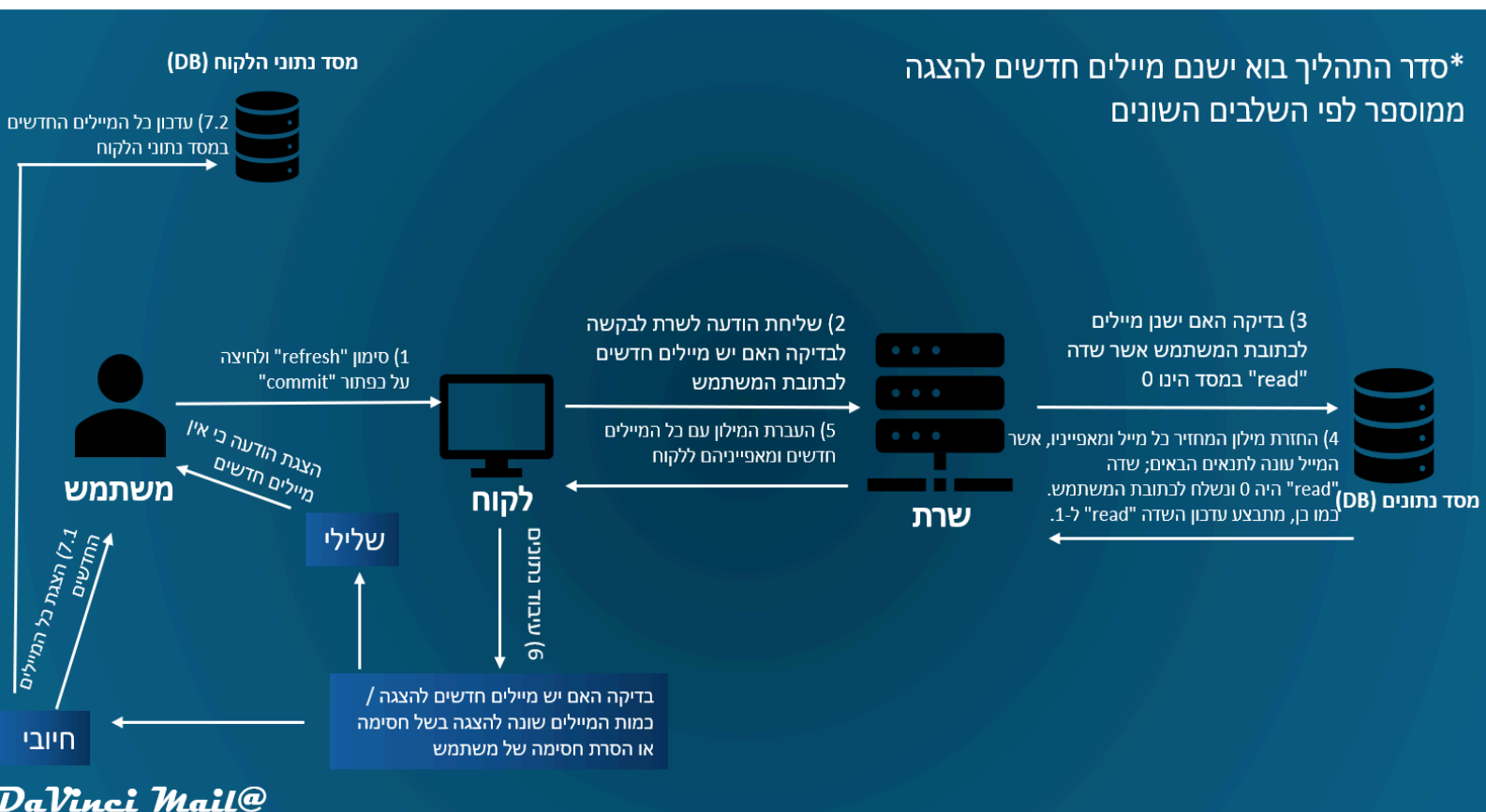
גיא משה טארנטו - DaVinci Mail

- שליחת מיילים (send message);
תרשים המתאר זרימת המידע בין הלקוח לשרת, עדכון הנתונים במסד נתוני השרת והשינויים הגרפים בעת ניסיון שליחת מייל ע"י משתמש מחובר למשתמש אחר.



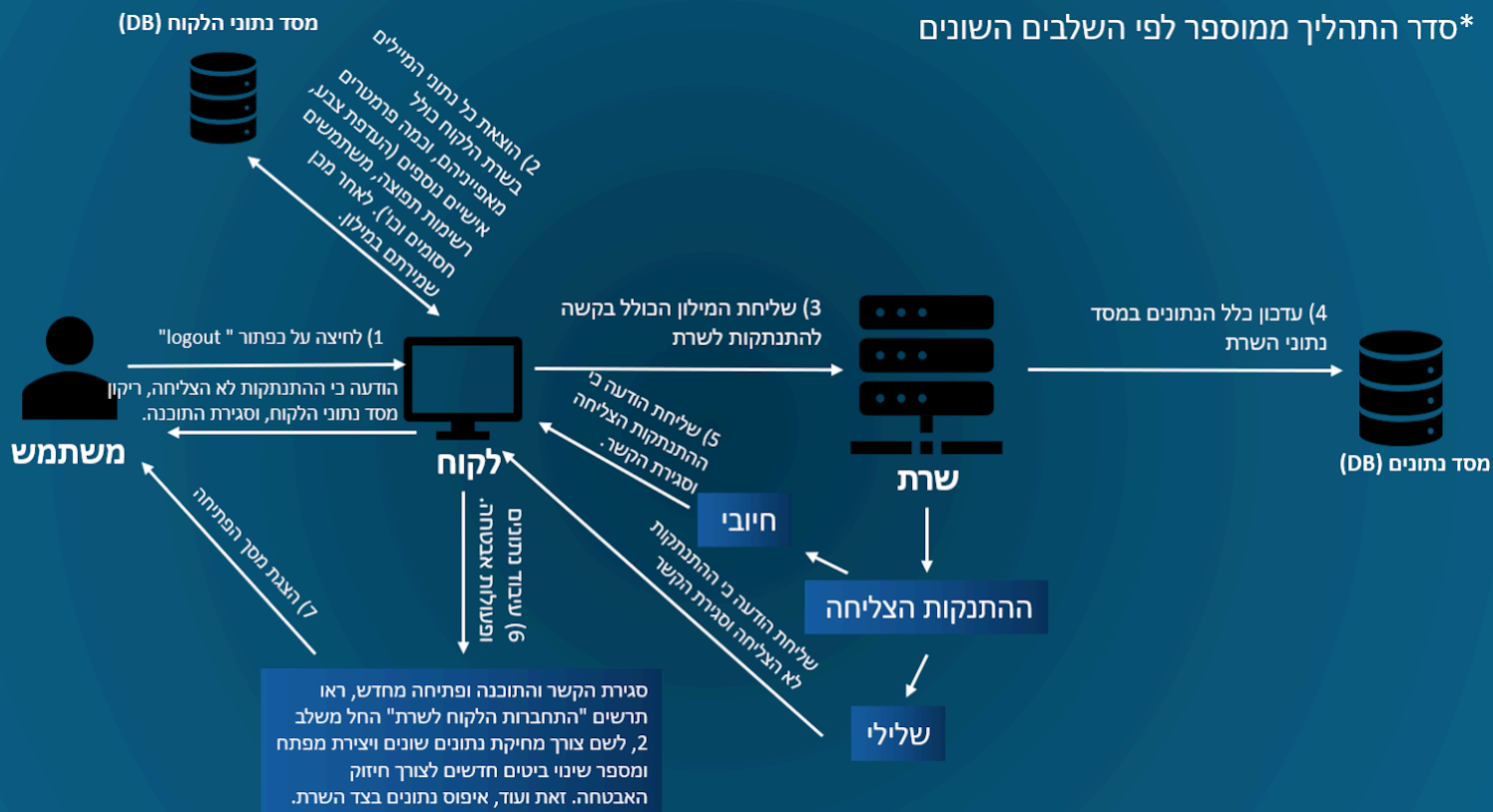
גיא משה טארנטו - DaVinci Mail

- קבלת מיילים חדשים - רענון (retrieve/get message); תרשים המתאר זרימת המידע בין הלקוח לשרת, עדכון הנתונים במסד נתוני הלקוח והשינויים הגרפים בעת בקשה של לקוח מחובר לרענון המיילים, כלומר בקשה לקבלת מיילים חדשים.



גיא משה טארנטו - DaVinci Mail

- התנתקות (logout); התנתקות המשתמש המחובר מהאפליקציה ומחיקת נתוניו במסד הנתונים הזמני בצד הלקוח. ההתנתקות כוללת גם עדכון סינונים שמשתמש ביצע למיילים (השמה ב-label, סימון כחשוב וכדומה), עדכון אודות חסימות שהמשתמש ביצע למשתמשים אחרים, רשימות תפוצה ועוד...
תרשים המתאר זרימת המידע בין הלקוח לשרת, עדכון הנתונים במסד נתוני השרת, ריקון הנתונים במסד נתוני הלקוח, השינויים הגרפיים והשינויים האבטחתיים בעת בקשה של לקוח מחובר להתנתקות מבלי לסגור את האפליקציה.



גיא משה טארנטו - DaVinci Mail

- סגירה האפליקציה על ידי לחיצה על "X":



תרשים המתאר זרימת המידע בין הלקוח לשרת, עדכון נתונים במסד נתוני השרת והשינויים הגרפים בעת לחיצה של לקוח **מחובר** על כפתור "X", כלומר סגירת האפליקציה.



DaVinci Mail@

תרשים סגירת האפליקציה ע"י הכפתור "X", כאשר אין עוד משתמש מחובר.



DaVinci Mail@

תיאור האלגוריתמים מרכזיים בפרויקט

- **ניסוח וניתוח של הבעיה האלגוריתמית** - בפרויקט אותו כתבתי ישנן מספר בעיות אלגוריתמיות שונות; **ראשית**, כיצד יש לאחסן את סיסמאות המשתמשים על מנת שלא יהיה חשופות לכל במסד הנתונים, למשל אם גורם חיצוני ולא רצוי מצליח לשים עליו את ידיו, או גורם עוין פנימי "בחברת השירות". האחסון אמור להיות בטוח כך שמצד אחד לא יהיה ניתן לגלות איזו סיסמא כל משתמש הגדיר ומצד שני יהיה ניתן להשתמש בשדה הסיסמא של כל משתמש במסד נתוני השרת לצורך התחברות הלקוח בעתיד (וידוי כי הערך המאובטח וערך הלא מאובטח אותו הלקוח הזין זהים מבחינת תוכן הסיסמא). **שנית**, כיצד לאבטח את התקשורת בין השרת ללקוח, כדי שהשירות אותו כתבתי יהיה מוצפן ומאובטח ברמה גבוהה. הכוונה הינה, כיצד למנוע ביעילות התקפת "man in the middle", שהינה מצב בו גורם שלישי מסניף את התקשורת בין הלקוח לשרת ומנסה לדלות מידע חסוי באמצעות זיהוי דפוסים, ניסיון לפיצוח המידע המוצפן וכדומה. לכן יש להצפין את המידע בצורה דו-כיוונית, כלומר להצפינו לפני שליחתו ברשת החיצונית (מהלקוח לשרת או להפך) ולפענחו בעת קבלה (בשרת או בלקוח) לאותו ממידע בדיוק. **שלישית**, (מתקשר לבעיה השנייה) כיצד יש לשלוח את ההודעות בצורה בטוחה (ללא שיבוש המידע) ויעילה, בעת העברת המידע מהלקוח לשרת או להפך. ההודעה הפשוטה, ההודעה משפת פרוטוקול התקשורת (plaintext data) עוברת עיבוד שונים רבים ועלינו לשמור על אי שיבוש המידע שבה.
- **רביעית**, כיצד למנוע מתקפת DOS, מתקפת מניעת שירות אשר בה גורם עוין ינסה לשלוח בקשות רבות במקביל על מנת להקריסו ולא לאפשר לו לענות לבקשות רגילות של משתמשים. כמו כן, ביצעתי גם מעין מניעה מובנית למתקפת DDOS, עליה ארחיב בהמשך. **חמישית**, כיצד לבנות ולנהל את כלל נתוני הלקוחות והמייילים ביעילות, השיטה הנכונה והיעילה ביותר העונה על דרישות הפרויקט. **שישית**, טיפול בקשות הלקוח (עקב פעולות המשתמש) בצד השרת; השירות מציע למשתמש לבצע פעולות רבות ומגוונות, יש לחשוב על הדרך הטובה והיעילה ביותר לטיפול בפעולות המשתמש שמתבטאות בבקשות הלקוח בצד השרת. **שביעית**, ה-Handshake של התוכנה, מהו אלגוריתם ההתחברות של לקוח לשרת השירות. באלו פעולות יש לנקוט מבחינת ביסוס אבטחת תעבורת התקשורת?

● תיאור אלגוריתמים קיימים לפתרון הבעיה וסקירת הפיתרון הנבחר -

- **אלגוריתם ראשון - גיבוב סיסמאות במסד הנתונים**; ראשית, כאשר משתמש נרשם לשירות ניתן באופן תיאורטי להצפין את הסיסמא אותה הגדיר לפני הזנתה למסד הנתונים. אומנם, הצפנה הינה תהליך **דו-כיווני** (ניתן לקבל מהערך המוצפן את הערך המקורי), לכן באופן תיאורטי כל תוקף שיצליח לשים את ידיו על מסד הנתונים או חלקו יוכל לפצח את מנגנון ההצפנה שבוצעה לסיסמאות המשתמשים. באמצעות הפיצוח שעשה יוכל התוקף לפענח את כלל סיסמאות המשתמשים, עובדה זו יכולה להוביל לבעיות אבטחת מידע חמורות ביותר ופגיעה קשה במשתמשים. לכן, כיום כמעט כל אחראי לאבטחת מידע המכבד את עצמו ישתמש בגיבוב (פונקציית Hash כזו או אחרת). בניגוד להצפנה, גיבוב הינו תהליך **חד-כיווני**, לא ניתן לפענח את המידע לאחר העברתו בתהליך גיבוב לערכו הראשוני, הערך "הנקי". גם שימוש בגיבוב צריך להעשות בחכמה - אם נשתמש באותה פונקציית גיבוב לכל סיסמא, לסיסמאות זהות יהיה אותו ערך לאחר הגיבוב. באמצעות כך ניתן לנסות ולפענח את הסיסמא לפני הגיבוב, בכך שהתוקף יבצע גיבוב לסיסמאות נפוצות עם פונקציות גיבוב נפוצות. כעת בידי התוקף יהיו טבלאות של סיסמאות נפוצות מגובבות, התוקף יצליב את הטבלאות שהכין עם הערכים המגובבים של סיסמאות משתמשי התוכנה המאוחסנות במסד הנתונים, וכך יוכל למצוא את הערך הראשוני של הסיסמא, ערכה לפני העברתה גיבוב - פגיעה חמורה ביותר באבטחת מידע.
- **דוגמה למסד נתונים בו מאוחסנות סיסמאות משתמשים לאחר העברתן תהליך גיבוב** באמצעות פונקציית גיבוב פשוטה (חשוב לשים לב כי למשתמשים 1254 ו-1256 יש אותו ערך בעמודת הסיסמא, כלומר הם בעלי אותה סיסמא, בשל כך שהגיבוב מחזיר ערך זהה לכל ערך זהה. כמו כן, ניתן יהיה להבין כי זוהי סיסמא נפוצה).

password	user_id
ab4f63f9ac65152575886860dde480a1	1253
21b72c0b7adc5c7b4a50ffcb90d92dd6	1254
47ad898a379c3dad10b4812eba843601	1255
21b72c0b7adc5c7b4a50ffcb90d92dd6	1256
5b9a8069d33fe9812dc8310ebff0a315	1257

זאת ועוד, ישנן עוד מתקפות ודרכים רבות לעקוף את תהליך הגיבוב כמו מתקפת Rainbow table שפעולות בשיטות רבות ומגוונות על פי זיהוי דפוסים שונים. לכן יש להשתמש בפונקציות גיבוב מתקדמות על מנת להקשות ואף כמעט ולמנוע לחלוטין את היכולת להפוך את פונקציית הגיבוב שהינה אמורה להיות באופן תיאורטי **חד-כיוונית** לסוג של פונקציה **דו-כיוונית** במציאות. פונקציות גיבוב מתקדמות משתמשות ב"מלח" (salt), אשר נועד "לתבל" את הערכים המגובבים ולגרום לכך שגם שני סיסמאות זהות יהיו בעלות ערך מגובב שונה (כך הדוגמה למתקפה עליה הסברתי קודם לא תעבוד). דגש חשוב הוא ש"המלח" (salt) יהיה רנדומלי ולא קצר, כך יהיה קשה יותר לנסות ולהנדס לאחר את הערך המגובב של הסיסמא. בעצם המלח משתלב עם פעולת הגיבוב הרגילה, כמעייין תוסף, אומנם התוסף שונה בכל פעם, שונה לכל משתמש, לכן יגרום לתוצאה סופית שונה - ערך מגובב שונה. ניתן להוסיף תיבול נוסף "פלפל" (pepper), שהינו תוסף נוסף, שזהה לכלל המשתמשים. מומלץ "לקבור אותו" כלומר לאכסנו במקום נפרד, או במקום לא צפוי וחסוי, כדי שלא יהיה קל למצוא אותו. ניתן להעביר את כל הסיסמאות עוד שכבה של גיבוב באמצעות ה"פלפל" וכך גם אם תוקף השיג את כלל הסיסמאות המגובבות וה-salt שלהם הוא עדיין לא יוכל להנדסן לאחר. זאת ועוד, ניתן אף לבצע גיבוב כמה וכמה פעמים לשם כך שאומנם חזרה על הגיבוב יקח מעט יותר זמן (כמה מילי שניות יותר), אך לתוקף שמנסה מיליוני קומבינציות שונות גם כמה מילי שניות הן מאוד משמעותיות. שילוב שלושת הדברים הללו, "מלח" (salt), "פלפל" (pepper) וביצוע גיבוב כמה וכמה פעמים יצור לנו יכולת גיבוב חזקה ביותר אשר תהיה קשה לפיצוח, תחזק את אבטחת המידע בשירותים שונים ותגן על פרטיות המשתמשים. לכן בפרויקט שלי בחרתי לא להמציא את הגלגל מחדש, אלא להעזר בפונקציית גיבוב מוכחת וחזקה אשר תעשה את עבודתה ברמה הגבוהה ביותר ותעניק פרטיות למשתמשים שלי. בחרתי להשתמש בספריית הגיבוב ב-python של האלגוריתם של Argon2 שהינו אלגוריתם מוכר, חזק שאף זכה בתחרויות שונות בתחום גיבוב סיסמאות.

□ **אלגוריתם שני - אבטחת תעבורת התקשורת בין השרת והלקוח ולהפך**; כפי שציינתי בחלקים הקודמים, בפרויקט רציתי למנוע מתקפת man in the middle שבמהלכה גורם שלישי מסניף את התקשורת בין השרת והלקוח ומנסה לדלות ממנה נתונים. ניתן לאבטח את התקשורת בין השרת ללקוח בדרכים רבות, שונות ומגוונות. כמו כן, דרכי האבטחה גם תלויות בדרך התקשורת בין השרת ללקוח, sockets על בסיס TCP כמו בפרויקט שלי או למשל באמצעות פרוטוקול HTTP. האבטחה צריכה להיות בצורה הבאה; הצפנת המידע לפני השליחה, קבלת המידע בצד השני ופענוחו, כלומר המידע עובר תהליך **דו-כיווני**, הצפנה ופענוח. לשם כך, צריכה להיות יכולת גם לשרת וגם ללקוח להצפין מידע בדרך שלצד השני תהיה יכולת לפענחו וכן יכולת לפענח את המידע הנשלח ע"י הצד השני. הצפנה הינה בעצם "שיבוש" המידע בדרך הפיכה, כך שמי שלא יודע את נוסחאת השיבוש לא יוכל להנדסו לאחר ומי שכן, יוכל להנדסו לאחר ולקבל את המידע המקורי. בבית הספר הוכונו לממש הצפנה באמצעות אלגוריתם RSA (פירוט נרחב בהמשך), שהינו אלגוריתם **אסימטרי** מוכר וידוע להצפנה ופענוח מידע באמצעות שני מפתחות - פרטי וציבורי. הוקדש זמן ללימוד הבסיס של האלגוריתם

והוכונו לממש בפרויקט RSA באמצעות כמה קודים בסיסיים שסופקו ע"י אחד המורים. יחד עם זאת, קראתי כי בדרך כלל לא נעשה שימוש ב-RSA בלבד בהעברת מידע באופן שוטף, בשל איטיות האלגוריתם והגבלת גודל המידע שניתן להצפין בהצפנה אחת. קראתי כי הפרוטוקול הינו מצוין להחלפת מפתח **סימטרי** בין שרת ללקוח (שליחת המפתח הסימטרי מצד הלקוח לצד השרת באופן מוצפן), ולאחר מכן הצפנת המידע תעשה רק באמצעות המפתח הסימטרי. בחרתי גם להתייעץ עם דודי ניר, המבין בתחום, ודודי המליץ לי לפעול כפי שקראתי ברשת - לבצע שימוש **היברידי** ב-RSA ובהצפנה סימטרית; הוא הציע לי להשתמש ב-RSA וב-AES בגרסאות (bit-256) שהינה שיטת אבטחה סימטרית מוכרת וידועה (עליה אפרט גם כן בהמשך). לכן ב-"Handshake" של השירות שהינו אלגוריתם בפני עצמו (האלגוריתם השביעי) מתבצעת העברת מוצפנת של מפתח **סימטרי** מצד הלקוח לצד השרת באמצעות שימוש ב-RSA. כמו כן, רציתי להוסיף שכבה אבטחה ייחודית נוספת לשירות (למעט המפתח הסימטרי) והיא הזזת מעגלית של הביטים בכל byte במידע העומד להישלח, עוד לפני השימוש במפתח הסימטרי (פירוט בהמשך). לכל לקוח יש מפתח סימטרי שונה ומספר הביטים להזזה מוגרל באופן רנדומלי (המספר הינו בין 1 ל-7, הרי יש שמונה ביטים בבית), שני אלו נקבעים בכל מחדש בכל הרצת קוד הלקוח, אפילו בעת התנתקות מבלי לסגור את התוכנה. עובדות אלו מגבירות את אבטחת המידע בשירות ואמורות להקשות על כל תוקף לנסות ולפצח את מנגנון הפענוח. יתר על כן, המימוש ל-RSA הינו קוד עצמאי אשר לא נתמך ע"י ספרייה חיצונית ומממש RSA מאפס (from scratch), התבססתי על קוד שאחד ממורי המגמה שלח לנו והתאמתי אותו לקוד שלי. אולם מימוש ה-AES אכן נעשה ע"י שימוש בספרייה חיצונית, אין ביכולתי להמציא את הגלגל מחדש ולדמות פרוטוקול AES ברמה גבוהה במסגרת הפרויקט, לכן רציתי ששכבת האבטחה הראשית והעיקרית בפרויקט תהיה ברמה הגבוהה ביותר. יתר על כן, הצפנת AES הינה יעילה מאוד להצפנת קבצים גדולים בשל המהירות והיעילות החשובות של אלגוריתם ההצפנה, דבר החשוב לשירות שלי.

פירוט אודות דרך ההצפנה והפענוח באלגוריתם RSA פשוט:

שלבי מציאת המפתח הציבורי (Public key);

- 1) מגרילים שני מספרים ראשוניים p ו- q הכרחי שהם יהיו גדולים כי כך קשה יותר לפענח את ההצפנה. כדי למצוא מספרים ראשוניים בקלות יחסית ניתן להשתמש באלגוריתם מילר-רבין.
- 2) נסמן $n = p \cdot q$ ונחשבו;
- 3) מחשבים עבור הפרמטרים את הביטוי הבא באמצעות פונקציית אוילר;

$$\phi(n) = (p - 1)(q - 1)$$

- 4) את המפתח הציבורי נסמן כ- e , שהינו מספר ראשוני טבעי בין 3 לערך פונקציית אוילר שחישבנו בעבור n , אומנם לשם שיפור האבטחה מומלץ תחילה לבחור כ- e את 65537. לאחר שבחרנו את המפתח הציבורי e , נבדוק האם ה- $\gcd$ (מחלק משותף מקסימלי) שלו עם ערך פונקציית אוילר בעבור n הוא 1. בדיקה זו מספקת לנו מידע האם המספרים הם Coprime, כלומר ראשוניים ביחס אחד לשני, אם כן נמשיך הלאה בתהליך, ובמידה ולא נחשב e חדש (גדול יותר מ-65537 וראשוני) עד למציאת מתאים. את המפתח הציבורי נרשום בצורה הבאה; (e, n) כאשר e הינו המפתח הציבורי ו- n הוא (כח החישוב) תוצאת פונקציית אוילר.
- 5) נצפין באמצעות המפתח הציבורי באופן הבא (m) הינו ערך להצפנה;

$$m^{e \pmod n} = c$$

מפתח פרטי (Private key);

(1) את המפתח הפרטי נסמן ב-d, בשל כך שאנו עובדים עם מספרים ראשוניים נוכל לדעת כי אם d שונה מ-e וראשוני לו, נוכל להשתמש במשפט אוילר ולקבוע כי:

$$e * d = 1 \bmod \varphi(n)$$

(2) פענוח עם המפתח הפרטי באופן הבא (m הינו הערך שהוצפן);

$$c^{d \bmod n} = m$$

פירוט אודות דרך ההצפנה והפענוח ב-AES (גרסת 256 bits);

ישנן מספר גרסאות למפתח מהסוג הנ"ל; 128 ביטים, 192 ביטים ו-256 ביטים בה אני משתמש. ככל שמספר הביטים עולה כך אורך המפתח גדל וגם מספר סבבי ההצפנה הינו המירבי ביותר - 14 ולכן ניסיון פיצוח ע"י צד שלישי עדין יהיה מאתגר יותר וכמעט בלתי אפשרי. ההצפנה לא נעשית בבת אחת, אלא המידע מחולק לבלוקים, כל בלוק מוצפן באופן נפרד ועובר תהליכים שונים כחלק מההצפנה. התהליכים חוזרים על עצמם בגרסאת 256 bits ב-14 סבבי הצפנה שונים (כפי שצינתי) על מנת למקסם את ההצפנה כמה שניתן. אולם, הצפנה זו אינה מספקת, יש להוסיף לכל הודעה IV ייחודי הנוצר עבורה, ה-IV הוא כמו "salt" במנגנון גיבוב. אם נבצע באמצעות אותו מפתח AES הצפנה לאותו מידע נקבל אותו צופן עבור המידע, לכן יש להשתמש עבור כל הודעה ב-IV שונה. ה-IV הייחודי גורם לכך שעבור אותו מידע או מידע דומה ועבור אותו מפתח AES כל פעם יתקבל מידע מוצפן שונה. אני מבצע הצפנה באמצעות מצב CBC של הצפנת AES - אשר בה כל הצפנה של כל בלוק תלויה בערך המוצפן של הקודם, ההצפנה של הבלוק הראשון תלויה גם ב-IV הייחודי להודעה, לכן בפועל כל הבלוקים בכל הודעה מושפעים מה-IV (הרי כולם מושפעים מהבלוק הראשון). לפני כל הודעה המועברת בין השרת ללקוח נשלח אורך ההודעה בבתים ובנוסף גם נשלח IV ייחודי לכל הודעה באורך 16 בתים, לכן כך מובטח שכל הצפנה באמצעות מפתח AES שבו הלקוח הנוכחי משתמש תהיה שונה לחלוטין מהשנייה. לסיכום, הצפנת ה-AES הינה ההצפנה העיקרית בשירות, היא אמינה וחזקה והשימוש ב-IV ייחודי לכל הודעה שומר על צופן ייחודי לכל הודעה.

פירוט אודות ה"פלפל" שהוספת להצפנת ה-AES הזאת הביטים בכל byte של מידע

מוצפן: למרות השימוש במפתח AES בדרגה הגבוהה ביותר 256 הביטים, רציתי להוסיף שכבה אבטחה מקורית ונוספת לתעבורת התקשורת של השירות אשר תשבש עוד את המידע. בדומה ל"פלפל" של מנגנון הגיבוב המשנה את התוצאה המגובבת באופן מערכתי (מלבד ה"מלח" שבגזרת AES הינו בדמות IV), רציתי להוסיף גם קצת "תיבול" ל-AES ונגיעה מקורית שלי באבטחת הפרויקט. עקב כך הוספתי שלפני הצפנת כל הודעה באמצעות מפתח ה-AES וה-IV ישנה פונקציה העוברת על כל byte ו-byte בתוכן ההודעה ומזיזה את הביטים שלו בצורה מעגלית בין 1 ל-7 מקומות (יש להזכיר כי מספר ההזזות נקבע באופן רנדומלי לכל לקוח, בין 1 ל-7 מכיוון שיש שמונה ביטים ב-byte אחד). כך ישנה שכבת הגנה נוספת ומקורית שלי, אם תוקף ינסה להסניף את המידע ולנסות ולפצחו באמצעות אלגוריתמים שונים, שכבה זו תקשה עליו מכיוון שהיא משבשת את כל המידע המוצפן, ללא ידיעה כמה מקומות כל ביט הוזז וללא ידיעה שהזזה זו בכלל נעשתה יהיה קשה לנסות ולפענח את המידע ע"י צד שלישי עדין.

אלגוריתם שלישי - שליחת ההודעה בדרך שתוכנה לא השתבש ובצורה יעילה; □

באמצעות שימוש בשליחת הודעות ב-Sockets ב-python כמו שאני מבצע בפרויקט לא ניתן לשלוח מבני מידע מורכבים כמו מילונים שהם מוגדרים כטיפוסי מידע מרוכבים של שפת הקוד, בהם אני משתמש בפרוטוקול התקשורת שלי באופן מרכזי ומוחלט. לכן יש להמיר את העצם של שפת python לטיפוס מסוג bytes (שניתן לשלוח דרך sockets) או למידע מטיפוס אשר ניתן לעשות לו encode, כלומר המרה ל-bytes ולשלוח אותו דרך sockets. לפי המידע אשר צברתי ממחקר באינטרנט, בין היתר מהקישור המצורף

בהמשך, הבנתי כי לפרויקט בסדר גודל שלי ומהסוג שלי אין כל כך הבדל באיזו דרך אשתמש לשם המרת מבני המידע המורכבים מטיפוסי python למידע אשר אוכל בסופו של דבר לשלוח דרך sockets. הבנתי גם כי שתי האפשרויות המומלצות לביצוע המרת טיפוס ה-python ללא פגיעה בו ועם יכולת המרה לbytes (או שכבר יהיה ב-bytes) הן באמצעות ספריית Json או באמצעות ספריית Pickle. הנה טבלת השוואה בין הספריות הנ"ל:

Feature	JSON	Pickle
Readability	Human-readable (text)	Not human-readable (binary)
Language Support	Cross-language	Python-specific
Object Support	Basic data types only	Almost any Python object
Security	Generally safer for untrusted data	Major security risk with untrusted data
Efficiency	Can be less efficient for complex objects	Potentially more efficient for complex objects
Complexity	Simpler for basic data	Simpler for complex Python objects
Use Cases	Web APIs, configuration files, data exchange between different systems	Python-specific object persistence, inter-process communication between Python processes

לבסוף בחרתי בספריית Json בשל כך שהיא מתאימה להעברת מידע בין מערכות שונות, כלומר אם ארצה למשל לפתח את הפרויקט בעתיד ולהוסיף גם חלק המתוכנת ב-WEB לא אוכל לעשות זאת באמצעות pickle, לכן פסלתי אותה ובחרתי ב-Json. על מנת לשלוח את המידע ביעילות, חוץ מהעברת המידע המתוארת בתרשים העברת המידע הראשון, הכוללת את החלפת המפתחות ומספר הביטים להזזה בין הלקוח לשרת ולהפך, לפני כל הודעה נשלחת באופן לא מאובטח (אין צורך) את אורכה על מנת שהשרת או הלקוח יצפו כל פעם למספר ה-bytes המדויק העומד להישלח. זאת ועוד, על מנת להעביר את המידע ביעילות למעט הודעות אורך ההודעה העומדת להתקבל, השתמשתי בספריית gzip על מנת לדחוס את המידע כדי, להקטין את גודלו לפני השליחה. ישנן מספר ספריות לדחיסת קבצים, אומנם חלקן מתמקדות בדחיסת מספר קבצים, לעומת זאת ספריית gzip מתמקדת בדחיסת קובץ אחד ויחיד, השימוש אליו אני זקוק לכן בחרתי בה.

□ אלגוריתם רביעי- מניעת מתקפת DOS ומניעת מתקפת DDOS; מתקפת DOS, אשר

כפי שצינתי לפני כן, הינה מתקפת מניעת שירות בה גורם עוין מנסה לגרום להצפת המערכת בבקשות עד אשר לא תוכל לטפל בבקשות שאר הלקוחות ובסוף גם אף להביא לקריסתה. ניתן לדמות זאת למצב שבו ישנו ילד בכיתה אשר קורא למורה שוב ושוב ללא הפסקה, לוקח את תשומת הלב של המורה ולא מאפשר לשאר הילדים לתקשר איתו. כדי למנוע מתקפת DDOS ו-DOS קראתי כי יש לבצע מעקב אחרי כתובות ip וכמות הבקשות שלהם, חלק ממליצים לבצע שימוש ב-Firewalls. בחירתי בהתמודדות עם מתקפת DOS ו-DDOS הייתה בעיקרה להתמודד עם כך שיהיה עומס על השרת ומכך שמשמש ינסה להציפו במספר רב של בקשות. אולם, אחד מהפיצ'רים שהוספתי הינו רשימת תפוצה, כל הודעה מוצפנת, נדחסת ונשלחת **בנפרד** לכל אחת מהכתובות ברשימה (על מנת לא לשלוח הודעה גדולה מאוד במקרה של כתובות רבות באותו רשימה, או מעט כתובות אבל שצורך להודעה קבצים), לכן במצב זה השרת יקבל מאותה כתובת ip כמה הודעות אחת אחרי השנייה ברצף (יש הפרש בזמני הקבלה, לא

מתבצע באותו הרגע, פירוט בהמשך), לכן אם אממש מעקב אחרי כתובות ip ישנו סיכוי גבוהה מאוד שמשתמשים ייחסמו לשווא. לכן רציתי לפעול בדרך מעט מקורית שלא נתקלתי בה בחיפוש ברשת, ובהתייעצות עם אחד ממורי המגמה נאמר לי שהינה דרך טובה להפחתת הסיכוי למתקפת DDOS/ DOS. הדרך המקורית אותה מימשתי הינה בעיקרה ביצוע דיליי מובנה ומכוון בשליחת הודעות; כל הודעה אשר נשלחת לשרת תשלח בין 0.1 ל-0.4 שניות לאחר הקשת המשתמש על הכפתור הגורם לכך באפליקציה, בלי שאר העיכובים הפוטנציאליים ברשת, בהתחשב שמהירות הטיפול של השרת הינה כמה מילישניות לבקשה, דיליי זה אמור להבטיח כי השרת לא יקרוס מעומס בקשות, גם אם יהיה אדם אשר ישלח הודעות בלתי פוסקות ממספר משתמשים ואף יפעיל בוט, השרת לא יקבל את כלל ההודעות ברגע אחד, ויהיה לו מספיק זמן לטפל בכל הודעה בנפרד. זאת ועוד, דבר נוסף שביצעתי הינו הגבלת כמות הלקוחות אשר יכולים להתחבר לשרת, שילוב של הגבלה זו עם הדיליי אמור לאפשר מניעת מתקפת DOS ובמיוחד DDOS כי כמות הלקוחות העוינים אשר יכולים להתחבר מוגבלת באופן משמעותי וכך לא יהיה מספר רב של לקוחות שיוכל להציף את השרת בבקשות עד לקריסתו.

□ אלגוריתם חמישי - ניהול כלל נתוני השירות - בבית הספר למדנו אודות ניהול נתונים

בהתבסס על שימוש SQLite ובניית מסד נתונים טבלאי, דבר זה הינו חלק מדרישות הפרויקט ולכן מימשתי זאת בפרויקט. באמצעות השימוש הטבלאי ב-SQLite ניתן לאחסן, באופן יעיל, אמין, ברור וקל לניטור את כלל נתוני המערכת (משתמשים ומיילים). ניתן לשלוף את הנתונים בטבלאות השונות במהירות וביעילות עם שאילתות (queries) המוגדרות מראש. זאת ועוד, במהלך הכנת הפרויקט עלה לי הרעיון לבנות מסד נתונים זמני (לזמן החיבור) בצד הלקוח על מנת לשמור את כלל הסינונים והפילטורים שמשמש מבצע למיילים (סימון כחשוב, מחיקה וכו'), מימשתי את רעיון זה ובניתי מנגנון יעיל, קל לתחזוק ולהוספת שינויים לסינון ופילטור המיילים השונים לפי העדפות הייחודיות של המשתמש. כפי שצוין בתרשימי העברת המידע בעת ההתנתקות, המידע במסד זה מרוקן ונשלח לשרת בעת התנתקות המשתמש.

□ אלגוריתם שישי - טיפול בקשות הלקוח בצד השרת; כל בקשת לקוח בנויה ממילון

שהמפתח הראשון בו הינו סוג הפעולה (פירוט בהמשך), לכן ישנה פונקציה בשרת המקבלת את המילון עם בקשת הלקוח והתוכן הרלוונטי. בהתאם לפעולה במפתח הראשון במילון היא תפנה את המידע לפונקציה המטפלת בפעולה, כלומר ניתן להגיד שהפונקציה בעצם מתפקדת כמתווכת, כמו שאדם מגדיר למתווך איזה בית הוא מחפש, המילון מגדיר באמצעות הפעולה במפתח הראשון שלו איזה פעולה הלקוח ביקש, לכן הפונקציה תמצא לו את "הבית" - איזו פעולה מטפלת בבקשה.

□ אלגוריתם שביעי - 'Handshake' השירות - על מנת לבסס את הקשר בין השרת

ללקוח באופן תקין ומאובטח, בעת התחברות לשרת, השרת שולח ללקוח מילון ובו מפתח אחד, שערכו הינו או True או False, אם התשובה שלילית - השרת עמוס, תוצג הודעה למשתמש ותסגר התוכנה. אם התשובה חיובית, השרת ישלח מייד לאחר מכן את אורך המפתח הציבורי שלו ואת המפתח עצמו, הלקוח ישמור אותו במכשיר שלו, הלקוח יצפין את מפתח ה-AES שלו באמצעות המפתח הציבורי של השרת ואת מספר הביטים שיש להזיז בכל בית באמצעות חישוב מתמטי קצר בהתאם לכתובת ה-IP של השרת (פירוט בהמשך) ושולח אותם לשרת. השרת יקבלם ומעתה והלאה ניתן לבצע תקשורת רגילה באמצעות מפתח ה-AES שנמצא בידי שני הצדדים וה-IV הייחודי לכל הודעה באופן שוטף. במקרה של תקלה מסוג אי הצלחה בשמירת המפתח הציבורי במחשב הלקוח, תהיה הודעה כי יש לסגור את התוכנה. במקרה של תקלת תקשורת, תהיה הודעה על כך, המשתמש יוכל לסגור את התוכנה או להמתין 15 שניות ולנסות להתחבר מחדש.

גיא משה טארנטו - DaVinci Mail

- הפנייה למקורות רלוונטיים בשביל אלגוריתמים קיימים לפתרון הבעיה וסקירת הפיתרון הנבחר עבור כל אחד מן ששת האלגוריתמים:
 - ★ [אלגוריתם ראשון - גיבוב סיסמאות במסד הנתונים - מאמר מקיף בנושא גיבוב של האתר "vaadata"](#)
 - ★ [אלגוריתם שני - אבטחת תעבורת התקשורת בין השרת הלקוח ולהפך - מאמר מקיף בנושא אבטחה סימטרית ואסימטרית של האתר "preyproject", מאמר kiteworks אודות דרך הפעולה של מפתח AES בדגש על גרסת 256 ביטים בה נעשה שימוש בשירות.](#)
 - ★ [אלגוריתם שלישי - שליחת ההודעה בדרך שתוכנה לא השתבש ובצורה יעילה - מאמר מקיף בנושא "Serialize Your Data With Python" מטעם אתר realpython.](#)
 - ★ [אלגוריתם רביעי - מניעת מתקפת DOS ומניעת מתקפת DDOS - מאמר בנושא מתקפת DOS מטעם האתר CloudFlare, מאמר בנושא מניעת מתקפת DDos מטעם האתר SecurityScorecard.](#)
 - ★ [אלגוריתם חמישי - ניהול כלל נתוני השירות - עיקר המידע שבו נעזרתי הינו המודל של המורה \(כמו מעין google classroom\) שהינו מותנה גישה לתלמידים לכן לא אוכל לצרף קישור.](#)
 - ★ [אלגוריתמים מספר 6 ו-7 הינם בפיתוח עצמי.](#)

תיאור סביבת הפיתוח

- פרוט כלי הפיתוח הדרושים לפיתוח - כלי הפיתוח הדרושים הינם שפת הקוד python על ספריותיה השונות (כולל מסד נתונים SQLite3). בנוסף הייתי זקוק לתוכנת wireshark לבדיקות שונות ולתוכנת DB browser for SQLite לשם ביצוע בדיקות בנוגע לתקינות מסד הנתונים.
[Wireshark](#) קישור להורדת
[DB browser](#) קישור להורדת
- פרוט הסביבה והכלים הנדרשים לבדיקות - על מנת לבדוק את השירות ניתן להשתמש לפחות במחשב אחד, יש להשתמש בארבעת קבצי הקוד; מחלקה ראשית, שרת, לקוח וספריית אבטחה. יש להתקין על המחשב את שפת הקוד python, ואת הספריות הבאות, אם לא קיימות כבר;

- ☐ socket
- ☐ struct
- ☐ random
- ☐ threading
- ☐ sqlite3
- ☐ argon2-cffi
- ☐ cryptography
- ☐ json
- ☐ gzip
- ☐ base64
- ☐ sys
- ☐ HybridEncryption (אני יצרתי, "ספריית האבטחה", אותה ציינתי)
- ☐ ServiceMainClass - אני יצרתי גם כן, ספריית האם, ממנה שתי הספריות (הראשיות - ספריית השרת והלקוח יורשות, בעלת תכונות ופעולות הקשורות (לשתייהן)
- ☐ datetime
- ☐ time
- ☐ socket
- ☐ os
- ☐ string
- ☐ customtkinter
- ☐ tkinter
- ☐ PIL

כמו כן, מומלץ להשתמש בסביבת העבודה Pycharm.

זאת ועוד, אם רוצים להשתמש בכמה מחשבים שונים באותה רשת פנימית, במחשב אשר ישמש כשרת יש לפתוח את ה-cmd, באמצעות לחיצה על כפתור windows ו-r במקביל, בחלונית שנפתחת יש להקיש "cmd", כך נפתח ה-command line של המחשב. יש לרשום "ipconfig" ולקחת את הכתובת "IPv4 Address" תחת "Ethernet adapter". את הכתובת הנ"ל יש לשנות בפונקציה הראשית לכל אחד מהמחשבים - המחשב שישמש כשרת ואלו שישמשו כלקוחות.

```
This is the mail class of the Davinci Mail service, this class has attributes which the two classes;
"MailClient" and "MailServer" should have in common. In addition this class has "encrypt_message" and "decrypt_message"
which the "MailClient" and "MailServer" should have in common. Furthermore, this class have the following two function -
"hash_password", "verify_password", which are not used in both "MailClient" and "MailServer", but the first one used in the first class and seconded
in the seconded respectively, because these function are two parts of the same purpose - hashing the passwords it more efficient to define them
in the same class, for easy changes that won't involve changes in each class but only in this class.
It is important to notice that "MailClient" and "MailServer" inherit from this class and that this class has more functions for the different service uses.
'''
def __init__(self, host='127.0.0.1', port=5555):
    self.host=host # The ip address of the computer whom hosts the server
    self.port=port # The port on the computer which will be used for the service data flowing, i read the port 5555 is rarely used in computers of "regular" individuals
    self.ph=PasswordHasher() # creating instance of argon2 password hasher in order to have a strong and reliable hasing capability to my service
```

כך ה-host של ה-server יוגדר ככתובת ip הפנימית של המחשב עליו הוא מורץ, והמחשבים עליהם יורצו קוד הלקוח ינסו להתחבר למחשב בעל הכתובת ip של השרת, מחלקת הלקוח והשרת יורשות מהמחלקה ממנה מצורף קטע הקוד הנ"ל (המחלקה הראשית). *פורט התוכנה הינו קבוע והינו 5555.

תיאור פרוטוקול התקשורת

תיאור מילולי;

הפרוטוקול בו השירות משתמש הינו פרוטוקול TCP. פרוטוקול TCP הינו פרוטוקול תקשורת מוכר וותיק שהוכיח עצמו, המאפשר העברה מהימנה של נתונים בין יישומים ברשתות מחשבים פנימיות וחיצוניות - כולל האינטרנט. הוא פועל בשכבת התעבורה (השכבה הרביעית) של מודל חמשת השכבות והוא פרוטוקול מבוסס קישור, מה שאומר שהוא יוצר קישור לפני העברת הנתונים ומסיים אותו לאחר מכן. יתרונותיו העיקריים כוללים העברה מהימנה של מידע כפקטות (חבילות מידע קטנות), על ידי שימוש במנגנוני אישור ושליחה חוזרת של פקטות שאבדו, הבטחת סדר הגעת הפקטות ליעד וכן מנגנוני בקרת עומס כדי להתמודד עם עומסים ברשת. תכונות אלו הופכות את TCP לפרוטוקול מתאים ליישומים הדורשים העברה מלאה וסדורה של נתונים, כגון גלישה באינטרנט, דואר אלקטרוני והעברת קבצים. ולכן בחרתי להשתמש בו בשירות שלי - שירות מייל - דואר אלקטרוני. כמות הבתים (bytes) בהודעות תעבורת התקשורת של השירות איננה קבועה, היא תלויה בפעולות השונות שהמשתמש מבצע, למשל בגודל תוכן המייל שהמשתמש שולח וגודל הקבצים אשר הוא מצרף. הכמות המינימלית היננה 4 בתים וגודל הקבצים המצורפים המקסימלי הניתן לשלוח בהודעה אחת לעת עתה הינו 25kb. *חשוב לזכור כי ה-IV היחודי לכל הודעה (גודל קבוע - 16 בתים) וכמות הבתים של ההודעה (גודל קבוע - 4 בתים) הצפויה להישלח נשלחים לפני כל הודעה בנפרד. *כל הודעה עוברת תהליך הכנה לשליחה - כל הודעה עוברת המרה ל-Json, מומרת לייצוג בבתים לשם הצפנתה, ישנו את תהליך שינוי הביטים בכל בית, תהליך ההצפנה באמצעות מפתח ה-AES וה-IV ותהליך הדחיסה באמצעות gzip. *סדר זה תקף לכל הודעה למעט ההודעות בתהליך ההתחברות - תרשים ראשון של העברת המידע. בפירוט בהמשך, בנוגע לכל הודעה המופיע בתרשים הראשון, מצויין האם מתבצעת הצפנה או לא (כולן עוברות המרה ל-json ודחיסה).

Transmission Control Protocol (TCP) Header 20-60 bytes

source port number 2 bytes				destination port number 2 bytes			
sequence number 4 bytes							
acknowledgement number 4 bytes							
data offset 4 bits		reserved 3 bits		control flags 9 bits		window size 2 bytes	
checksum 2 bytes				urgent pointer 2 bytes			
optional data 0-40 bytes							

ה-Header הוא כמו בול המצורף למכתב, הוא מכיל בתוכו נתונים שונים וחשובים ההכרחיים לכך שההודעות המשתמשות בפרוטוקול יגיעו ליעדם בצורה נכונה, ויטופלו כראוי על ידי המערכות השונות בדרך. ניתן לראות שה-Header של הפרוטוקול TCP מכיל נתונים רבים כמו פורט מקור, פורט יעד והכי חשוב - "sequence number" ו-"acknowledgment number", ששניהם הינם מספרים חשובים לתקשורת בין הלקוח והשרת המייצגים את החוזקה של הפרוטוקול, כל הודעה מקבלת אישור קבלה ומקבלת מספר ברצף השליחה ביניהם על מנת שאף הודעה לא תתפספס וסדר השליחה יהיה ידוע וברור.

פירוט כלל ההודעות הזורמות במערכת

בשירות ניתן לחלק את ההודעות לשתי קבוצות מוחלטות ויחידות, החלוקה הינה מבחינת מקור ויעד ; הודעות שנשלחות מהלקוח לשרת והודעות אשר נשלחות מהשרת ללקוח. לכן אציג את ההודעות לפי שתי הקטגוריות הנ"ל, מלבד הודעות אורך ההודעה הצפויה וה-IV. * כל הודעה למעט ההודעה בה השרת שולח את המפתח הציבורי שלו, הודעות האורך והודעות ה-IV הינן במבנה נתונים מסוג מילון.

הודעות הנשלחות מהלקוח לשרת ;

* כל הודעה המועברת מהלקוח לשרת בנויה בתבנית הבסיסית הבאה ;
פעולה - סוג הפעולה, מידע מצורף (לא תמיד, יכול להיות בכמה פרמטרים שונים, תלוי בסוג הפעולה, ראו פירוט בהמשך).

- 1. סוג ההודעה - שליחת מפתח AES (מוצפן ע"י המפתח הציבורי של השרת) ומספר ההזזות הביטים בכל בית לאחר שהוסף לו מספר גדול. בשל כך שאינני מצפין את מספר הביטים להזזה, חשבתי לבצע טריק מעניין לשם כך שבמקרה של מתקפת man in the middle, לא יובן איזה מספר נשלח ומהו שימוש ; אני לוקח את ארבעת המספרים השונים בכתובת ה-ip של השרת, מכפיל כל אחד בשבע, מוסיף להכפלה של כל אחד בשבע את המספר 13 ולאחר מכן מכפיל את התוצאה הסופית של כל אחד ב-2. לאחר מכן אני מחבר את ארבעת המספרים הללו לאחד, את מספר זה אני מוסיף למספר הביטים להזזה בכל בית, כך הוספתי שיטת "הצפנה" מקורית משלי במקרה הנקודתי הנ"ל ומובטח שלא יהיה דפוס זהה במספר הנ"ל לאורך זמן רב, הרי כתובות ip פנימיות הן דינמיות ומוחלפות כל כמה שעות. בעת קבלת המספר בשרת, מתבצע החישוב המתואר לעיל באמצעות כתובת ה-ip, מוחסר מהמספר שנשלח ומתקבל מספר הביטים להזזה, כך יש תיאום בין השרת ללקוח גם בנושא זה.**

שדות ההודעה ופירוט אודותיהם-

encrypted_key, num

השדה הראשון מייצג את הערך המוצפן של המפתח הסימטרי והשדה השני, ששמו עלום - "num", מייצג את מספר הביטים להזזה ועוד המספר לפי מנגנון החישוב עליו פירטתי.

- 2. סוג ההודעה - הרשמה למערכת (register), נשלחת לשרת בעת בקשה של משתמש באפליקציה לביצוע הרשמה לחשבון חדש.**

שדות ההודעה ופירוט אודותיהם -

action, email, password , security_q1, security_a1, security_q2, security_a2, birthday date.

השדה הראשון מייצג את הפעולה - "register", השדה השני מייצג את כתובת המייל אותה המשתמש רוצה לשייך לעצמו, השדה השלישי מייצג את הסיסמא אותה הוא בחר (לאחר שעברה גיבוב), השדה הרביעי מייצג את סוג שאלת האבטחה הראשונה עליה המשתמש בחר לענות, השדה החמישי מייצג את תשובתו עליה, השדה השישי מייצג את סוג שאלת האבטחה השנייה עליה המשתמש בחר לענות, השדה השביעי מייצג את תשובתו עליה והשדה השמיני מייצג את תאריך הלידה של המשתמש.

3. סוג ההודעה - התחברות למערכת (login), נשלחת לשרת בעת בקשה של משתמש באפליקציה להתחברות באמצעות הזנת שמו וסיסמא לחשבון שלו.

שדות ההודעה ופירוט אודותיהם -

action, email, password

השדה הראשון מכיל את הפעולה - "login", השדה השני מכיל את המייל של המשתמש הרוצה להתחבר, והשדה השלישי מכיל את הסיסמא שהזין (ללא גיבוב).

4. סוג ההודעה - שליחת מייל "send message", נשלחת לשרת בעת ניסיון של המשתמש לשלוח מייל לחשבון אחר או לעצמו.

שדות ההודעה ופירוט אודותיהם -

action, sender, recipient, title, message, file_data, image_data

השדה הראשון מייצג את הפעולה - "send_message", השדה השני מייצג את כתובת המוען, השדה השלישי מייצג את כתובת הנמען, השדה הרביעי מייצג את כותבת המייל, השדה החמישי מייצג את תוכן ההודעה, השדה השישי מייצג את תוכן קובץ ה-txt, או string גנרי לכך שלא צורף קובץ txt והשדה השביעי מייצג את תוכן תמונת ה-PNG (לאחר שעבר המרה ל-string, גיליתי באמצעות שגיאה כי לא ניתן לאחסן bytes בטבלת SQLite) או string גנרי לכך שלא צורפה תמונה כלשהי.

5. סוג ההודעה - בקשה לבדיקה האם ישנם מיילים חדשים (refresh), נשלחת לשרת בעת בקשה של משתמש מחובר באפליקציה לרענון ההודעות, כלומר האם ישנן חדשות.

שדות ההודעה ופירוט אודותיהם -

action, recipient,

השדה הראשון מייצג את הפעולה - "retrieve_messages", השדה השני מייצג את הנמען, מייל המשתמש המחובר (הרי הוא מבקש את המיילים ששלחו אליו, יש לו גישה רק אליהם).

6. סוג ההודעה - התנתקות (logout), בקשה של משתמש מחובר להתנתק מהמערכת, בין אם בלחיצה על הכפתור המתאים ובין אם בסגירת האפליקציה ע"י הכפתור "X":

שדות ההודעה ופירוט אודותיהם -

action, mail, blocked_users, messages, color, violations, distribution_list1_names, distribution_list2_names, distribution_list3_names, labels_names

השדה הראשון מייצג את הפעולה - "logout", השדה השני מייצג את המייל של המשתמש המחובר, השדה השלישי מייצג את החשבונות אשר המשתמש חסם, השדה הרביעי מייצג את כל ההודעות אשר נשלחו למשתמש עם סינונים ייחודיים שהמשתמש ביצע לגביהם (רשימת מילונים), השדה החמישי מייצג את הצבע העיקרי (בערך הקסדצימלי) של התצוגה לפי בחירת המשתמש (אם לא שינה אז יהיה לפי ה-default), השדה השישי מייצג את מספר ההפרות שהמשתמש ביצע, השדות השביעי, השמיני והתשיעי מייצגים בהתאמה את הכתובות שהמשתמש הוסיף לרשימת התפוצה, הראשונה, השנייה והשלישית. השדה העשירי והאחרון מייצג את שמות ה-Lables שהמשתמש הגדיר (או הגנריים במקרה שלא שינה אותם).

7. סוג ההודעה - שליחת הודעה לשרת כי משתמש בעל חשבון שכח את סיסמתו (forgot_password), לכן על השרת לשלוח את השאלות האבטחה אשר המשתמש ענה עליהם בעת הרשמתו לשירות כדי שיוכל בעזרתם לאמת את עצמו וליצור סיסמא חדשה.
שדות ההודעה ופירוט אודותיהם -

action, email
השדה הראשון מייצג את הפעולה או יותר נכון בקשה במקרה הנ"ל - "forgot_password" והשדה השני מייצג את מייל המשתמש.

8. סוג ההודעה - איפוס סיסמא (reset_password), נשלחת לשרת לאחר שהלקוח קיבל את שתי שאלות האבטחה (עליהן ענה המשתמש בעת יצירת המייל שהוא רוצה לשנות את סיסמתו), הציג למשתמש, המשתמש ענה עליהם והגדיר סיסמא תקנית ולחץ על כפתור - "reset_password".

שדות ההודעה ופירוט אודותיהם -
action, email, security_a1, security_a2, new_password
השדה הראשון מייצג את הפעולה - "reset_password", השדה השני מייצג את המייל של המשתמש המבקש לבצע את האיפוס, השדות השלישי והרביעי מייצגים בהתאמה את התשובות לשאלות האבטחה הראשונה והשנייה אותן הלקוח הזין והשדה החמישי מייצג את הסיסמא החדשה אשר הלקוח מבקש להגדיר (מתבצע לה גיבוב בשרת).

9. סוג ההודעה - הודעה לשרת כי הלקוח סגר את התוכנה לפני שביצע תהליך login (יכול לקרות גם לאחר התנתקות באמצעות כפתור logout במסך הראשי, ולאחר אישורה, סגירת התוכנה ששום משתמש לא מחובר, כלומר במסך הפתיחה).

שדות ההודעה ופירוט אודותיהם -
action
השדה היחיד, מייצג את הפעולה - "close before logged in".

הודעות הנשלחות מהשרת ללקוח;
* כל הודעה המועברת מהשרת ללקוח בנויה משתי תבניות הבסיסיות הבאות;
תבנית ראשונה - סטטוס; הצלחה, מידע מצורף; מגוון ויכול להתפרס על כמה שדות, הודעה; הודעה מצורפת מהשרת. (יכול להיות רק מידע מצורף או רק הודעה ולא בהכרח שניהם, ראו פירוט בהמשך).
תבנית שנייה - סטטוס; תקלה / שגיאה, הודעה; הודעה מצורפת מהשרת.
* בהודעות שבהם קיימת אפשרות שהסטטוס שלילי עקב מידע לא תואם מהלקוח (סיסמא שגויה למשל), הצגתי גם את מקרה אי האישור / הצלחה. להודעות שאי הצלחה נובעת מתקלה לא צפויה הוספתי התייחסות לכך.

1. סוג ההודעה - אשרת החיבור (כל עוד לא השרת לא הגיעה למגבלת הלקוחות המחוברים תאושר), הודעה הינה לא מוצפנת.
שדות ההודעה ופירוט אודותיהם -

accept
להודעה יש שדה אחד בלבד, וערכו הינו בוליאני, יכול להכיל True או False.

2. סוג ההודעה - העברת המפתח הציבורי של השרת ללקוח, הודעה הינה לא מוצפנת.
שדות ההודעה ופירוט אודותיהם - הודעה זו הינה ההודעה היחידה בכל פרוטוקול התקשורת של השירות שאיננה מילון, היא מכילה שדה אחד בלבד - המפתח הציבורי של השרת.

3. סוג ההודעה - אישור / אי אישור הרשמה (register)
שדות ההודעה ופירוט אודותיהם -

status, message
אם אושרה ההרשמה ישלח בשדה הראשון "success" ובשני הודעה מתאימה, אחרת (המייל כבר קיים או תקלה במסד הנתונים) ישלח בשדה הראשון - "error" והודעה מתאימה.

4. סוג ההודעה - אישור / אי אישור חיבור (login)

שדות ההודעה ופירוט אודותיהם - שדות במקרה של אישור;

status, message, blocked_users, last_color, dist1, dist2, dist3, labels_names
 בשדה הראשון יהיה "success", בשני הודעה מתאימה, בשדה השלישי יתקבל רצף של כלל המשתמשים החסומים כאשר / מפריד בין כל אחד מהם (יש בדיקה לכך שאין כתובת מייל עם / ולא ניתן להזין כזו בעת חסימת משתמש), בשדה השלישי יהיה הערך של צבע התצוגה הראשי אותו בחר המשתמש בערך הקסדצימלי (אם לא בוצע שינוי יהיה None), בשדה הרביעי, החמישי והשישי יתקבלו בהתאמה רצף של כלל המשתמשים הנמצאים ברשימת התפוצה הראשונה, השנייה והשלישית כאשר / מפריד בין כל אחד מהם. בשדה השביעי והאחרון יהיו שמות ה-labels שהמשתמש הגדיר, אם לא הגדיר יהיו השמות הגנריים.

שדות במקרה של אי אישור או תקלה;

status, message

בשדה הראשון יהיה "error" ובשדה השני סיבת אי האישור או הודעה על תקלה.

5. סוג ההודעה - אישור / אי אישור קבלת מייל אשר נשלח

שדות ההודעה ופירוט אודותיהם -

status, message

במקרה של הצל אם אושרה קבלת המייל ישלח בשדה הראשון "success" ובשני הודעה מתאימה. אם לא אושרה קבלת המייל או שהתרחשה תקלה (במסד הנתונים), ישלח בשדה הראשון "error" ובשני הודעה מתאימה.

6. סוג ההודעה - תגובה לבקשה לרענון המיילים

שדות ההודעה ופירוט אודותיהם -

status, messages

השדה הראשון יהיה "success" במקרה של הצלחה והשני יהיה רשימת מילונים לכל ההודעות, כאשר כל מילון מייצג כל מייל אשר נשלח ללקוח. אם התרחשה תקלה לא צפויה (במסד הנתונים) השדה הראשון יהיה "error" ובשני תישלח הודעה מתאימה.

7. סוג ההודעה - תגובה להודעת "forgot_password" שמגיע מהלקוח.

שדות במקרה של הצלחה;

status, message, security_q1, security_q2

השדה הראשון יהיה "success" במקרה של הצלחה לקבלת שאלות האבטחה בהתאם למייל והשני יהיה הודעה מתאימה, השדות השלישי והרביעי יכילו את שאלות האבטחה עליהן המשתמש ענה בעת הרשמתו לשירות.

שדות במקרה של אי הצלחה או תקלה;

status, message

אם המייל לא נמצא, או אם התרחשה תקלה לא צפויה (במסד הנתונים) השדה הראשון יהיה "error" ובשני תישלח הודעה מתאימה.

8. סוג ההודעה - תגובה להודעת "reset_password" שמגיע מהלקוח.

שדות ההודעה ופירוט אודותיהם -

status, message

השדה הראשון יהיה "success" במקרה של אישור שינוי הסיסמא (התשובות תואמות) והשני יהיה הודעה מתאימה. במקרה של אי אישור עקב כתובת מייל שאיננה קיימת, התשובות לשאלות האבטחה אינן זהות לאלו שבמסד הנתונים או תקלה במסד הנתונים, השדה הראשון יהיה "error" והשני הודעה מתאימה.

9. סוג ההודעה - תגובה לבקשת התנתקות לקוח מחובר (אם לחץ ישר X לא יראה את התגובה).
שדות ההודעה ופירוט אודותיהם -

status, message

השדה הראשון יהיה "success" במקרה של התנתקות מוצלחת ושדה השני יכיל הודעה מתאימה, במקרה של אי הצלחה (תקלה במסד הנתונים) השדה הראשון יהיה "error" והשני יהיה הודעה מתאימה.

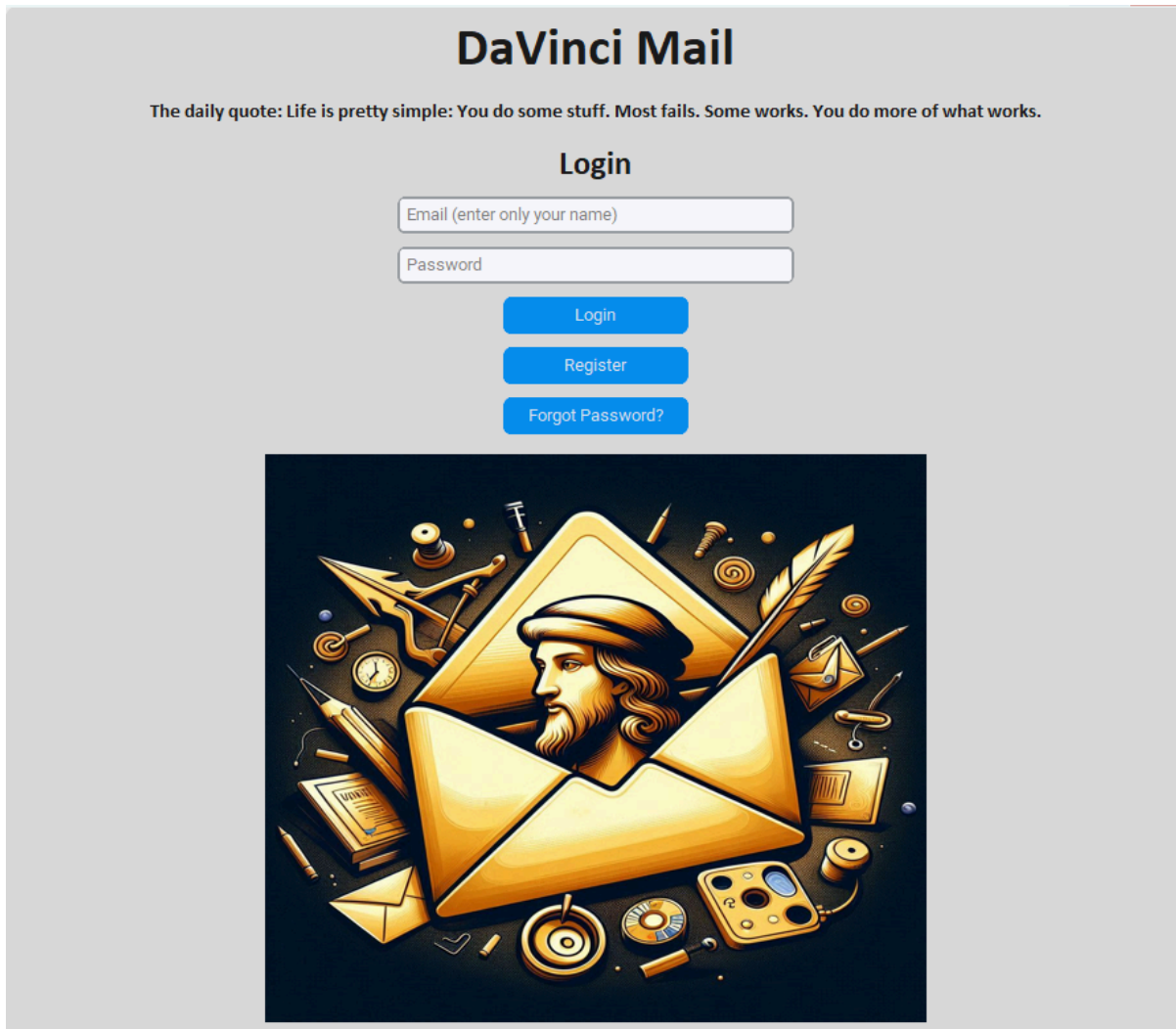
תיאור מסכי המערכת

תיאור כל מסך

● מסך כניסה;

תיאור המסך; המסך הנפתח בעת פתיחת התוכנה, ניתן להתחבר ע"י הזנת שם/שם העסק בלשונית המייל, הזנת הסיסמא בלשונית הסיסמא ולחיצה על כפתור "Login" (במקרה של תקינות הנתונים יהיה מעבר למסך הראשי), ניתן לעבור למסך ההרשמה ע"י לחיצה על כפתור "Register" וניתן להתחיל בתהליך שינוי סיסמא ע"י לחיצה על כפתור "Forgot Password"? ניתן לראות כי ישנו את הציטוט היומי (daily quote), בחרתי להעשיר את המשתמשים בציטוטים נבחרים של לאונרדו דה-וינצ'י אשר על שמו נקרא הפרויקט.

תמונה;



- **מסך הרשמה;**

- **תיאור המסך;**

לאחר לחיצה על כפתור "Register" במסך הפתיחה יפתח מסך ההרשמה יש למלא את השוניות השונות לשם מילוי סיסמא, ראשית יש למלא את שמך, להזין את הסיסמא, להזין את הסיסמא שוב (מניעת טעויות), לאחר מכן יש להזין את תאריך הלידה ולענות של שתי שאלות אבטחה (שונות כמובן - ישנה בדיקה לכך). תמונה של כל ארבעת השאלות:

What was your childhood nickname?
What is your favorite pet?
What is your mother's maiden name?
What was your favorite subject in high school?

לאחר מכן יש לבחור על כפתור register ולחכות לתגובה מהשרת, לדוגמה;

Registration Successful X

i Your email is daniel@DaVinci.cl

אישור

בכל רגע נתון ניתן ללחוץ על כפתור "Back to Login" ותייהיה חזרה למסך הכניסה.
תמונה;

Register

Name

Password

Enter password again

Enter your date of birth, dd/mm/yyyy

What was your childhood nickname? ▾

Answer 1

What is the name of your favorite pet? ▾

Answer 2

Register

Back to Login

Password requirements;

At least six characters, one capital letter and one small letter.

Password recommendations;

For a strong password, use at least 12-16 characters with a mix of uppercase and lowercase letters, numbers, and special symbols (#,%,!), avoiding personal info, common words, and password reuse.

- **מסך שחזור סיסמא ראשון;**

תיאור המסך; לאחר לחיצה על כפתור "Forgot Password?" במסך הכניסה יפתח המסך הראשון כחלק מתהליך שחזור הסיסמא, יש להזין בו את המייל שיש רצון לשנות את סיסמתו. בכל רגע ניתן לסגור את המסך ולחזור למסך הכניסה. *המסך הינו מסוג Toplevel כלומר מסך נפרד מהאפליקציה עצמה.

תמונה;

- **מסך שחזור סיסמא שני;**

תיאור המסך; לאחר לחיצה על כפתור "Submit" במסך שחזור הסיסמא הראשון יופיע חלק נוסף, יש למלא את השוניות שבו, מענה על שתי שאלות האבטחה עליהן ענה המשתמש בעת הרשמתו לשירות ולהזין את הסיסמא פעם אחת ולאחר מכן שוב פעם לצורך מניעת טעויות. אם הסיסמא תוגדר כחזקה ע"י השירות ובמקרה של הצלחה יקפוץ חלון עם הודעה מתאימה, לאחר לחיצה על אישור מסך השחזור יסגר. במקרה של כישלון יקפוץ חלון לסיבה, והמשתמש יוכל לתקנה אם יבחר בכך. בכל רגע ניתן לסגור את המסך ולחזור למסך הכניסה. *המסך הינו בעצם שינוי ה-Toplevel, כלומר שינוי מסך השחזור הראשון ע"י הוספת frame אליו.

תמונה;

- מסך טעינה

תיאור המסך; מסך טעינה זה מופיע במעבר בין מסך הכניסה למסך הראשי לאחר ביצוע login מוצלח, יש סאונד שיצרתי בעזרת בינה מלאכותית ברקע.
תמונה;

Loading Your Data (:



DAVINCI MAIL

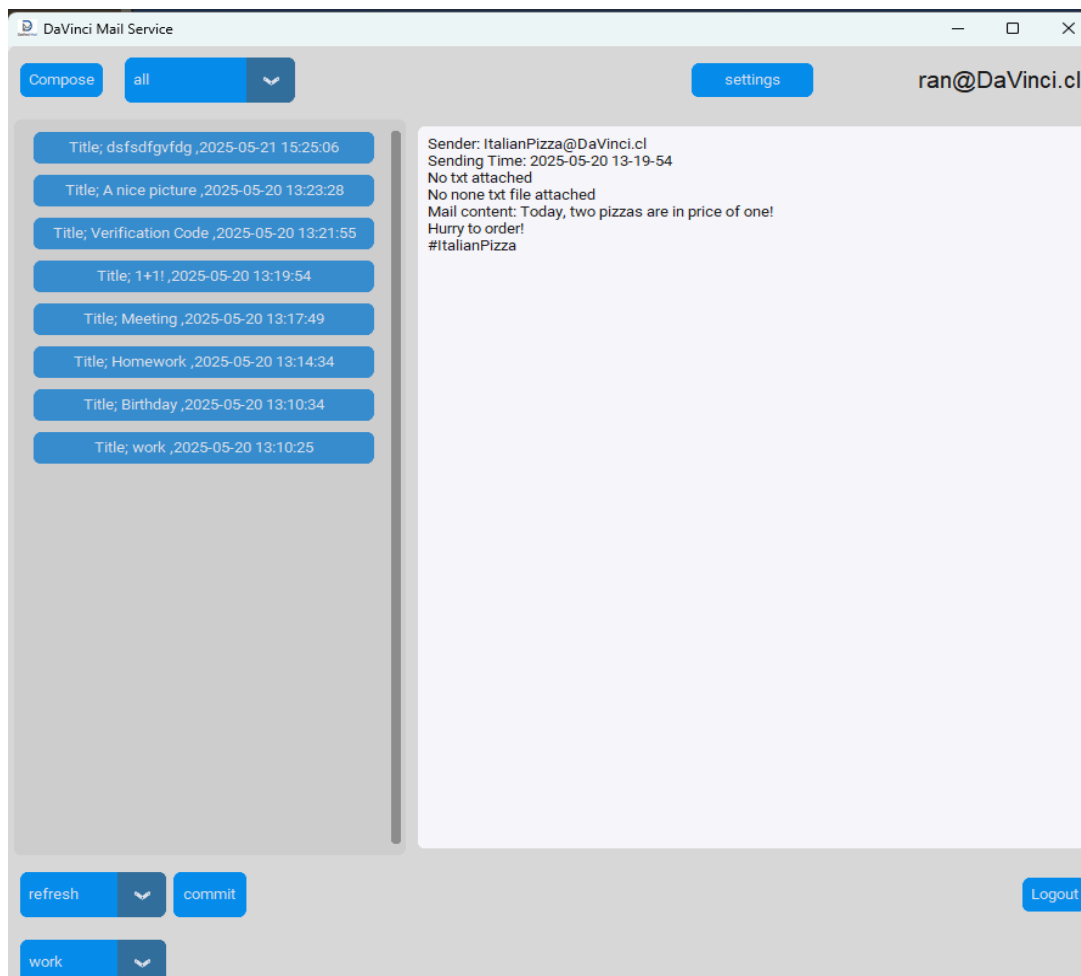
• מסך ראשי;

תיאור המסך;

המסך הראשי של התוכנה, בצד שמאל ניתן לראות את עמודת המיילים אשר בה יוצגו כל המיילים לפי קטגוריית הסינון/פילטור או ה-Label, כאשר נלחץ על אחד המיילים (על הכפתור המייצג אותו) יופיע במלבן הלבן המידע הבא; שולח, תאריך שליחה, האם צורף קובץ טקסט או תמונה ותוכן.

ניתן ללחוץ על כפתור "compose" לפתיחת חלונית לשליחת הודעות, ניתן ללחוץ על כפתור "settings" למעבר למסך ההגדרות, ניתן ללחוץ על כפתור "logout" למעבר למסך הראשי וביצוע התנתקות. כאשר נלחץ על לשונית הבחירה מצד ימין למעלה שבתמונה מתחת מופיע בה "all", נוכל לבחור להציג את כל המיילים לפי אפשרות סינון/פילטור הרצויה או את כלל המיילים תחת Label מסוים. לאחר מכן נלחץ על כפתור "commit", ויוצגו כלל המיילים לפי הקטגוריה שבחרנו בעמודת המיילים השמאלית. אם נרצה לבצע רענון (לאחר שינוי קטגוריה, או תחת קטגוריה "all" - בקשה מהשרת לבדיקה האם יש מיילים חדשים), להוסיף מייל לקטגוריה (כולל לאחד ה-Labels), למחוק מייל, או לבצע forward למייל עלינו לסמן זאת בלשונית הנמצאת מצד שמאל לכפתור "commit" ולאחר מכן ללחוץ עליו (יבוצע על המייל האחרון עליו הלקוח לחץ (על הכפתור המייצגו בעמודת המיילים השמאלית)). אם נוסיף מייל לאחד ה-Labels הוא יתווסף ל-Label שסומן בלשונית הבחירה השמאלית והתחתונה ביותר - היכן שרשום "work" בתמונה המצורפת, משתמש זה הגדיר את המילה "work" כשם של אחד מה-Labels.

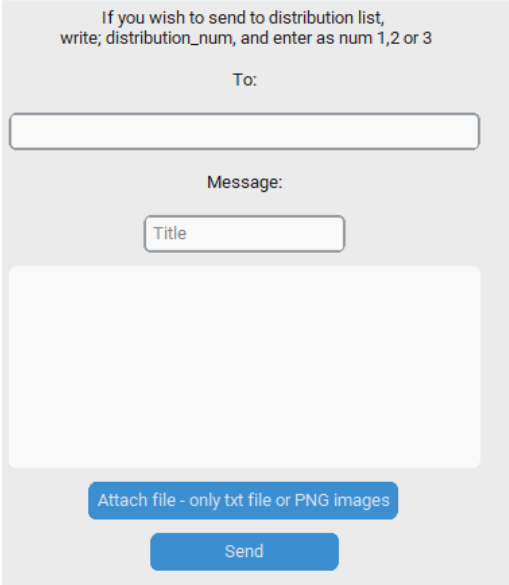
תמונה;



- **מסך שליחת מיילים;**

תיאור המסך; כאשר נלחץ על כפתור "compose" במסך הראשי חלונית זו תיפתח. בלשונית העליונה יש להזין את כתובת המייל אליה אנו רוצים לשלוח את המייל, בלשונית האמצעית יש להזין את כותרת המייל ובמלבן התחתון יש להזין את תוכן ההודעה. ניתן להוסיף קובץ txt ותמונה מסוג PNG עד 25kb (גודל שניהם יחד), אם תיהיה חריגה תקפוץ חלונית עם הודעה מתאימה והמידע של הקובץ אשר גרם לחריגה ימחק. ניתן לצרף עד שני קבצים, קובץ txt אחד וקובץ PNG אחד, לאחר שליחת ההודעה וקבלת אישור מהשרת על קבלתה תוצג הודעה מתאימה והחלונית תיסגר. לשליחת ההודעה לרשימת תפוצה יש לפעול כמוסבר. אם כתובת המייל שהוזנה איננה בפורמט השירות - name@DaVinci.cl - תוצג הודעה מתאימה (בדיקה בצד הלקוח לצורך יעילות) ואם כתובת המייל שהוזנה איננה קיימת השרת יחזיר הודעה מתאימה ללקוח שתוצג למשתמש.

תמונה;



● מסך הגדרות;

*שם בכתובת המייל - name, כתובת היא במבנה name@DaVinci.cl
תיאור המסך; במסך הגדרות ניתן לשנות צבע כפתורי המערכת באמצעות הלשונית העליונה, ניתן לחסום משתמשים באמצעות לחיצה על כפתור "Block user", תפתח חלונית ובה יש לרשום את השם בכתובת המייל של המשתמש אותו רוצים לחסום. ע"י לחיצה על כפתור "Remove blocked user", תפתח חלונית ובה יהיה רשום את כלל המשתמשים אשר נחסמו, בחלונית ישנה לשונית שבה ניתן להזין את השם בכתובת המייל של הכתובת אותה אנו רוצים להוריד מהרשימה (מיילים של משתמשים חסומים לא יוצגו, ישמרו במסד הנתונים למקרה שתבצע הסרה של משתמש מרשימת החסומים). ניתן לשנות את רשימת התפוצה המופיע בתפריט הסימון. בעת לחיצה על כפתור "Add to distribution list", תפתח חלונית ובה לשונית להוספת השם בכתובת המייל של הכתובת אשר המשתמש רוצה להוסיף לרשימה המסומנת, בעת לחיצה על כפתור "Remove from distribution list", תפתח חלונית ובה רשימת כל כתובות המייל ברשימת התפוצה שסומנה, בלשונית ניתן לרשום השם בכתובת המייל שיש רצון להסירה. בלחיצה על כפתור "change label name" תפתח חלונית ובה שמות כל ה-labels הקיימים והסבר לקלט שיש להזין על מנת לשנות שם ספציפי.
ע"י לחיצה על כפתור "Service regulations", ניתן לעבור למסך רגולציה / תקנון השירות וע"י לחיצה על כפתור "Back" ניתן לחזור למסך הראשי.

תמונה;



- מסך רגולציה / תקנון השירות; תיאור המסך; במסך ניתן לקרוא את תקנון השירות, ניתן ללחוץ על כפתור "Back" על מנת לחזור למסך ההגדרות.
תמונה;

Service Policy

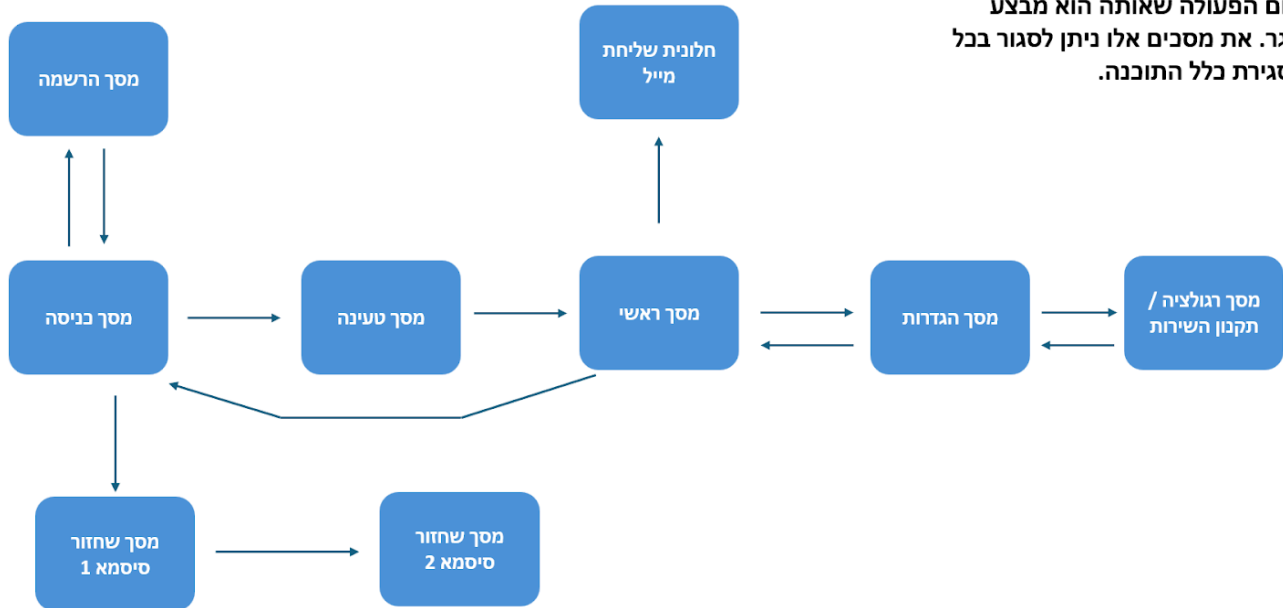
1. Use Respectful Language:
 - Always communicate politely when contacting other users. Avoid using offensive or inappropriate words in any communication.
2. Avoid Sending Harmful Content:
 - Never send files containing malicious software, viruses, or harmful links. Ensure all shared content is safe, legal, and appropriate.
6. Respect Privacy and Security:
 - Avoid sharing sensitive personal or financial information unless absolutely necessary.
7. Do Not Abuse the Service:
 - Refrain from using the mail service for unlawful activities, harassment, or spamming.

Violating these rules will result a ban from the service!

Back

תרשים מסכים המתאר את היררכיית המסכים והמעברים ביניהם (Screen flow diagram)

*אם יש מסך שאין ממנו חץ חזרה למסך אחר, בסיום הפעולה שאותה הוא מבצע המסך נסגר. את מסכים אלו ניתן לסגור בכל רגע ללא סגירת כלל התוכנה.



גיא משה טארנטו - DaVinci Mail

תיאור מבני הנתונים

פירוט מבני הנתונים (מסד נתונים וקבצים);

אכסנת כלל הנתונים של כלל המשתמשים במערכת נעשית ע"י מסד נתונים טבלאי מסוג SQLite3. לשרת ישנו את מסד הנתונים הראשי הנקרא; mail_server.db. לכל לקוח ישנו מסד נתונים זמני שמוזנים בו נתונים כל עוד הלקוח מחובר ומאוכסנות בו ההודעות אשר הוא מקבל, לצורך יעילות ונוחות הסינון של המערכת. מסד נתונים זה נקרא; user_message.db.

קבצי קוד;

server.py, client.py, mainClass.py, encryptionLib.py

קבצי תמונות / שמע;

image#1.png, logo.jpeg, app_icon.ico, modern_electronic_melody.mp3

קבצי מפתחות RSA;

מפתח פרטי - private_key.txt, מפתח ציבורי - public_key.txt.

פירוט מסדי הנתונים של השירות;

לקוח;

שם הקובץ - user_message.db

טבלת messages

מפתח ראשי - id

שם השדה	תיאור	טיפוס השדה	דוגמאות	מאפיינים מיוחדים
id	מספר ייחודי לכל הודעה	INT	155	NOT NULL PRIMARY KEY AUTOINCREMENT
sender	מייל השולח	TEXT	guy@DaVinci.cl	אין
title	כותרת המייל	TEXT	My recent client code	אין
message	תוכן המייל	TEXT	attached, the recent version of the client code and a nice picture.	אין
timestamp	חותמת זמן של זמן השליחה	TEXT	2025-05-02 15-06-18	אין
important	סינון - חשיבות	INT	0	DEFAULT 0
readlater	סינון - קריאה מאוחר יותר	INT	1	DEFAULT 0
deleted	נמחק	INT	0	DEFAULT 0
file_data	מכיל את הטקסט של קובץ הטקסט	TEXT	import socket import struct import...	אין
image_data	מכיל את המידע של התמונה, לא ניתן לשמור כ-bytes ב-sqlite, לכן הומר לייצוג טקסט.	TEXT	H+vX5BGbjWqYXR0/DX GsrTZZcCqOWFtEyBAG AABAgRqCxQPWLURH oCBAGQIECAQO8CALbv K2R8BAGQI...	אין

גיא משה טארנטו - DaVinci Mail

אין	NNYNY	TEXT	שיטה ייחודית שפיתחתי, כמספר ה-labels (חמש לכל משתמש) ישנו string של חמש אותיות 'N' או 'Y', לדוגמה הרצף הבא ; NNYNY, כך למשל מסומן כי מייל משויך ל-label השלישי והחמישי, ומחלקת MailClient יודעת בהתאם ל-string הנ"ל של כל הודעה לאיזה label שויכה (אם שויכה).	labels
-----	-------	------	---	--------

שרת ;
 שם הקובץ - mail_server.db
 טבלת users
 מפתח ראשי - email

שם השדה	תיאור	טיפוס השדה	דוגמאות	מאפיינים מיוחדים
email	מייל ייחודי לכל משתמש	TEXT	guy@DaVinci.cl	PRIMARY KEY, UNIQUE
password	סיסמת המשתמש (לאחר גיבוב)	TEXT	\$argon2id\$v=19\$m=65536,t=3,p=4\$x83t4H+3y7W0og48nAo4GQ\$WVAasCknhfUq2A+ReZY7Fy4bgnZoJ3UNsiKUkxEW7bg	אין
securiy_q1	שאלת האבטחה הראשונה עליה ענה המשתמש	TEXT	What was your childhood nickname?	אין
securiy_a1	תשובה לשאלת האבטחה הראשונה עליה ענה המשתמש	TEXT	guyguy	אין
securiy_q2	שאלת האבטחה השנייה עליה ענה המשתמש	TEXT	What was your favorite subject in high school?	אין
securiy_a2	תשובה לשאלת האבטחה השנייה עליה ענה המשתמש	TEXT	history	אין
blocked_users	רשימת המשתמשים החסומים מופרדים, כאשר יש / בין כל כתובת (שיטה מקורית שלי לשמירת רשימות של דברים בשדה אחד)	TEXT	ItalianPizza@DaVinci.cl/yoram@DaVinci.cl	אין

גיא משה טארנטו - DaVinci Mail

DEFAULT 0	#ff9233	TEXT	צבע התצוגה אותו המשתמש בחר בפעם האחרונה בערך הקסדצימלי	color
DEFAULT 0	0	INT	מספר ההפרות שהמשתמש ביצע לתקנות השירות	violations
אין	0	INT	נחסם / לא נחסם מהשירות	ban
אין	8/9/2007	TEXT	תאריך הלידה	birthday_date
אין	aner@DaVinci.cl/ori@DaVinci.cl/tomer@DaVinci.cl/daniel/@DaVinci.cl	TEXT	רשימת תפוצה ראשונה כאשר יש / בין כל כתובות.	distribution_list1
אין	NULL	TEXT	רשימת תפוצה שנייה כאשר יש / בין כל כתובות.	distribution_list2
אין	NULL	TEXT	רשימת תפוצה שלישית כאשר יש / בין כל כתובות.	distribution_list3
DEFAULT label1/label2/label3/label4/label5 (שמות ה-labels ההתחלתיים כאשר מופרדים ב- /)	School/Work/Meeting/ProjectX/Friends	TEXT	שמות ה-labels שהמשתמש הגדיר לכל אחד מהחמש, אם לא הגדיר יהיו את השמות הגנריים המופעים בעמודת "מאפיינים מיוחדים" בשורה הנ"ל.	labels_names

גיא משה טארנטו - DaVinci Mail

מפתח ראשי - email

טבלת messages

מפתח ראשי - id

שם השדה	תיאור	טיפוס השדה	דוגמאות	מאפיינים מיוחדים
id	מספר ייחודי לכל הודעה	INT	155	NOT NULL PRIMARY KEY AUTOINCREMENT
sender	מייל השולח	TEXT	guy@DaVinci.cl	FOREIGN KEY (sender) REFERENCES users(email)
recipient	מייל הנמען	TEXT	rom@DaVinci.cl	FOREIGN KEY (recipient) REFERENCES users(email)
title	כותרת המייל	TEXT	My recent client code	אין
message	תוכן המייל	TEXT	attached, the recent version of the client code and a nice picture.	אין
timestamp	חותמת זמן, זמן השליחה	TEXT	2025-05-02 15-06-18	אין
important	סינון - חשיבות	INT	0	DEFAULT 0
readlater	סינון - קריאה מאוחר יותר	INT	1	DEFAULT 0
deleted	נמחק	INT	0	DEFAULT 0
file_data	מכיל את הטקסט של קובץ הטקסט	TEXT	import socket import struct import...	אין
image_data	מכיל את המידע של התמונה, לא ניתן לשמור כ-bytes ב-sqlite, לכן הומר לייצוג טקסט.	TEXT	H+vX5BGbjWqYXR0/DX GsrTZZcCqOWFtEyBAG AABAgRqCxQPWLUHrH oCBAGQIECAQO8CALbv K2R8BAGQI...	אין
labels	מכיל string האומר לאיזה label ההודעה שויכה (אם שויכה).	TEXT	NNNYN	אין

סקירת חולשות והאיומים

סקירת האיומים שיתכנו לשירות ופתרונם ע"פ מאפייני המערכת השונים: שכבת האפליקציה (השכבה החמישית במודל חמשת השכבות);

- **sql injection** - סוג של הזרקה, בה תוקף ינסה לפגוע, לעקוף, להקריס או לשאוב נתונים ממסד נתונים באמצעות כך שיכניס בלשוניות הקלט קלט זדוני שאיננו מידע (למשל סיסמא, תוכן הודעה), אלא קלט שהינו קוד SQL אשר באמצעות ביצוע שאילתה (query) עם הקלט הזדוני תוכל להתבצע פגיעה, עקיפת מנגנוני אבטחה, שאיבת נתונים ואף השבתת מסד הנתונים והשרת המתקשר איתו. למשל, אם התוקף יזין את הקלט הבא בלשונית הסיסמא במסך הפתיחה של השירות; "password1 or 1=1", נקבל True עבור כל סיסמא שתזון לכל כתובת מייל וכך כל אדם יוכל להיכנס בקלות לכל חשבון שרק יחפוץ בו. לפתירת הבעיה אני משתמש ב-parameterized queries, כלומר השאילתות התנהלו עם פרמטרים, לכן במקום הזנת קלט המשתמש כחלק מקוד השאילתה, השאילתה תוכן מראש והיא תישלח בנפרד לנתוני הקלט, כך מסד הנתונים מבצע את השאילתה ומתייחס לנתוני הקלט כנתוני חיפוש בלבד ולא יוצר שאילתה עם הקלט עצמו העלול להיות זדוני. שאילתה הינה בעצם פקודה לביצוע במסד הנתונים. דבר זה מונע הכנסת תווים מיוחדים כמו - ! לתוך השאילתה עצמה, דבר אשר יכול לגרום לתקלה וקריסת המסד. השימוש בפרמטרים מונע גם את עקיפת מנגנוני האבטחה באמצעות קלט כפי שצינתי מקודם, שתמיד יהיה נכון ויעקוף את מנגנון המייל והסיסמא (שם משתמש וסיסמא).
דוגמה למימוש parameterized queries במסד הנתונים של הפרויקט;

```
self.cursor.execute( sql: 'SELECT password FROM users WHERE email=? ', parameters: (email,))
```

- מעבר לכך, בקלטים מסוימים ישנה בדיקה לתווים אסורים שיכולים לגרום לבעיה בתפקוד קוד השירות, לכן קלטים המכילים תו אסור אחד או יותר לא התקבלו.
- **עבודה עם אתרי web** - אין בפרויקט כלל עבודה עם אתרי web, ולפיכך נוטרלו האיומים הטמונים בעבודה עמם, כדוגמת התקפת XSS (הזרקת קוד זדוני בדפדפנים של משתמשים), CSRF (תוקף גורם למשתמש מאומת (שמחובר לאתר) לבצע פעולה לא רצויה מבלי ידיעתו) ועוד רבות נוספות...
- **תהליך ה-login אימות ווידוא** - הסברתי על מניעת SQL injection, נכלל כמובן גם במהלך ניסיון התחברות, בעת הרשמה לשירות יש ליצור סיסמא חדשה (אחרת ההרשמה לא מתאפשרת), כל התקשורת ההכרחית (למעט שליחת הודעת אישור ההתחברות, אורך הודעה וה-IV הייחודי) כפי שהוסבר באלגוריתמים מרכזיים ובתיאור פרוטוקול התקשורת הינה מוצפנת באמצעות AES, לכן התקשורת עם השרת ובין היתר הגדרת המייל איננה חשופה במקרה של מתקפת man in the middle (הסנפה של התקשורת בין השרת ללקוח וניסיון לדלות נתונים קריטיים כמו סיסמאות). בנוסף, כפי שהסברתי באלגוריתמים המרכזיים, מיד לאחר יצירת סיום תהליך ההרשמה הכולל הגדרת סיסמא מתבצע גיבוב לסיסמא באמצעות מודל argon2 המשלב "salt". מודל זה הינו מתקדם וחזק ביותר וזכה בתחרויות גיבוב שונות, כך גם אם יש לתוקף את מסד הנתונים, הצליח לפרוץ לתקשורת בין השרת ללקוח או פיצח את מערכת ההצפנה, עדיין אין בידיו את הסיסמא כמו שהוגדרה על ידי המשתמש, אלא את הערך שלה לאחר שעברה גיבוב, שהמרתו לערך הסיסמא המקורית איננה אפשרית. גם ניסיון ההתחברות עם הסיסמא המוצעת נשלחת באופן מוצפן עם מפתח ה-AES וה-IV שנוצר באופן ייחודי לכל הודעה. לכן ישנם מענים חזקים ביותר לניסיון עקיפת תהליך האימות והווידוא או לניסיון גניבת סיסמא.
- **מתקפת MITM** (*ניתן להכליל גם בשכבות נוספות, למשל שכבת הרשת) - על מנת למנוע מצב של מתקפת man in the middle, מצב שבו גורם שלישי עוין מאזין וינסה לדלות בחשאי פרטים רגישים מהתקשורת בין השרת ללקוח (כמו סיסמאות) יישמתי הצפנה בפרויקט. כפי שהסברתי באלגוריתמים מרכזיים, כל הודעה למעט הודעות אישור החיבור, אורך ההודעה העומדת להתקבל וה-IV שאין שום צורך להצפינם, כל הודעה מוצפנת באמצעות מפתח AES מגרסאת bits 256. לכל הודעה יש IV ייחודי הנוצר עבודה, לשם כך שגם אפילו עבור מידע דומה הערך המוצפן יהיה שונה לחלוטין. יתר על כן, ישנה את שכבת האבטחה אותה אני הוספתי (הזזת הביטים בכל בית), עקב כך, כל הודעה העוברת בין השרת ללקוח המצריכה הצפנה, מוצפנת

ברמה הגבוהה ביותר ולכן יש מענה משמעותי וכמעט בלתי ניתן לפיצוח למתקפת in the middle. יתרה מזאת, כפי שהסברתי באלגוריתמים מרכזיים, העברת המפתח הסימטרי נעשית עם הצפנת RSA, כך אין חשש כי תוקף יצליח להשיגו בעת העברתו לשרת בעת פתיחת התוכנה. לסיכום, ההצפנה בפרויקט הינה היברידית - העברת מפתח AES מהלקוח לשרת באמצעות RSA והמשך ההצפנה באמצעות מפתח ה-AES (סימטרי) המשותף שיש בידי השרת והלקוח.

- **מתקפות DOS/DDOS** - מתקפת DOS - מצב שבו משתמש שולח בקשות רבות לשרת ומתקפת DDOS מצב שכמה משתמשים עושים זאת עד שהשרת לא יכול להגיב לתעבורת התקשורת הנורמלית והגעה למצב קריסה. כפי שהסברתי באלגוריתמים מרכזיים, ישנו דילי מובנה במערכת כך שכל הודעה תישלח בדילי שבין 0.1 ל-0.4 שניות, כך גם במצב של עומס השרת יקבל פחות הודעות ביחד ויהיה לו יותר זמן לטפל בכל אחת בנפרד. זאת ועוד, ישנה הגבלה לכמות הלקוחות אשר יכולים להתחבר לשרת בו זמנית.

שכבת התעבורה (השכבה הרביעית במודל חמשת השכבות);

- **פרוטוקול TCP וביסוס הקישור (Three way handshake)** - כפי שהסברתי בתיאור פרוטוקול התקשורת, הפרוטוקול בו השירות משתמש הינו פרוטוקול TCP. פרוטוקול הינו פרוטוקול תקשורת מוכר וותיק שהוכיח עצמו, המאפשר העברה מהימנה של נתונים בין יישומים ברשתות מחשבים פנימיות וחיצוניות - כולל האינטרנט. הוא פועל בשכבת התעבורה (השכבה הרביעית) של מודל חמשת השכבות והוא פרוטוקול מבוסס קישור (באמצעות פרוצדורת Three way handshake), מה שאומר שהוא יוצר קישור לפני העברת הנתונים ומסיים אותו לאחר מכן. יתרונותיו העיקריים כוללים העברה מהימנה על ידי שימוש במנגנוני אישור ושליחה חוזרת של פקטות שאבדו, הבטחת סדר הגעת המנות ליעד, בקרת זרימה למניעת הצפה של המקלט, וכן מנגנוני בקרת עומס כדי להתמודד עם עומסים ברשת. אם תוקף יצליח לחבל בתקשורת בין השרת ללקוח, יש בקוד הלקוח בפעולה המתקשרת עם השרת exception המזהה כשל בתקשורת. אם יהיה זיהוי של כשל בתקשורת, לכל פעולה המערבת תקשורת שהלקוח ינסה לבצע, ישר תופיע חלונת אשר מודיעה על אי חיבור לשרת וכי יש לסגור ולפתוח מחדש את האפליקציה (לכל כשל אחר תשלח פקודה כי יש לסגור את התוכנה ולפתוח מחדש). כמו כן, גם בפעולת התקשורת עם הלקוח בצד השרת ישנו exception המזהה כשל בתקשורת, ישנו גם exception לכל תקלה אחרת, בזיהוי כשל מכל סוג שהוא הקשר עם הלקוח יסגר באותו הרגע וגם ה-thread המנהל אותו.

מימוש הפרויקט

חלק א'

סקירת כל המודולים / מחלקות המרכיבים את המערכת וקשרי הגומלין ביניהם;
מודולים / מחלקות אשר לא פיתחתי (מיובאים)

שם המודול / מחלקה	ייעוד המודול / מחלקה
socket	יישום תקשורת במודל שרת - לקוח
struct	המרה של אורך ההודעה העומדת לשליחה לייצוג בינארי הניתן לשליחה באמצעות ספריית socket
threading	בניית השרת כשרת מרובה משתתפים בהתבססות על תהליכונים.
sqlite3	שימוש במודל לשם בניית מסד נתונים טבלאי, הן בצד השרת והן בצד הלקוח ומימוש מנעול לפני שימוש במסד הנתונים.
gzip	דחיסת ההודעות לפני השליחה.
Json	המרת מבני נתונים מורכבים (כגון dictionaries) על מנת שיהיה אפשר להמירם לבתים בלי לפגוע בטיפוס ה-python.

sys	עצירת הרצת קוד השרת במקרים מסוימים.
base64	המרה של מפתח ה-AES לייצוג ASCII על מנת להצפינו באמצעות RSA.
datetime	קבלת התאריך הנוכחי לצורך בדיקה האם למשתמש יש יום הולדת ובדיקה האם נשלחה כבר למשתמש הודעת מזל טוב ביום הולדתו (כדי שלא תשלח הודעת מזל טוב כמה פעמים, אם התחבר כמה פעמים במהלך יום הולדתו לשרות).
random	הגרלת הזמן הדיילי לשליחת הודעת המשתמש ומספר הביטים הרנדומלי להזזה.
time	יצירת דיילי לשליחת הודעת המשתמש, והמתנה של 15 שניות לניסיון התחברות לשרת, אם בעת פתיחת התוכנה, לא הוצלח ביסוס הקשר בין הלקוח לשרת.
os	שמירת קבצים מצורפים בתיקיית downloads
cryptography	יצירת מפתח AES ו-IV וביצוע הצפנה באמצעותם
tkinter	בניית ממשק גרפי (gui) לשירות
customtkinter	הפיכת הממשק הגרפי של השירות למודרני יותר
PIL	חלק מתהליך הצגת התמונות בממשק הגרפי
string	שימוש במנגנון סינון תוכן לא הולם
argon2-cffi	ביצוע הגיבוב לסיסמאות בעת הרשמה והשוואה של סיסמא מוצעת לשם התחברות בעת תהליך התחברות.
pygame	השמעת סאונד בעת מסך הטעינה

מודולים / מחלקות בפיתוח אישי (נעזרות במודולים / מחלקות מיובאות)

1. שם המחלקה; MailClient

תפקיד המחלקה; המחלקה האחראית על כל ההיבטים של הלקוחות בשירות;
רשת - הקמת ערוץ תקשורת עם השרת באמצעות פרוטוקול TCP, ניהול פרוטוקול תקשורת השירות עם השרת ושליחת וקבלת נתונים אל השרת וממנו. **הצפנה, אבטחת נתונים ואבטחת סייבר** - יצירת מפתח AES ויצירת IV חדש לכל הודעה לפני שליחתה. **Gui** - יצירת instance של כל מחלקות ה-gui השונות וניהול השימוש בהם. **DB** - אחסון ושליפת מידלים ונתונים הייחודיים במסד הזמני של נתוני הלקוח.
תכונות המחלקה;

שם התכונה	תפקיד ושימוש
shifting_bits_num	אחסון מספר הביטים אשר מוזזים בכל בית
aes_key	אחסון מפתח ה-AES של הלקוח
client_iv	אחסון ה-IV הייחודי לכל הודעה
encoded_aes_key	אחסון מפתח ה-AES לאחר שעבר המרה לייצוג ASCII לשם הצפנתו
encryption_inst	אחסון מופע (instance) של מחלקת האבטחה של השירות

אחסון ה-socket של הלקוח	socket
אחסון האם יש תקלה במסד הנתונים	db_error
אחסון כתובת המייל של הלקוח (לפי פורמט השירות)	email
אחסון האם המשתמש מחובר לשרת או לא	connected
אחסון הקישור הפיזי למסד נתוני הלקוח	conn
אחסון "מפעיל" הקישור למסד נתוני הלקוח, ה"מפעיל" מבצע פעולות על מסד הנתונים (שולח שאילתות וכו').	cursor
אחסון ברשימה את שמות כתובות המייל החסומות	blocked_users_names
אחסון ברשימה את שמות כתובות המייל ברשימת התפוצה הראשונה	distribution_list1_names
אחסון ברשימה את שמות כתובות המייל ברשימת התפוצה השנייה	distribution_list2_names
אחסון ברשימה את שמות כתובות המייל ברשימת התפוצה השלישית	distribution_list3_names
אחסון שמות ה-Labels כפי שהשמשמש הגדיר.	labels_names
אחסון מספר הפרות המשתמש את כללי השירות	violations
אחסון גובה המסך	screen_height
אחסון רוחב המסך	screen_width
אחסון מידת גודל המסך	screen_size
אחסון שם הצבע הראשי של התצוגה	current_color
אחסון את הצבע הראשי של התצוגה בייצוג הקסדצימלי איתו ה-customtkinter יכול לעבוד.	current_color_hex
אחסון את שמות צבעי התצוגה הזמינים	app_colors_names
אחסון את צבעי התצוגה הזמינים בייצוג RGB	app_colors_RGB
אחסון object הבית של customtkinter, כל השאר יורשים ממנו.	root

גיא משה טארנטו - DaVinci Mail

פעולות המחלקה;

בהתאם לפורמט;

טענת כניסה - פרמטרים אותם הפעולה מקבלת

טענת יציאה - מה הפעולה מבצעת? מה הפעולה מחזירה?

שם הפעולה	טענת כניסה	טענת יציאה
connet_to_server	אין	ביצוע; התחברות לשרת, אם החיבור לא מתקבל - מוצג עדכון ללקוח והאפליקציה נסגרת. אם החיבור התקבל - יש קבלה של המפתח הציבורי של השרת, המפתח נשמר ובאמצעותו מתבצעת הצפנה למפתח ה-AES של הלקוח. מספר ההזת הביטים מוצפן גם כן (ע"י המנגנון עליו פירטתי), המפתח והמספר המוצפנים נשלחים לשרת. אם השרת לא זמין בעת ביסוס הקשר בשל תקלת תקשורת, המשתמש יוכל לבחור להמתין 15 שניות ולנסות להתחבר לשרת, הוא יכול לעשות זאת שוב ושוב, אלא אם יבחר לסגור את האפליקציה. מחזירה; לא מחזירה ערכים.
init_db	אין	ביצוע; מתחברת, מאתחלת ויוצרת קישור למסד נתוני הלקוח ויוצרת את הטבלאות (אם לא קיימות). מחזירה; לא מחזירה ערכים
empty_db	אין	ביצוע; מרוקנת את מסד נתוני הלקוח. מחזירה; לא מחזירה ערכים.
send_action	data	ביצוע; מייצרת IV ייחודי להודעה, מבצעת הכנה לשליחת המידע (דחיסה והצפנה באמצעות פעולה נוספת), שולחת את אורך המידע לאחר הכנתו ואת ה-IV הנוכחי וכמובן את המידע עצמו לאחר שהוכן לשליחה. מקבלת את ה-IV של הודעת התגובה של השרת, את אורך התגובה ואת התגובה עצמה, מבצעת חילוץ ופענוח באמצעות פעולה נוספת. אם זוהתה exception הקשור לתקשורת התכונה connected של המחלקה תושם ל-False, לכן המשתמש יקבל הודעת תקלה עבור כל פעולה המערבת תקשורת שינסה לבצע. על כל תקלה אחרת תתקבל הודעה כי יש לסגור את התוכנה גם כן. מחזירה; המידע אותו שלח השרת לאחר חילוץ ופענוח או הודעה על כך שיש בעיה (סוגים שונים) ויש לסגור את התוכנה.
register_user	user_data	ביצוע; שלב בביצוע הרשמת משתמש, מוסיפה את הפעולה עצמה ("register") למילון המידע אותו מקבלת ומעבירה לפעולת send_action. מחזירה; את תגובת השרת (שמוחזרת מ-send_action) * כל עוד יש חיבור לשרת, אחרת יוחזר כי יש לסגור את האפליקציה.
login_user	login_data	ביצוע; שלב בביצוע התחברות משתמש, מוסיפה את הפעולה עצמה ("login") למילון המידע אותו מקבלת ומעבירה לפעולת send_action. מחזירה; את תגובת השרת (שמוחזרת מ-send_action) * כל עוד יש חיבור לשרת, אחרת יוחזר כי יש לסגור את האפליקציה.
fetch_organization	rows	ביצוע; העברת המידע המתקבל אודות כל מייל והמאפיינים הייחודיים אליו למילון נפרד (מילון עבור כל מייל והמאפיינים שלו) וחיבור כל המילונים לרשימה. מחזירה; רשימת המילונים. * כל עוד יש חיבור למסד הנתונים, אחרת יוחזר כי יש לסגור את האפליקציה.
fetch_all_messages	including_deleted (אופציונלי)	ביצוע; ייצוא מיילים וכלל מאפייניהם ממסד נתוני הלקוח. מחזירה; הפרמטר אותו היא מקבלת הינו מאותחל באופן שלילי, אם ישאר כך יוחזרו כלל המיילים במסד נתוני הלקוח למעט אלו שנמחקו או ששולחיהם נחסמו. אם הפרמטר ישתנה לחיובי, יוחזרו כלל המיילים במסד

גיא משה טארנטו - DaVinci Mail

נתוני הלקוח. * כל עוד יש חיבור למסד הנתונים, אחרת יוחזר כי יש לסגור את האפליקציה.		
ביצוע; ייצוא כלל המיילים במסד הנתונים העונים לסינון / פרמטר מסוים. מחזירה; כלל המיילים במסד הנתונים העונים לסינון / פרמטר מסוים ברשימת מילונים, כאשר כל מילון מייצג מייל ואת מאפייניו הייחודיים. * כל עוד יש חיבור למסד הנתונים, אחרת יוחזר כי יש לסגור את האפליקציה.	category, search_paramete r (אופציונלי)	fetch_messages_by_c ategory
ביצוע; בודקת האם גודל המסך צריך להיות קטן ולא רגיל, יוצרת מופע מכלל מחלקות ה-gui, איגודם למילון, שינוי צבע כפתורי התצוגה והצגת מסך הפתיחה. *האיגוד למילון נותן גישה מהירה ונוחה לביצוע פעולה בכל מחלקה של כל Frame. מחזירה; לא מחזירה ערכים.	אין	setup_gui
ביצוע; מציגה את ה-Frame אותו היא מקבלת במסך האפליקציה. מחזירה; לא מחזירה ערכים.	frame	show_frame
ביצוע; שלב בביצוע שליחת מייל חדש, מוסיפה את הפעולה עצמה ("send_message") למילון המידע ומעבירה לפעולת send_action. מחזירה; את תגובת השרת (שמוחזרת מ-send_action) * כל עוד יש חיבור לשרת, אחרת יוחזר כי יש לסגור את האפליקציה.	message_data	send_message
ביצוע; שלב בביצוע קבלת מיילים חדשים שנשלחו למשתמש (או את כלל היסטוריית המיילים בעת התחברות), מוסיפה את הפעולה עצמה ("get_messages") למילון המידע ומעבירה לפעולת send_action. מחזירה; את תגובת השרת (שמוחזרת מ-send_action) * כל עוד יש חיבור לשרת אחרת יוחזר כי יש לסגור את האפליקציה.	אין	get_messages
ביצוע; מנתק את המשתמש המחובר, שולח לשרת את מידע (סינוני שהמשתמש ביצע, חסימות וכו'), סוגר את החיבור הנוכחי, מאתחל מחדש מספר תכונות המחלקה, קורא לפונקציית "connect_to_server", מרוקן את מסד נתוני הלקוח וחוזר לדף הבית. מחזירה; לא מחזירה ערכים. * כל עוד יש חיבור לשרת ואין תקלה במסד הנתונים, אחרת יוחזר כי יש לסגור את האפליקציה.	אין	logout
ביצוע; מנתק את המשתמש המחובר, שולח לשרת את מידע (סינוני שהמשתמש ביצע, חסימות וכו'), סוגר את החיבור, מרוקן את מסד נתוני הלקוח הנוכחי ואת האפליקציה. מחזירה; לא מחזירה ערכים. * כל עוד יש חיבור לשרת, אחרת יוחזר כי יש לסגור את האפליקציה.	אין	close_while_running

2. שם המחלקה; MailServer

תפקיד המחלקה; מחלקת השרת בפרויקט, אחראית על ניהול התקשורת עם כלל הלקוחות המחוברים (עד למגבלת הלקוחות אותם היא יכולה לשרת). זאת ועוד, אחראית על מסד נתוני כלל המשתמשים (users) והמיילים (למשל ביצוע פעולות עדכון והוספה).
תכונות המחלקה;

שם התכונה	תפקיד ושימוש
encryption_ins	אחסון מופע (instance) של מחלקת האבטחה של השרת.
private_key	אחסון המפתח הפרטי של השרת
public_key	אחסון המפתח הציבורי של השרת
server_socket	אחסון ה-socket של השרת
max_connections	אחסון מספר החיבורים המקסימלי לשרת
db_lock	אחסון המנעול לשימוש במסד נתוני השרת

פעולות המחלקה;

שם הפעולה	טענת כניסה	טענת יציאה
init_db	אין	ביצוע; מתחברת, מאתחלת ויוצרת קישור למסד נתוני השרת ויוצרת את הטבלאות (אם לא קיימות). מחזירה; לא מחזירה ערכים
start_server	אין	ביצוע; מקבלת חיבור של לקוחות חדשים, אם מספר הלקוחות המחוברים (כולל המחובר החדש) שווה למספר המקסימלי המוגדר בתכונה "max_connections" לא יאושר החיבור בין socket השרת ל-socket הלקוח החדש, תשלח מיד הודעה כי השרת לא מקבל את החיבור ותסגור את הקשר בין ה-sockets. אם מספר הלקוחות המחוברים בהתחשב בלקוח החדש הרוצה להתחבר נמוך ממספר החיבורים המקסימלי, הפעולה תפעיל תהליכון אשר יפעיל את פעולת "handle_client" בהתאם ל-socket וכתובת הלקוח. מחזירה; לא מחזירה ערכים.
handle_client	client_socket, addr	ביצוע; שולחת הודעה לשרת כי אושר החיבור, שולחות את אורך המפתח הציבורי של השרת ללקוח והמפתח עצמו, מחכה לקלט מפתח ה-AES של הלקוח ומספר ההזזת הביטים, מפענחת את המפתח באמצעות פעולה נוספת, ממירה את המפתח AES לייצוג בבתים, מפענחת את מספר הביטים להזזה, לאחר מכן עוברת להיות אחראית, על קבלה ושליחת תעבורת התקשורת בין השרת ללקוח; יש לולאת while המקבלת הודעות מהלקוח. הלולאה ממשיכה כל עוד הלקוח לא מתנתק בזמן החיבור לשרת (הן ע"י לחיצה על כפתור logout או ע"י לחיצה על כפתור "X" כאשר נמצא במסך הראשי), במידה והתנתק כאשר מחובר (עם משתמש) הפעולה תשלח הודעה ללקוח בנוגע לסטטוס ההתנתקות, אם התנתק כאשר לא היה מחובר (עם משתמש), לא תשלח כלל הודעה, בשני המקרים תבצע break בלולאה, תסגור את ה-socket וה-thead של הלקוח יסגר. לכל הודעה הפעולה תקבל את ה-IV הייחודי שלה, את אורך ההודעה ואת ההודעה עצמה, באמצעות פעולה נוספת תבצע חילוץ, פענוח והמרה לטיפוס Python, לאחר מכן תעביר את המידע לפעולת

"process_request", תקבל את התגובה, תיצור באמצעות פעולה נוספת IV להודעה החדשה, תשלח אותו, תשלח את אורכה ולבסוף תשלח אותה, לאחר מכן מתחילה איטרציה חדשה בלולאה. אם מזוהה כשל תקשורת או כל כשל אחר, יתבצע break ללולאה וה-thread יסגר. מחזירה; לא מחזירה ערכים		
ביצוע; פעולת ביניים מתווכת - מקבלת מ-handle_client את המידע מהלקוח, בודקת את סוג הפעולה המצוין במידע, וקוראת לפעולה המטפלת בסוג הבקשה. מחזירה; המידע המתקבל מהפעולה המטפלת בסוג הבקשה.	data	process_request
ביצוע; בודקת האם משתמש קיבל הודעת יום הולדת בתאריך ההרצה מחזירה; True - קיבל, אחרת False * כל עוד אין תקלה במסד הנתונים, אחרת תיחיה exception, לא תיחיה קריסה, הפעולה המזמנת תתפוס אותה.	recipient	twice_birthday_today
ביצוע; מעדכנת במסד הנתונים אפיונים שהמשתמש עשה להודעות, משתמשים חסומים, רשימות תפוצה, שמות Labels ומספר הפרות. מבצעת בדיקה האם משתמש צריך להיחסם. כמו כן, מעדכנת את ההודעות כלא נקראו. מחזירה; מוחזרת הודעה לגבי האם ההתנתקות הצליחה או לא - תקלה במסד נתוני השרת.	data	logout
ביצוע; מוציאה ממסד הנתונים את השאלות אבטחה שעליהם המשתמש ענה בהתאם למייל שלו. מחזירה; הצלחה - מוחזרת הודעה ובה שאלות האבטחה, אי הצלחה - מוחזרת הודעה על כך שהמייל לא קיים / תקלה במסד הנתונים.	data	forgot_password
ביצוע; מקבלת את התשובות לשאלות האבטחה עליהן המשתמש ענה ואת הסיסמא החדשה שהגדיר, בודקת האם התשובות זהות לאלו שבמסד הנתונים, אם כן מעדכנת את הסיסמא הישנה לחדשה שקיבלה. מחזירה; הצלחה - מוחזרת הודעה ובה שאלות האבטחה, אי הצלחה - מוחזרת הודעה על כך שהתשובות לא נכונות / תקלה במסד הנתונים.	data	reset_password
ביצוע; רושמת משתמש למערכת בהתאם לנתונים המתקבלים (עדכון במסד הנתונים) כל עוד המייל המוצא איננו קיים כבר. מחזירה; הצלחה - מוחזרת הודעה על כך שהמשתמש נרשם בהצלחה, אי הצלחה - מוחזרת הודעה על כך שהמייל המוצע כבר קיים / תקלה במסד הנתונים.	data	register_user
ביצוע; בדיקה האם יש התאמה בין הסיסמא שהמשתמש הזין בצד הלקוח והערך המגובב של הסיסמא הקיימת במסד הנתונים (הסיסמא שהוגדרה בעת ההרשמה של המשתמש עם כתובת המייל שהוזן בצד הלקוח). במקרה של התאמה, בודקת אם יש למשתמש יום הולדת ביום ההתחברות ואם לא נשלחה לו כבר היום הודעת יום הולדת (פונקציית "twice_birthday_today"), אם שני תנאים אלו חיוביים תכניס למסד נתוני השרת מייל "מזל טוב" מעת החשבון הרשמי של השירות. מחזירה; התאמה - מוחזרת הודעה שההתחברות אושרה ומידע נוסף, אי התאמה / הצלחה - מוחזרת הודעה כי המייל לא קיים או כי סיסמא לא נכונה או שמוחזרת הודעה כי יש תקלה במסד הנתונים.	data	login_user
ביצוע; בודקת האם כתובת מייל קיימת במסד הנתונים מחזירה; קיימת True, אחרת False. * כל עוד אין תקלה במסד הנתונים, אחרת תיחיה exception, לא תיחיה קריסה, הפעולה המזמנת תתפוס אותה.	email	email_exits
ביצוע; שומרת מייל חדש למסד הנתונים בהתאם למידע המתקבל.	data	receive_message

מחזירה ; הצלחה - הודעה האם המייל נשלח בהצלחה - נשמר במסד, אי הצלחה - מחזירה הודעה ובה כי הכתובת אליה נשלח המייל לא קיימת, או כי יש תקלה במסד הנתונים.		
ביצוע ; מוציאה ממסד נתוני השרת את כל המיילים שלא נקראו ע"י המשתמש (כלומר הקיימים במסד הנתונים, שייכים למשתמש המבקש ומסומנים בשדה "read" כ-0). כמו כן, מעדכנת את ההודעות כנקראו כדי לא לשלוח כמה פעמים את אותה ההודעה לצד הלקוח. מחזירה ; הצלחה - הודעה ובה רשימה המכילה את כל ההודעות שנשלחו לכתובת המתבקשת כמילונים שסומנו כלא נקראו. אי הצלחה - הודעה על כך שיש תקלה במסד (צד הלקוח מטפל במקרה שאין הודעות חדשות שלא נקראו).	data	retrieve_message

3. שם המחלקה; ServiceMainClass

תפקיד המחלקה; מחלקת "האם" של הפרויקט, במחלקה זו ישנן פעולות הקשורות לכלל התנהלות השירות, הן בצד השרת והן בצד הלקוח ודברים אשר אין צורך לשנות בקוד הלקוח / שרת עצמו, אלא עדיף שישתנו דרכה. *שתי המחלקות MailServer ו-MailClient יורשות ממנה.

תכונות המחלקה;

שם התכונה	תפקיד ושימוש
host	אחסון את כתובת ה-IP של השרת
port	אחסון את מספר ה-port של השרת
ph	אחסון מופע של מחלקת argon2 לצורך ביצוע גיבוב
forbidden_characters	אחסון כלל התווים האסורים להזנה ב-entries מסוימים.
DaVinciQoutes	אחסון ציטוטים שונים של לאונרדו דה וינצ'י

פעולות המחלקה;

שם הפעולה	טענת כניסה	טענת יציאה
encrypt_message	data, aes_key, client_iv, shifting_bytes_num	ביצוע ; המרת המידע לייצוג באמצעות Json, המרה המידע לייצוג באמצעות בתיים, הצפנת המידע באמצעות מפתח ה-AES וה-IV (מתבצע באמצעות פעולה נוספת), לאחר מכן מתבצעת הזזת הביטים בכל בית (מתבצע באמצעות פעולה נוספת) ולבסוף מתבצע דחיסה. מחזירה ; המידע לאחר תום התהליך המתואר.

decrypt_message	data ,aes_key, client_iv, shifting_bytes_num	ביצוע ; חילוץ המידע, הזזת הביטים בחזרה (מתבצע באמצעות פעולה נוספת), פענוח המידע שהוצפן באמצעות מפתח ה-AES וה-IV והמרת המידע בחזרה למבנה נתונים של python. מחזירה ; מבנה הנתונים שנשלח מהלקוח.
hash_password	password	ביצוע ; הסיסמא המצורפת עוברת גיבוב באמצעות ספרייט argon2. מחזירה ; string אשר נוצר ע"י המחלקה המייצג בין היתר את הסיסמא לאחר גיבוב, ה-"salt" ופרמטרים נוספים המאפשרים בהמשך לבדוק האם סיסמא מוצעת בעת התחברות תואמת לסיסמא שעברה גיבוב.
verify_password	regular_password, hashehd_password	ביצוע ; בודקת האם הסיסמא שהתקבלה זהה לסיסמא לפני שעברה גיבוב. מחזירה ; האם הסיסמאות זהות.
is_birthday_today	birth_date_str	ביצוע ; בודקת האם למי שנוולד בתאריך אותו היא קיבלה יש יומולדת ביום ההרצה. מחזירה ; True/False
strong_passowrd	password	ביצוע ; בודקת האם סיסמא הינה חזקה. מחזירה ; True או הודעה, לסיבה לחולשת הסיסמא.
check_content	content	ביצוע ; בודקת האם התוכן נקי ממילים פוגעניות (באמצעות רשימת מילים שהגדרתי מראש). מחזירה ; True/False

4. שם המחלקה; HybridEncryption

תפקיד המחלקה; נתינת שירותי הצפנה באמצעות RSA ובאמצעות AES, הצפנת ה-RSA ממומשת באופן עצמאי ואילו הצפנת ה-AES ממומשת באמצעות ספרייט cryptography. **תכונות המחלקה**;

שם התכונה	תפקיד ושימוש
default_block_size	אחסון גודל הבלוק הסטנדרטי
byte_size	אחסון גודל כל הבית

פעולות המחלקה;

שם הפעולה	טענת כניסה	טענת יציאה
getBlocksFromText	message	ביצוע ; ממיר string לרשימת בלוקי מספרים מחזירה ; רשימת בלוקי המספרים
getTextFromBlocks	blockInts, messageLength	ביצוע ; ממיר את רשימת בלוקי המספרים ל-string המקורי מחזירה ; ה-string המקורי
readKeyFile	keyFilename	ביצוע ; קוראת קובץ המכיל מפתח ציבורי / פרטי מחזירה ; אורך המפתח ו-tuple המכיל את n ו-d עבור מפתח פרטי ואת n ו-e עבור מפתח ציבורי.
encrypt_blocks	message, key	ביצוע ; ממיר את הודעת ה-string לרשימת בלוקי מספרים באמצעות פעולת "getBlocksFromText" ומצפין כל בלוק. מחזירה ; רשימת הבלוקים המוצפנים.
encrypt_message_RSA	keyFilename, message	ביצוע ; קוראת את המפתח הציבורי באמצעות פעולת "readKeyFile", מצפינה את הבלוקים באמצעות פעולת "encrypt_blocks" וממירה את רשימת בלוקי המספרים המוצפנים ל-string אחד ארוך. מחזירה ; מילון עם ה-string הארוך, אורך ההודעה שהוצפנה (המיוצגת על ידי ה-string) וגודל כל בלוק.
decrypt_message_RSA	encryptedContent (המילון (שהפעולה הקודמת החזירה), keyFilename	ביצוע ; קוראת את המפתח הפרטי באמצעות פעולת "readKeyFile", ממירה את המידע לערכים מספריים גדולים, כלומר לבלוקי מספרים, מפענחת את המידע המוצפן בבלוקים באמצעות פעולת "decrypt_blocks" שממירה ל-string אשר הוצפן. מחזירה ; את ה-string אשר הוצפן.
decrypt_blocks	encryptedBlocks, messageLength, key (tuple)	ביצוע ; מפענחת את המידע המוצפן בבלוקים ומקבלת את characters ה-string מהבלוקים המפוענחים באמצעות פעולת "getTextFromBlocks". מחזירה ; את ה-string אשר הוצפן.
write_file_content	file_path, content	ביצוע ; כותבת לקובץ נתון מחזירה ; לא מחזירה ערכים
read_file	file_path	ביצוע ; קוראת קובץ נתון מחזירה ; תוכן הקובץ
save_to_downloads	content, file_name (אופציונלי)	ביצוע ; יוצרת קובץ, כותבת לתוכו את

התוכן המצורף ושומרת אותו בתיקיית ההורדות. מחזירה ; את ה-path של הקובץ.		
ביצוע ; יצירת מפתח AES מסוג 256 bits מחזירה ; המפתח שנוצר	אין	generate_aes_key
ביצוע ; יצירת IV למפתח AES מסוג 256 bits מחזירה ; ה-IV שנוצר	אין	generate_aes_iv
ביצוע ; מצפינה את המידע באמצעות המפתח וה-IV. מחזירה ; המידע המוצפן	message_bytes, aes_key, iv	encrypt_message_AES
ביצוע ; מפענחת את המידע באמצעות המפתח וה-IV. מחזירה ; המידע המפוענח	ciphertext, aes_key, iv	decrypt_message_AES
ביצוע ; מזיזה את הביטים בכל בית באופן מעגלי שמאלה לפי המספר המתקבל (בין 1 ל-7) מחזירה ; המידע הבינארי לאחר ההזזה	data_bytes, shift_amount	custom_encrypt_shift
ביצוע ; מזיזה את הביטים בכל בית באופן מעגלי ימינה לפי המספר המתקבל (בין 1 ל-7) מחזירה ; המידע הבינארי לאחר ההזזה	encrypted_bytes, shift_amount	custom_decrypt_shift

5. **מחלקות gui** - בנוסף לכלל המחלקות הנ"ל ישנן כשמונה מחלקות gui, אחת לכל מסך;

- LoginFrame
- LoadingFrame
- RegisterFrame
- PasswordRecoveryWindow
- SettingsFrame
- PolicyFrame
- MainFrame
- ComposeWindow

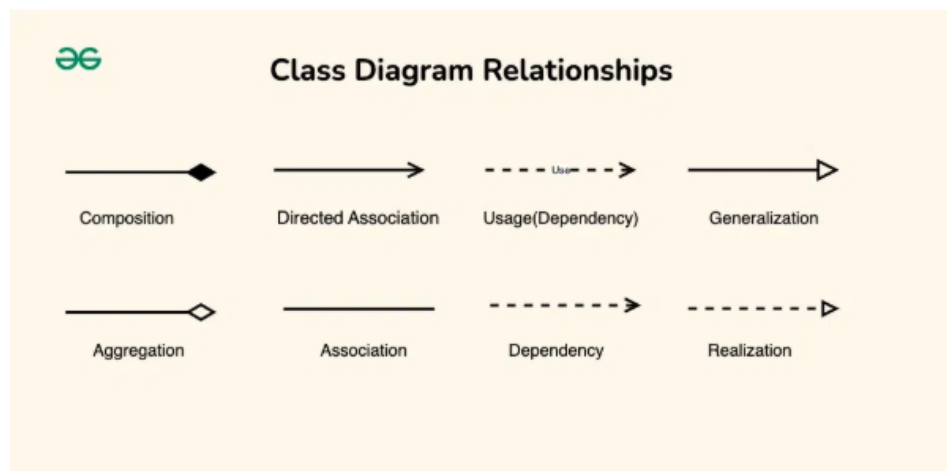
גיא משה טארנטו - DaVinci Mail

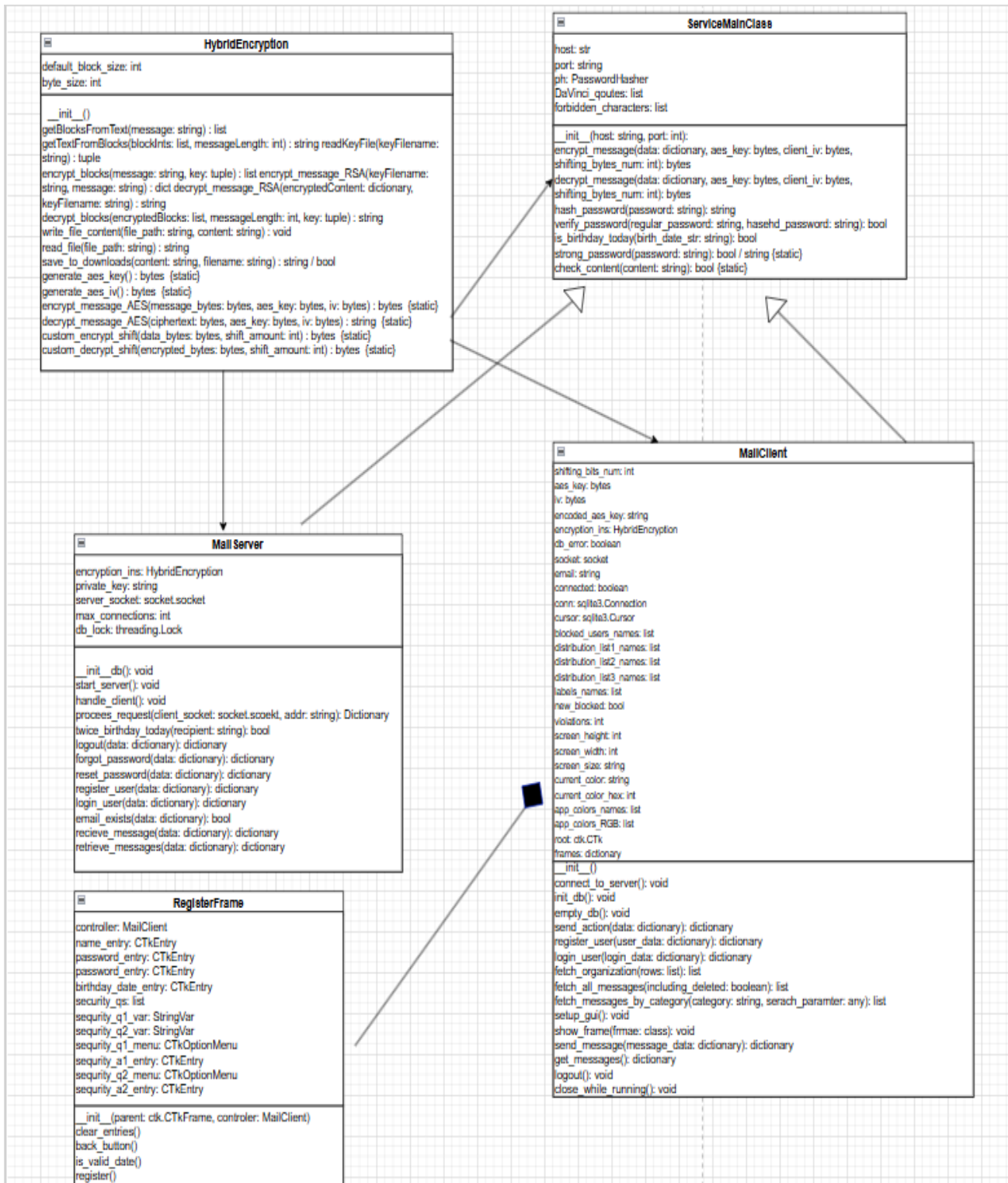
מצפייה בספרים משנים קודמות ומהתייעצות עם מורי המגמה נאמר לי כי אין צורך לפרט על תכונות ופעולות מחלקות אלו. חלק מהתכונות והפעולות חוזרות על עצמן, חלקן אחראיות על עיצוב ה-gui וחלקן לוקחות מידע מ-entries או תיבת טקסט, מבצעות בדיקות שונות (כמו תווים אסורים, מבנה נכון של מייל ועוד...). ויוצרות בסיס להודעות כחלק מהתהליכים השונים (מילון עם המידע להשלמת הפעולות). במחלקת MailClient ישנן את הפעולות הממשיכות את תהליכים אלו כפי שהסברתי בטבלאות לעיל. כל זה תקף למעט שתי פעולות במחלקת "PasswordRecoveryWindow" עליהן אפרט;

שם הפעולה	טענת כניסה	טענת יציאה
submit_email	message	ביצוע ; קולטת את המידע מה-entry להזנת המייל, בונה מילון עם המייל ועם הפעולה "forgot password", מעבירה את המילון לפעולת "send_action" במחלקת MailClient על מנת שתעבירה לשרת. אם אין חיבור לשרת או שהמייל שהוזן לא בפורמט השירות, לא תתקדם בתהליך ותציג הודעה מתאימה. מחזירה ; לא מחזירה ערכים
reset_password	blockInts, messageLength	ביצוע ; קולטת את המידע מה-entries השונים, בונה מילון עם הפעולה "reset_password", מעבירה את המילון לפעולת "send_action" במחלקת MailClient על מנת שתעבירה לשרת. אם אין חיבור לשרת, סיסמא לא חזקה ועוד, לא תתקדם בתהליך ותציג הודעה מתאימה. מחזירה ; לא מחזירה ערכים.

תרשים המחלקות וקשריהם (למעט מחלקות ה-gui, שמתי אחת לדוגמה)

* הקשרים בין המחלקות מיוצגים בחצים בהתאם לפורמט המקובל הבא; נלקח מהאתר "geeks for geeks", הכתובת מופיעה בביבליוגרפיה:





מחלקת ServiceMainClass:

ServiceMainClass
host: str port: string ph: PasswordHasher DaVinci_quotes: list forbidden_characters: list
__init__(host: string, port: int): encrypt_message(data: dictionary, aes_key: bytes, client_iv: bytes, shifting_bytes_num: int): bytes decrypt_message(data: dictionary, aes_key: bytes, client_iv: bytes, shifting_bytes_num: int): bytes hash_password(password: string): string verify_password(regular_password: string, hasehd_password: string): bool is_birthday_today(birth_date_str: string): bool strong_password(password: string): bool / string {static} check_content(content: string): bool {static}

מחלקת HybridEncryption:

HybridEncryption
default_block_size: int byte_size: int
__init__() getBlocksFromText(message: string) : list getTextFromBlocks(blockInts: list, messageLength: int) : string readKeyFile(keyFilename: string) : tuple encrypt_blocks(message: string, key: tuple) : list encrypt_message_RSA(keyFilename: string, message: string) : dict decrypt_message_RSA(encryptedContent: dictionary, keyFilename: string) : string decrypt_blocks(encryptedBlocks: list, messageLength: int, key: tuple) : string write_file_content(file_path: string, content: string) : void read_file(file_path: string) : string save_to_downloads(content: string, filename: string) : string / bool generate_aes_key() : bytes {static} generate_aes_iv() : bytes {static} encrypt_message_AES(message_bytes: bytes, aes_key: bytes, iv: bytes) : bytes {static} decrypt_message_AES(ciphertext: bytes, aes_key: bytes, iv: bytes) : string {static} custom_encrypt_shift(data_bytes: bytes, shift_amount: int) : bytes {static} custom_decrypt_shift(encrypted_bytes: bytes, shift_amount: int) : bytes {static}

מחלקת MailServer:

MailServer
encryption_ins: HybridEncryption private_key: string server_socket: socket.socket max_connections: int db_lock: threading.Lock
__init_db(): void start_server(): void handle_client(): void process_request(client_socket: socket.socket, addr: string): Dictionary twice_birthday_today(recipient: string): bool logout(data: dictionary): dictionary forgot_password(data: dictionary): dictionary reset_password(data: dictionary): dictionary register_user(data: dictionary): dictionary login_user(data: dictionary): dictionary email_exists(data: dictionary): bool receive_message(data: dictionary): dictionary retrieve_messages(data: dictionary): dictionary

מחלקת RegisterFrame:

RegisterFrame
controller: MailClient name_entry: CTkEntry password_entry: CTkEntry password_entry: CTkEntry birthday_date_entry: CTkEntry security_qs: list security_q1_var: StringVar security_q2_var: StringVar security_q1_menu: CTkOptionMenu security_a1_entry: CTkEntry security_q2_menu: CTkOptionMenu security_a2_entry: CTkEntry
__init__(parent: ctk.CTkFrame, controller: MailClient) clear_entries() back_button() is_valid_date() register()

MailClient	
shifting_bits_num: int	
aes_key: bytes	
iv: bytes	
encoded_aes_key: string	
encryption_ins: HybridEncryption	
db_error: boolean	
socket: socket	
email: string	
connected: boolean	
conn: sqlite3.Connection	
cursor: sqlite3.Cursor	
blocked_users_names: list	
distribution_list1_names: list	
distribution_list2_names: list	
distribution_list3_names: list	
labels_names: list	
new_blocked: bool	
violations: int	
screen_height: int	
screen_width: int	
screen_size: string	
current_color: string	
current_color_hex: int	
app_colors_names: list	
app_colors_RGB: list	
root: ctk.CTk	
frames: dictionary	
__init__()	
connect_to_server(): void	
init_db(): void	
empty_db(): void	
send_action(data: dictionary): dictionary	
register_user(user_data: dictionary): dictionary	
login_user(login_data: dictionary): dictionary	
fetch_organization(rows: list): list	
fetch_all_messages(including_deleted: boolean): list	
fetch_messages_by_category(category: string, search_paramter: any): list	
setup_gui(): void	
show_frame(frmae: class): void	
send_message(message_data: dictionary): dictionary	
get_messages(): dictionary	
logout(): void	
close_while_running(): void	

חלק ב':

אלגוריתם ראשון - גיבוב נתונים;

כפי שהסברתי הגיבוב נעשה באמצעות ספריית argon2, ספרייה זוכת תחרויות גיבוב, הפעולה "hash_password" תחזיר string שהספרייה יצרה, המכיל בתוכו את הערך המגובב של הסיסמא שהתקבלה, את ה-salt שלה ופרטים נוספים. פונקציית הגיבוב;

```
def hash_password(self,password):
    '''
    hashing password using argon2 library which is will recomended library
    for hashing password whom even some contests in password hashing
    :param password:
    :return the hashed password as string:
    '''
    hashed_password = self.ph.hash(password)
    return hashed_password
```

פונקציית בדיקת ההתאמה;

```
def verify_password(self,regular_password,hasehd_password):
    '''
    verify if password is the same as hashed password, which hashed with
    argon2 library.
    :param regular_password - the password itself:
    :param hasehd_passowrd:
    :return are the hashed and the regular passwords are the same?:
    '''
    try:
        self.ph.verify(hasehd_password[0], regular_password)
        return True
    except VerifyMismatchError:
        return False
```

אלגוריתם שני - אבטחת תעבורת התקשורת בין השרת והלקוח ולהפך;

הקודים המעריבים את ההצפנה הינם ספריית האבטחה של השירות EncryptionLib, ה-handshake של האפליקציה (העברת המפתח הציבורי של השרת ללקוח ושמירתו במחשב הלקוח, הצפנתו של מפתח ה-AES באמצעות המפתח הציבורי ושליחתו לשרת בחזרה) ושתי פעולות במחלק ServiceMainClass, כלל הדברים הללו מהווים כמה מאות שורות קוד, לכן לא אצרפם כעת, הם יופיעו עם כלל הקוד בסוף הספר.

אלגוריתם שלישי - שליחת ההודעה בדרך שתוכנה לא השתבש ובצורה יעילה;

*שתי הפעולות הנ"ל מופעלות לכלל הודעות בתעבורת התקשורת של השירות למעט ההודעות ב-Handshake שלו. מצורפת הפעולה אשר מכינה את ההודעה לשליחה;

```
def prepare_message(self, data, aes_key, client_iv, shifting_bytes_num):
    '''
    preparing the message; convert to json, encode the json data to bytes,
    encrypting the bytes using EncryptionLib library.
    Finnally, compress the encrypted bytes.
    :param data:
    :param aes_key:
    :param client_iv:
    :param shifting_bytes_num:
```

```
:return the encrypted and compressed data:
'''
    json_data = json.dumps(data) # convert dict to JSON string
    bytes_data = json_data.encode('utf-8') # encoding the JSON data to
bytes
    encrypted_data = HybridEncryption.encrypt_message_AES(bytes_data,
aes_key,client_iv) # encrypting the data
    shifted_bytes_data =
HybridEncryption.custom_encrypt_shift(encrypted_data, shifting_bytes_num)
# shifting the bits
    compressed_data = gzip.compress(shifted_bytes_data) # compress the
encrypted bytes
    return compressed_data
```

מצורפת הפעולה אשר ממירה את ההודעה המתקבלת בחזרה לטיפוס ה-python שהצד השני שלח;

```
def treat_message(self, data, aes_key, client_iv, shifting_bytes_num):
    '''
        treating the message; decompress, decrypting the bytes using
EncryptionLib library, and then converts from json.
        :param data:
        :param aes_key:
        :param client_iv:
        :param shifting_bytes_num:
        :return the dictionary which the server / clients sent:
    '''
    decompressed_data = gzip.decompress(data) # decompress the data
    decrypted_data =
HybridEncryption.custom_decrypt_shift(decompressed_data,shifting_bytes_num
) # reshifting the bits
    decrypted_data =
HybridEncryption.decrypt_message_AES(decrypted_data,aes_key, client_iv) #
decrypting the data
    response = json.loads(decrypted_data) # Convert JSON string back to
dictionary
    return response
```

גיא משה טארנטו - DaVinci Mail

אלגוריתם רביעי- מניעת מתקפת DOS/DDOS;
הקוד הממש את הדיליי פונקציה המעבירה את בקשות הלקוח לשרת;

```
def send_action(self, data):
    '''
    Send a request to the server and receive a response; activates the
    delay mechanism, generates a new IV, call the function whom encrypt and
    compress the data,
    sends the generated iv, the encrypted and compressed data length, then
    receive the response length, after that the response itself, then call the
    function whom decrypts
    and decompress the data - getting the response form the server.
    :param data: The data to be sent to the server.
    :return: The response from the server.
    '''
    try:
        time.sleep(random.uniform(0.1, 0.4))
```

רק לאחר סיום "השינה" נתקדם לקטע הקוד השולח ההודעה לשרת,
*יש לפונקציה המשך, לא הכרחי להציגו.
הגבלת מספר הלקוחות בשרת;

```
def start_server(self):
    '''
    Start the server, listen for incoming connections, and handle each
    client connection in a new thread.
    :return:
    '''
    self.server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    self.server_socket.bind((self.host, self.port))
    self.server_socket.listen()
    print("[STARTED] Server is running and waiting for connections...")
    while True:
        if (threading.active_count() - 1) < self.max_connections:
            client_socket, addr = self.server_socket.accept()
            thread = threading.Thread(target=self.handle_client,
            args=(client_socket, addr))
            thread.start()

            print(f"[ACTIVE CONNECTIONS] {threading.active_count() - 1}")
        else:
            print(f"[CONNECTION LIMIT REACHED] Maximum of
            {self.max_connections} active connections. New connection attempt
            rejected.")
            client_socket, addr = self.server_socket.accept() # Still need
            to accept to close the connection
            try:
                # A "server busy" message to the client
                json_data = json.dumps({'accept': False}) # Convert dict
                to JSON string
```

ניתן לראות הלולאה הראשית המקבלת חיבורי לקוחות חדשים לשרת, אם מספר ה-threads, מספר
התהליכונים (עם כל לקוח חדש מופעל תהליכון חדש) פחות אחד (השרת עצמו הינו תהליכון, לכן נחסיר
אותו) קטן ממספר המשתמשים המקסימלי נקבל את המשתמש. אחרת נשלח לו הודעה כי לא ניתן
לקבלו ונסגור את החיבור עימו. *יש לפונקציה המשך, מיותר כעת להציגו.

גיא משה טארנטו - DaVinci Mail

אלגוריתם חמישי - ניהול כלל נתוני השירות;
מצורף הקוד היוצר את המסד הנתונים הזמני בצד הלקוח;

```
def init_db(self):
    '''
    Initialize the SQLite database and create required table if doesn't
    exist.
    :return:
    '''
    try:
        self.conn = sqlite3.connect('user_messages3.db',
check_same_thread=False)
        self.cursor = self.conn.cursor()
        self.cursor.execute('''CREATE TABLE IF NOT EXISTS messages (
                                id INTEGER PRIMARY KEY
                                AUTOINCREMENT,
                                sender TEXT,
                                title TEXT,
                                message TEXT,
                                timestamp TEXT,
                                important INT DEFAULT 0,
                                readlater INT DEFAULT 0,
                                deleted INT DEFAULT 0,
                                file_data TEXT,
                                image_data TEXT,
                                labels TEXT
                                )''')
        self.conn.commit()
    except:
        self.db_error=True
        messagebox.showerror("Data Base Error", "There is problem in the
service's Data Base, please log out, and try to connect later")
```

אלגוריתם שישי - ניהול כלל נתוני השירות;
מצורפת הפונקציה אשר משמשת כ"מתווכת", קוראת לפונקציה המתאימה בהתאם לפעולה בבקשת
הלקוח.

```
def process_request(self, data):
    '''
    Process the incoming request and call the appropriate action.
    :param data: The data received from the client.
    :return: A dictionary containing the response to the client.
    '''
    action = data['action']
    if action == 'register':
        return self.register_user(data)
    elif action == 'login':
        return self.login_user(data)
    elif action == 'send_message':
        return self.receive_message(data)
    elif action == 'retrieve_messages':
        return self.retrieve_messages(data)
    elif action == 'forgot_password':
        return self.forgot_password(data)
```

```
elif action == 'reset_password':
    return self.reset_password(data)
elif action == 'logout':
    return self.logout(data)
elif action == "rsa public key":
    return {'status': 'success', 'public_key':
self.public_key.encode()}
else:
    return {'status': 'error', 'message': 'Invalid action.'}
```

אלגוריתם שביעי - 'Handshake' השירות;

מצורף הקוד בצד הלקוח האחראי על ההתחברות לשרת;

```
def connect_to_server(self):
    """
    Connect to the mail server using the specified host and port via TCP
    protocol, if the connection is not accepted; updates the client and close
    the app. If the connection is accepted; gets the sever's public key,
    save it and then encrypts the client's AES key and bit shifting num, after
    that -
    send them to the server. If the server isn't available there will be
    endless loop which every 15 seconds will try to connect to the server,
    unless the user will close the app.
    if the saving of the server's public key wasn't succeeded there will be
    a message to close the app.
    :return Nothing:
    """
    try: # service 'Handshake'
        # connect to server
        self.socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.socket.connect((self.host, self.port))
        # gets if the server can get more users
        length = self.socket.recv(4)
        length = struct.unpack('>I', length)[0]
        response = self.socket.recv(length)
        response = gzip.decompress(response).decode('utf-8')
        response = json.loads(response)
        if response['accept']==True: # the server can get more users
            # gets the server's public key
            length = self.socket.recv(4)
            length = struct.unpack('>I', length)[0]
            response=self.socket.recv(length)
            json_data = gzip.decompress(response).decode('utf-8')
            sever_public_key = json.loads(json_data)
            # saves the key in the downloads folder and gets the path
            full_file_path=self.encryption_ins.save_to_downloads(sever_public_key)
            if full_file_path==False: # There was error in the save
            process, the app can't keep running
                raise
            # encrypting the client's aes key and then sending it.
            encrypted_aes_key=self.encryption_ins.encrypt_message_RSA(full_file_path,s
            elf.encoded_aes_key)
```

```

        # adding a number to the shifting bits in each byte num
        according to the ip address of the host in order to encrypt the shifting
        num
        ip_parts_str = self.host.split('.') # splits the ip address of
        the host for four string in a list
        ip_tuple_int = tuple(int(part) for part in ip_parts_str) #
        convert them to int
        sum=0
        for i in ip_tuple_int:
            sum+=((i*7)+13)*2
        # sending the client encrypted AES key and the encrypted bits
        shifting num
        data = {'encrypted_key': encrypted_aes_key, 'num':
        (self.shifting_bits_num + sum)}
        json_data = json.dumps(data) # Convert dict to JSON string
        compressed_data = gzip.compress(json_data.encode('utf-8')) #
        Compress the JSON string
        length_bytes = struct.pack('>I',len(compressed_data)) # Pack
        length as a big-endian unsigned integer (4 bytes)
        self.socket.sendall(length_bytes) # sends the len of the
        message in bytes
        self.socket.sendall(compressed_data) # sends the message itself
        self.connected = True
    else:
        messagebox.showinfo("Server Limit","We are sorry, but the
        server has too many connections, it can't serve you right now.")
        self.root.destroy()
    except (ConnectionResetError, BrokenPipeError, socket.error):
        messagebox.showerror("Connection Error", "Unable to connect to the
        server, Retrying in 15 seconds...")
        time.sleep(15)
        self.connect_to_server()
    except:
        messagebox.showerror("Error", "Error in saving necessary data,
        please close the app and restart your device")

```

מצורף הקוד בצד השרת האחראי על התחברות הלקוח במידה ואושרה, (קטע הקוד מראה גם את האישור לבדיקת החיבור באלגוריתם הרביעי - פעולת "start_server", כפי שציינתי); *יש לפעולה המשך, מיותר כעת להציגו.

```

def handle_client(self, client_socket, addr):
    '''
    Handle communication with a connected client, first there are few steps
    for data security - sending the public key for each new connected client,
    then gets from the client its AES key and number of bits shifting num
    (which is random for each client for more advanced protection).
    After this procedure there is endless lupe (until logout or closing the
    app in the clients side) which have the repeated procedure;
    :param client_socket: The socket object representing the client
    connection.
    :param addr: The address of the connected client.
    '''
    connected=True
    try: # service 'Handshake'

```

```

print(f"[NEW CONNECTION] {addr} connected.")
# sending that server can accept new clients
json_data = json.dumps({'accept': True}) # Convert dict to JSON
string
compressed_data = gzip.compress(json_data.encode('utf-8')) #
Compress the JSON string
length_bytes = struct.pack('>I', len(compressed_data)) # Pack
length as a big-endian unsigned integer (4 bytes)
client_socket.sendall(length_bytes)
client_socket.sendall(compressed_data)
# sending the server's public key to the user
json_data = json.dumps(self.public_key) # Convert the key to JSON
string
compressed_data = gzip.compress(json_data.encode('utf-8')) #
Compress the JSON string
length_bytes = struct.pack('>I', len(compressed_data)) # Pack
length as a big-endian unsigned integer (4 bytes)
client_socket.sendall(length_bytes)
client_socket.sendall(compressed_data)
# receives the client's aes key and decrypt it to bytes
length = client_socket.recv(4)
length = struct.unpack('>I', length)[0]
message = client_socket.recv(length)
data = gzip.decompress(message) # Decompress and convert back to
string
data = json.loads(data)
encrypted_aes_key = data['encrypted_key']
# decrypt the AES key
encoded_aes_key =
self.encryption_ins.decrypt_message_RSA(encrypted_aes_key,
"private_key.txt")
# decodes the AES key to bytes
aes_key_bytes = base64.b64decode(encoded_aes_key)
# decrypt the bits shifting num according to the server ip
ip_parts_str = self.host.split('.') # splits the ip address of the
host for four
ip_tuple_int = tuple(int(part) for part in ip_parts_str) # convert
them to int
sum = 0
for i in ip_tuple_int:
    sum += ((i * 7) + 13) * 2
bytes_shifting_num = data['num'] - sum
except (ConnectionResetError, BrokenPipeError, socket.error):
    connected=False

```

שתי הפעולות הנ"ל מייצגות את תהליך ה-Handshake של השירות, משני הצדדים השונים - לקוח ושרת.

חלק ג'

מסמך בדיקות מלא;

שם הבדיקה	מטרת הבדיקה	ביצוע	תוצאות	בעיות ופתרון
העברת מידע באופן תקין	לבדוק שכלל תהליך העברת המידע בין השרת ללקוח הינו תקין.	הדפסה המידע לפני שליחתו ולפני העברתו את תהליכי העיבוד השונים (כמו, המרה ל-json), הדפסת הייצוגים השונים של המידע בין כל אחד משלבי העיבוד השונים בצד השולח ולאחר מכן בתהליכי אי-העיבוד בצד המקבל. בדיקה כי ההודעה בשפת פרוטוקול התקשורת של השירות שנשלחה מהלקוח לשרת ולהפך הינה זהה לחלוטין בשני הצדדים. זאת ועוד, ישנם גם את צעדי שליחת ה-IV הייחודי לכל הודעה ושליחת אורך ההודעה לפניה. יתר על כן, באמצעות צפייה ב-wireshark ראיתי שהתקשורת עוברת בסדר שהגדרתי ובאופן תקין.	תחילה היו כמה שגיאות, לבסוף התאמה מלאה!	לפני מימוש הקוד קראתי על הנושא, שכלל גם את בניית ספריית ההצפנה, התחלתי לממש את הקוד לאט ובהירות, היו כמה תקלות פה ושם שנבעו מטעויות של מתחילים, מעולם לא ביצעתי הצפנה בעצמי (בהתבסס על קוד המורה שסופק ובהתאמה לקוד שלי) וגם לא באמצעות ספרייה, לכן לקח לי יחסית זמן לרשום את הכל, אבל לבסוף הצלחתי, ההשקעה משתלמת.
בדיקת שההגנות אכן עובדות	לדמות מתקפת Man in the middle וניסיון sql injection	באמצעות צפייה ב-wireshark ראיתי שתעבורת התקשורת שאותה הצפנתי אכן לא חשופה, ואף ניסיתי לתת ל-AI לפצח ולשמחתי הוא אמר לי שבלי שום מידע נוסף (על אופי ההצפנה) הוא לא יוכל לפצח את המידע המוצפן והדחוס מתעבורת התקשורת שנתתי לו. כמון כן, ניסיתי לבצע SQL injection על התוכנה, באמצעות הכנסת קלטים שונים כמו -1=1 OR, אך לשמחתי קיבלתי הודעה תקינה כי המייל לא קיים / סיסמא לא נכונה.	הצלחה מלאה.	הצלחה מלאה. לפני תחילת העבודה עם מסד הנתונים קראתי על איך לעבוד איתו בצורה המונעת sql injection, העברת קלט המשתמש כפרמטרים (כפי שהסברתי כבר).
תפקוד מסד הנתונים	כלל המידע נשמר במסד באופן תקין, בהתאם לפעולות המשתמשים השונים.	הורדתי את תוכנת "db browser for sqlite", כך יכולתי לפתוח כל פעם את מסדי הנתונים של השרת או של הלקוח, לראות ששינויים אכן בוצעו כנדרש ובאופן תקין.	שגיאות בדרך, בסוף הצלחה.	בשל טעויות syntax שקרו מידי פעם בכתיבת ה-queries השונים הייתה לי תופעה מוזרה, היו מקרים שבהם מסד הנתונים לא החזיר שום תקלה למרות שה-query היה לא תקין (למשל שגיאת כתיב בשם של שדה, שכתתי לשם סימן שאלה וכדומה), ביצעתי בדיקות וראיתי שכל שאר התהליך היה תקין (בצד הלקוח או השרת). לא הבנתי מדוע המידע לא מתעדכן / מתקבל ממסד הנתונים, לפעמים אחרי לא מעט זמן בניסיון למציאת

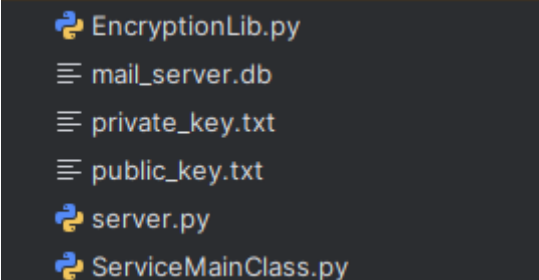
				גורם הבאג הבנתי שבניסוח ה-query הייתה שגיאה. מאז אותם מקרים, כל פעם שכתבתי query כלשהוא עברתי עליו בקפידות יתרה. יתרה מכך, ניסיתי לשמור מידע המיוצג באמצעות בתים במסד הנתונים, קיבלתי תקלה ולמדתי כי דבר זה איננו אפשרי במסד נתונים מסוג SQLite.
שמירת מידע אישי לאחר התנתקות ובדיקת תקינות קלט	האם לאחר התנתקות המשתמש בשתי הדרכים השונות - לחיצה על כפתור "X" או לחיצה על כפתור logout במסך הבית, המידע נשלח באופן תקין ומתעדכן במסד הנתונים. יתרה מכך, האם בעת התחברות מחדש כל המידע מוצג לפי הסינונים שהמשתמש ביצע. בנוסף, בדיקה האם תווים מסוימים בקלטים מסוימים של המשתמש יכולים להזיק לתפקוד השירות.	באמצעות תוכנת DB browser יכולתי לראות האם המידע מתעדכן באופן תקין במסד נתונים בעת התנתקות, והאם המידע מתעדכן באופן תקין במסד נתונים הלקוח בעת התחברות משתמש. זאת ועוד, בדקתי כי ה-gui מתעדכן כמו שצריך. כמו כן הגדרתי במחלקת "האם" מחלקת "ServiceMainClass" רשימת תווים אסורים, שבקלטים מסוימים מתבצעת בדיקה האם הלקוח הזינם, במקרה שכן הקלט לא מתקבל ומופיעה חלופית עם הודעה מתאימה.	שגיאות בדרך, בסוף הצלחה.	דוגמה לתקלה הינה שהודעת ההתנתקות הינה הפעולה ההודעה היחידה שאיננה נשלחת דרך פעולת send_action במחלקת MailClient, על מנת שלא יהיה דילי בעבודה (למקרה הקצה החרג שבו יאבד הקשר עם השרת בעת ההמתנה לסיום ביצוע הדילי, ואז ההתנתקות והשינויים שהלקוח ביצע לא יתעדכנו במסד נתונים), לכן ששינתי דברים בתהליך שליחת הודעה (למשל הוספת מנגנון הדחיסה, תחילה לא ביצעתי את כל ההצפנה והדחיסה / פענוח וחילוץ בשתי פעולות המאגדות הכל יחד), ולא הוספתי את השינויים לקוד השולח את בקשת ההתנתקות, תהליך קבלת הודעות ההתנתקות ושליחתן לא היה תקין, ולכן התקבלו errors, ביצעתי תיקון. בנוסף בעת קבלת המידע בהתחברות מחדש לא היו לי תקלות מיוחדות, רק הייתי צריך להתאים את ה-gui לכך, היו כמה פעמים שלא התעדכן כראוי.
פרטיות	האם לאחר התנתקות משתמש א' והתחברות משתמש ב' מבלי לסגור את האפליקציה כל המידע הפרטי שלו לא מוצג / נשמר. בנוסף, שתוכן הלשוניות השונות במסך הפתיחה נמחק לאחר התחברות, ושתוכן הלשוניות במסך ההרשמה נמחק, לאחר ביצוע הרשמה או לאחר חזרה למסך הפתיחה ללא ביצוע הרשמה.	בעת התנתקות משתמש, באופן אוטומטי מסד הנתונים בצד הלקוח נמחק על כל תכולתו, כלל ההודעות המוצגות במסך הראשי נמחקות וגם תכונות שונות של מחלקת MailClient מאותחלות, כמו כן, תוכן הלשוניות במסך ההרשמה נמחק לאחר ביצוע הרשמה / חזרה למסך הפתיחה. תוכן הלשוניות במסך הפתיחה לא זמין למשתמש חדש המתחבר לאחר שמשתמש אחר התנתק ללא סגירת האפליקציה.	הצלחה מלאה.	לא נתקלתי בבעיות.
מקרי קצה	בדיקה לאיזה מקרי קצה שונים עליי להתייחס	שני מקרה הקצה העיקריים אליהם אני מתייחס הם ניתוק לא צפוי בקשר בין השרת ללקוח ותקלה במסד הנתונים, הן של השרת והן של הלקוח. לכן מימשתי מענה לכל אחד ממקרים אלו - ניתוק לא צפוי, למשתמש	שגיאות מינוריות, לבסוף הצלחה מלאה.	לא נתקלתי בבעיות משמעותיות, השקעתי זמן ומחשבה לחסימת כל אפשרות לקלט מזיק או לא תקין מטעם הלקוח.

		תוצג הודעה כי יש בעיית תקשורת וכי יש לכבות את התוכנה עבור כל פעולה שיבצע המערכת תקשורת, ה-thread הקשור ללקוח בשרת יסגר. ביצעתי ניתוק מכוון הן של השרת והן של הלקוח לא באופן תקין (עצירת פתאומית של הרצת קוד השרת, ובמקרה אחר עצירה פתאומית של קוד הלקוח), ראיתי כי ה-exceptions אכן תופסים את שגיאות התקשורת וכי מתקבלות הודעות מתאימות בצד הלקוח (במקרה של התנתקות השרת) וכי thread התקשורת עם הלקוח נסגר (במקרה של התנתקות הלקוח). בנוסף אם יש תקלה במסד נתוני השרת תוחזר הודעה מתאימה ללקוח ותוצג למשתמש חלונת ובה הודעה כי עליו לסגור את התוכנה. אם יש תקלה במסד נתוני הלקוח תוצג למשתמש חלונת ובה הודעה כי יש לסגור את התוכנה. כל פנייה למסד הנתונים מוגנת באמצעות try ו-except. גרמתי לשגיאות מכוונות הן במסד נתוני הלקוח והן במסד נתוני השרת וראיתי כי מוצגות ללקוח הודעות מתאימות. זאת ועוד, מקרה קצה נוסף הינו קלט מזיק או לא תקין מטעם המשתמש לכן יש בדיקות שונות בצד הלקוח בנוגע לתקינות הקלט לפני התחלת תהליכים שונים. מה שמבטיח כי השרת מקבל נתונים תקינים לחלוטין.	
למדנו במגמה לבנות gui באמצעות tkinter ברמה בסיסית. בעקבות זאת, תוך כדי בניית ה-gui בפרויקט שעשיתי ב-costumetkinter (המבוסס על tkinter אבל קצת שונה), נתקלתי בבעיות שונות, למשל שימוש לא נכון באובייקטים שונים של הספרייה (למשל הגדרת פרמטרים שגויים וכו'). לבסוף, אחרי תרגול תוך כדי עבודה הבנתי איך משתמשים ב-custometkinter ואני חושב שעשיתי עבודה טובה.	שגיאות רבות, עקב למידה תוך כדי בניית ה-gui	נתתי למספר חברים ובני משפחה להתנסות במערכת אחרי שבחנתי אותה בעצמי כמובן והקשבתי להארותיהם ולהערותיהם.	בדיקת gui בדיקה שה-gui פועל כשורה, קל ונוח לתפעול ותואם לפעולות המשתמש.

מדריך למשתמש

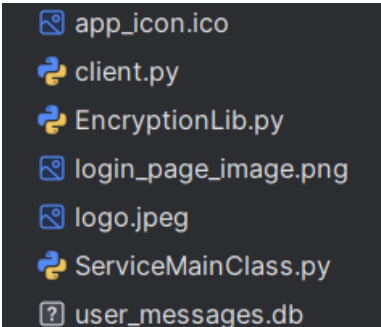
פירוט כלל קבצי המערכת

1. **client.py** - קוד הלקוח (ספריית MailClient וכלל ספריות ה-gui), יש להריץ בצד הלקוח.
 2. **server.py** - קוד השרת, יש להריץ בצד השרת.
 3. **ServiceMainClass.py** - ספריית האם של השירות, צריך להיות הן בצד הלקוח והן בצד השרת.
 4. **EncryptionLib** - ספריית האבטחה של השירות, צריך להיות הן בצד הלקוח והן בצד השרת.
 5. **mail_server.db** - מסד נתוני השירות, צריך להיות בצד השרת בלבד (אם לא זמין ייוצר חדש בהרצת הקוד).
 6. **user_messages.db** - מסד נתוני השירות, צריך להיות בצד הלקוח בלבד (אם לא זמין ייוצר חדש בהרצת הקוד).
 7. **private_key** - מפתח פרטי של השרת, צריך להיות בצד השרת בלבד.
 8. **public_key** - מפתח ציבורי של השרת, צריך להיות בצד השרת בלבד.
 9. **login_page_image.png** - התמונה המוצגת במסך הפתיחה, צריך להיות בצד הלקוח בלבד (יפעל גם בלי).
 10. **logo.jpeg** - התמונה המוצגת במסך הטעינה, צריך להיות בצד הלקוח בלבד (יפעל גם בלי).
 11. **app_iocn.ico** - אייקון התוכנה, צריך להיות בצד הלקוח בלבד (יפעל גם בלי).
 12. **loading_melody.mp3** - המנגינה המושמעת במסך הטעינה, צריך להיות בצד הלקוח בלבד (יפעל גם בלי).
- *יש לשמור את כל הקבצים הקשורים לצד הלקוח תחת אותו namespace וגם את כל הקבצים הקשורים לצד השרת יש לשמור תחת אותו namespace.
- עץ קבצים (שרת):



```
EncryptionLib.py
mail_server.db
private_key.txt
public_key.txt
server.py
ServiceMainClass.py
```

עץ קבצים (לקוח):



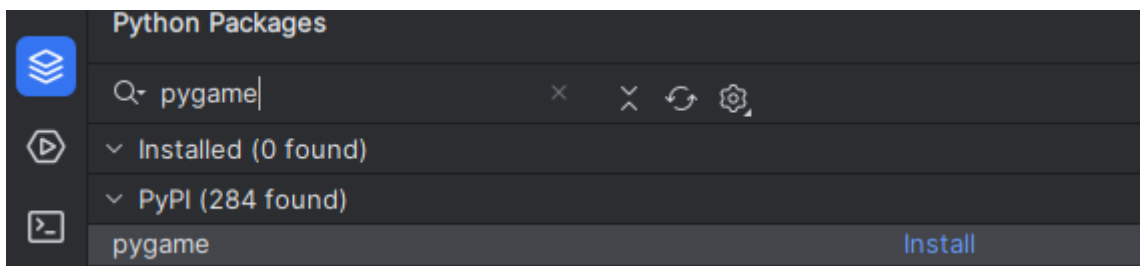
```
app_icon.ico
client.py
EncryptionLib.py
login_page_image.png
logo.jpeg
ServiceMainClass.py
user_messages.db
```

התקנת מערכת;

- **פירוט הסביבה הנדרשת -** יש להתקין python ו-pycharm על המחשב המייצג את השרת, יש לפתוח פרויקט חדש ב-pycharm ולשמור בתיקייה המייצגת אותו את כל הקבצים הדרושים לשרת אשר מוזכרים בפירוט כלל קבצי המערכת. יש להתקין על כל אחד מהמחשבים אשר ישמשו כלקוחות python ו-pycharm, בכל מחשב יש לפתוח פרויקט חדש ב-pycharm ולשמור בתיקייה המייצגת אותו את כל הקבצים הדרושים ללקוח אשר מוזכרים בפירוט כלל קבצי המערכת.
- * הורדת והתקנת pycharm כוללת גם התקנת python.
- * הפרויקט נכתב בשפת python גרסת 3.8.10, ניתן להריץ את הפרויקט בבטחה בכל מחשב שבו מותקנת python בגרסאות 3.8.0 ומעלה.
- על מנת לבדוק את גרסאות ה-python המותקנות במערכות ההפעלה windows10/11 יש לכתוב ב-cmd (פתיחה- כפתור "windows" ולחיצה על "r" במקביל, לאחר מכן בחלונות הנפתחת יש לרשום cmd) את הפקודה python3 --version, או את הפקודה py --version, יכול להיות שיוצג לכל פקודה מספר גרסה שונה (יכול להיות שבוצעו כמה התקנות של שפת python על המחשב הנבדק).
- **פירוט הכלים הנדרשים -**
- * התקנת ספריות, ניתן לרשום ב-cmd את הפקודה pip install library_name, כמו כן ניתן להוריד ספריות דרך pycharm באופן הבא; בצד השמאלי התחתון של התוכנה ישנו את הסרגל הבא;



יש ללחוץ על הכפתור אחד לפני העליון (מסומן בתמונה בכחול) ויפתח התפריט הבא;



בלשונית החיפוש יש לרשום את שם הספרייה, וללחוץ install על הספרייה הרצויה.

צד השרת (בלבד) - יש להוריד במחשב הישמש את צד השרת את הספריות הבאות (אם לא קיימות כבר);

- ☐ threading
- ☐ sys

צד הלקוח (בלבד) - יש להוריד במחשב הישמש את צד הלקוח; את הספריות הבאות (אם לא קיימות כבר);

- ☐ random
- ☐ time
- ☐ pygame

- ☐ customtkinter
- ☐ tkinter
- ☐ PIL

בצד השרת ובצד הלקוח יש להתקין;

- ☐ socket
- ☐ struct
- ☐ base64
- ☐ os
- ☐ gzip
- ☐ sqlite3
- ☐ json
- ☐ datetime
- ☐ argon2-cffi
- ☐ string
- ☐ cryptography

- **מיקומי הקבצים** - לכל צד יש לפתוח פרוייקט pycharm ולהעתיק את כל הקבצים הקשורים לכל צד.
- **נתונים התחלתיים** - אין חובה לנתונים התחלתיים, ניתן להפעיל את השירות כאשר מסד נתוני השרת (נתוני הלקוחות) ריק לחלוטין. אם לא הוספו מראש קובץ מסד נתוני השרת בצד השרת או קובץ מסד נתוני הלקוח בצד הלקוח הם ייווצרו באופן אוטומטי בעת הרצת הקוד.
- **רשת** - כל המחשבים צריכים להיות מחוברים לאותה רשת פנימית (Lan). המערכת משתמשת בספריית socket הנעזרת בפרוטוקול TCP, לכן על הלקוח לדעת מהי כתובת ה-IP של השרת על מנת ליצור חיבור, לכן יש להגדיר את הפרמטר host במחלקת "ServiceMainClass", הנמצאת בקובץ "ServiceMainClass.py" לכתובת ה-IP של השרת (גם בצד השרת וגם בצד הלקוח). יש לעשות זאת באופן הבא;

```
class ServiceMainClass():
    """
    """
    def __init__(self, host='192.168.228.1', port=5555):
```

בדוגמה הנ"ל כתובת ה-IP של השרת הוגדרה כ-192.168.228.1
כיצד למצוא את כתובת ה-IP במחשב השרת? יש לפתוח את ה-cmd (פתיחה- כפתור windows" ולחיצה על "r" במקביל, לאחר מכן בחלונית הנפתחת יש לרשום cmd), לאחר מכן יש להקיש "ipconfig", לאחר מכן יודפס באופן אוטומטי מידע רב (לא להיבהל 😊), יש לחפש את הכותרת;

Ethernet adapter Ethernet:

ולרשום את הכתובת שמימין ל-

IPv4 Address. :

המייצגת את כתובת ה-IP הפנימית של המחשב.

• ארכיטקטורה מינימלית נדרשת -

לצד השרת דרוש;

- ☐ מעבד מודרני עם מספר ליבות
- ☐ נפח אחסון פנוי במחשב
- ☐ זיכרון RAM, כמה שיותר יותר טוב (הגדלת כמות המשתתפים), 8GB אמור להספיק.
- ☐ מערכת הפעלה windows10/windows11 (מערכות ההפעלה שהיו זמינות בעבורי לבדיקת הפרוייקט).

לצד הלקוח דרוש;

- ☐ מעבד מודרני עם מספר ליבות
- ☐ נפח אחסון פנוי במחשב
- ☐ זיכרון RAM, כמה שיותר יותר טוב, 4GB אמור להספיק.
- ☐ מערכת הפעלה windows10/windows11 (מערכות ההפעלה שהיו זמינות בעבורי לבדיקת הפרוייקט).

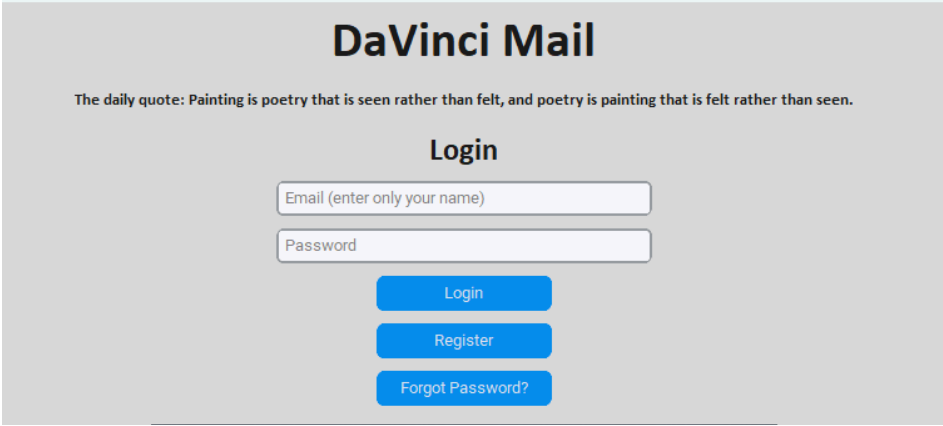
אופן הפעלת המערכת;

• הסבר הפעלה לחלקים המשותפים של מנהל המערכת ומשתמש קצה;

על מנת להריץ את המערכת יש לפעול לפי ההוראות המוסברות תחת הכותרת "התקנת המערכת" בחלק "פירוט סביבה נדרשת". לאחר מכן יש להתקין את הספריות **המשותפות** לכל צד, **הייחודיות** לכל צד ויש להגדיר את כתובת ה-IP של השרת לצד הלקוח ולצד השרת. את שני צעדים אלו יש לעשות כפי שמוסבר תחת הכותרת "התקנת המערכת" בחלק "פירוט כלים נדרשים". לאחר ביצוע כלל הדברים הללו, במחשב השרת יש ללחוץ עם המקש הימני בעכבר על הסימניה ב-pycharm בה רשום "server" וללחוץ על אופציה "Run server", במחשבי הלקוחות יש ללחוץ עם המקש הימני בעכבר על הסימניה ב-pycharm בה רשום "client" וללחוץ על אופציה "Run client". *יש להפעיל קודם את קוד השרת ולאחר מכן את קוד הלקוח. גם במקרה של טעות, לא תיהיה בעיה; אם יורץ קודם קוד הלקוח, לא יהיה ללקוח שרת להתחבר אליו, תוצג הודעה מתאימה במסך המחשב על המשתמש ללחוץ "אישור" ולאחר 15 שניות הלקוח ינסה שוב להתחבר לשרת. אם השרת הופעל במשך 15 השניות (כלומר לאחר הפעלת הלקוח ולחיצת "אישור"), הלקוח יחבר אליו לאחר תום 15 השניות.

• הסבר שימוש בשירות למשתמש קצה;

לאחר התחברות מוצלחת לשרת יפתח מסך הפתיחה (ישנה תמונה במסך הפתיחה, ניתן לראות אותה ב"מבנה / ארכיטקטורה" תחת הכותרת "תיאור מסכי המערכת";



אם המשתמש רשום לשירות עליו להזין את השם והסיסמא שהגדיר בעת ההרשמה בשדות המתאימים וללחוץ על כפתור "Login", אם ברצון המשתמש ליצור משתמש חדש עליו ללחוץ על כפתור "Register" ואם שכח את הסיסמא יש באפשרותו ללחוץ על כפתור "Forgot Password".

גיא משה טארנטו - DaVinci Mail

בעת לחיצה על כפתור "Register" נעבור למסך ההרשמה;

Register

What was your childhood nickname?

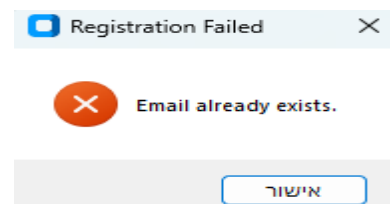
What is your favorite pet?

Register

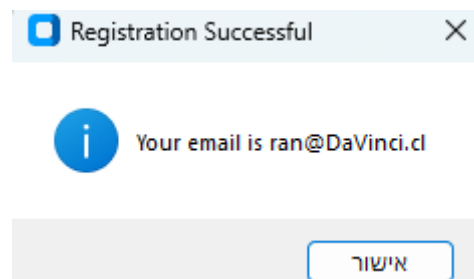
Back to Login

Password requirements;
At least six characters, one capital letter and one small letter.
Password recommendations;
For a strong password, use at least 12-16 characters with a mix of uppercase and lowercase letters, numbers, and special symbols (#,%,!), avoiding personal info, common words, and password reuse.

יש למלא את השדות בהתאם לדרישות לסיסמא חזקה המוצגות במסך. במקרה של קלט לא תקין, או כל שגיאה אחרת תוצג הודעה מתאימה, למשל;



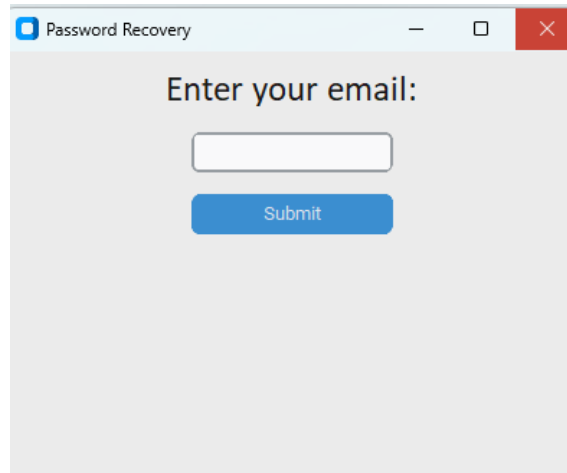
לאחר הרשמה מוצלחת תוצג ההודעה;



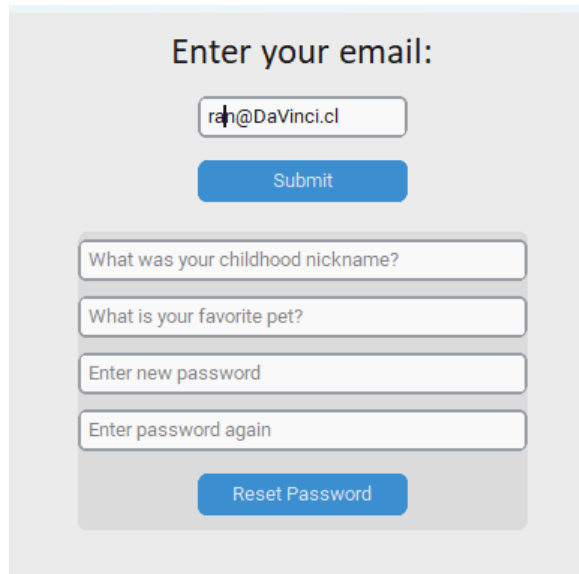
לאחר לחיצה על "אישור" המסך יועבר באופן אוטומטי למסך הפתיחה בחזרה.

גיא משה טארנטו - DaVinci Mail

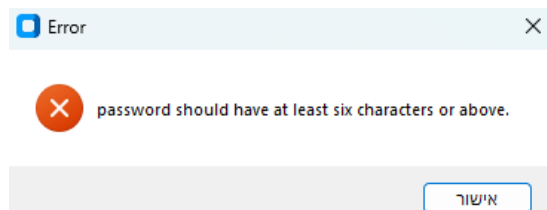
בעת לחיצה על כפתור "Forgot Password" תקפץ חלונית ובה יש להזין את כתובת המייל אשר רוצים לשנות את סיסמתה ;



בעת הזנת כתובת תקינה וקיימת המסך ישתנה ויראה כך ;



יש להזין את הנתונים בהתאם לשדות המתאימים, גם במסך זה הדרישות לסיסמא חזקה תקפות. במידה וסיסמא לא עונה על דרישה או קלט לא תקין אחר תופיע חלונית מתאימה, למשל ;



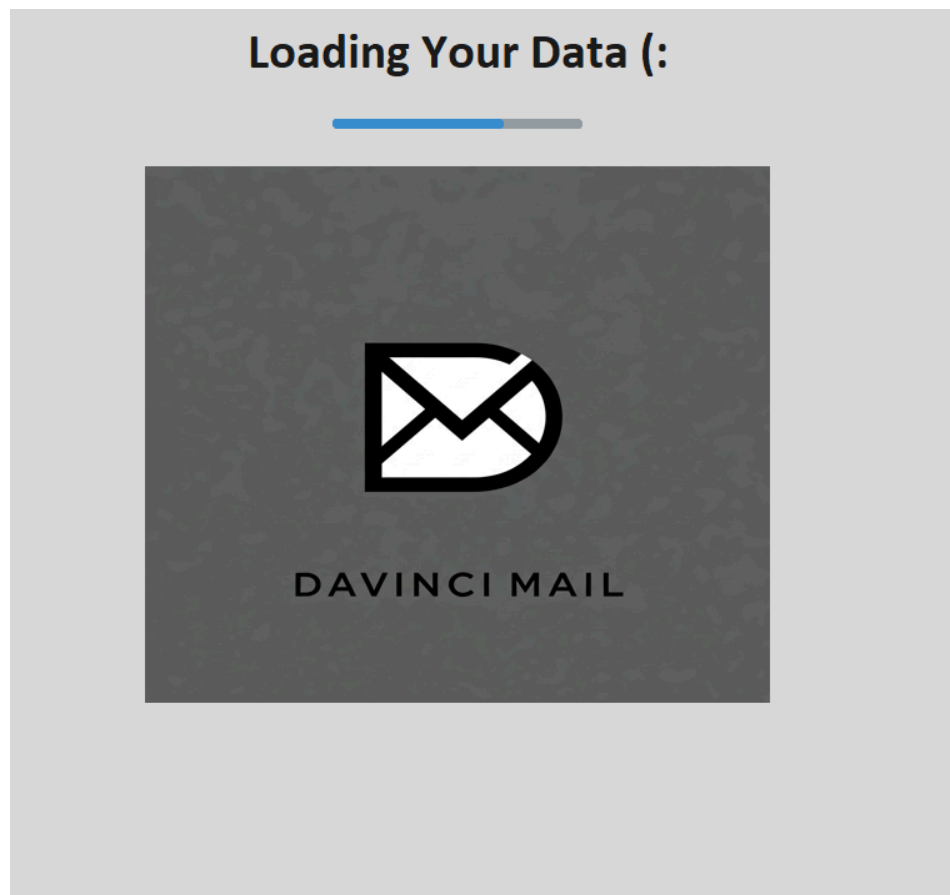
במקרה של הצלחה תופיע החלונית ;



לאחר לחיצה על "אישור" חלון ה-Password Recovery ייסגר ויופיע המסך פתיחה בלבד.

גיא משה טארנטו - DaVinci Mail

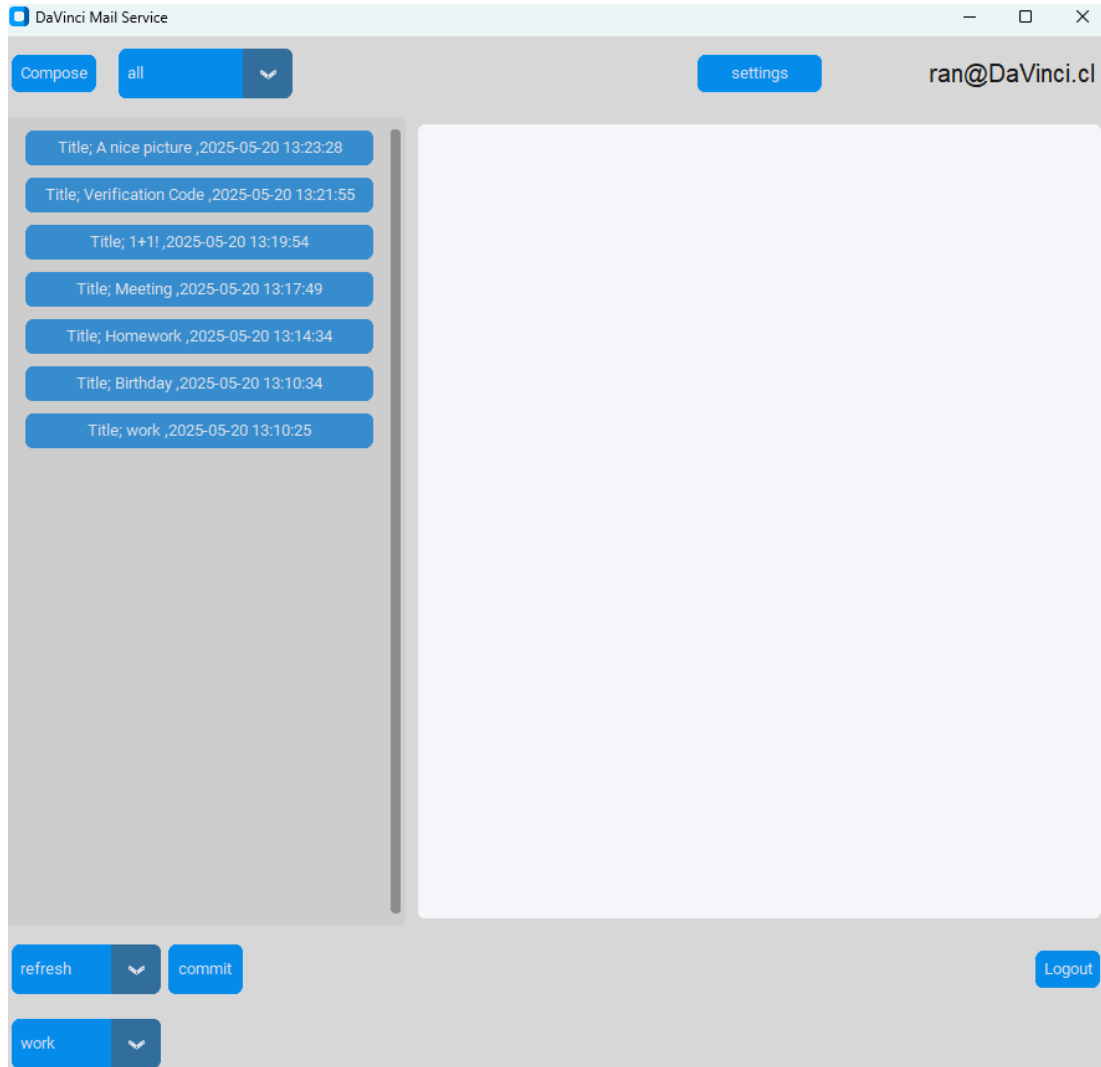
אם המשתמש לחץ על כפתור Login והזין את שם משתמש (כלומר השם לפני @DaVinci.cl) וסיסמא נכונה הוא יועבר למסך הטעינה ;



במסך הטעינה יש Progress Bar שני וסאונד נחמד.

גיא משה טארנטו - DaVinci Mail

לאחר סיום ה- Progress Bar מופיע המסך הראשי;

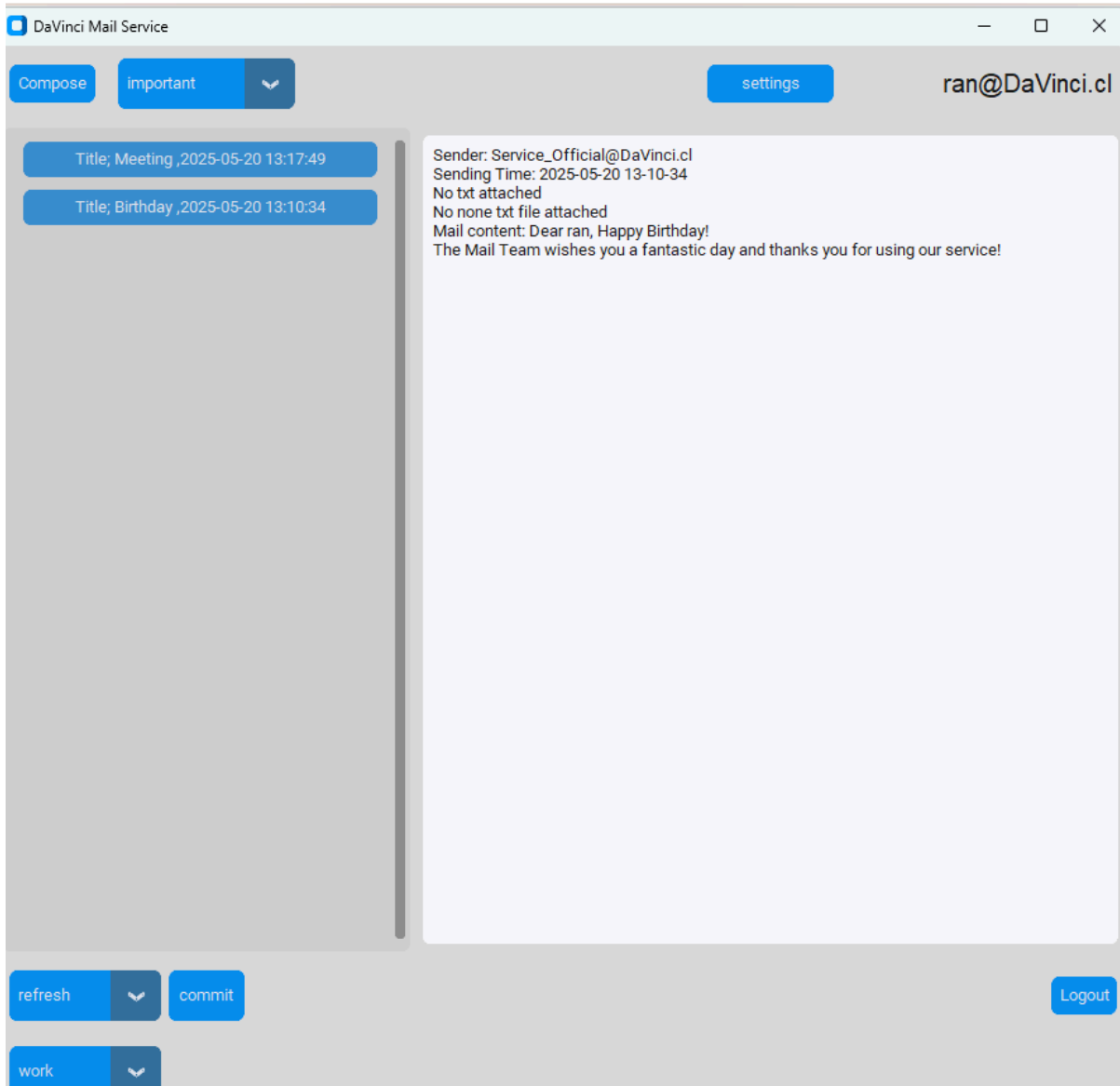


במסך הראשי ישנם כפתורים ולשוניות שונות, בעמודה השמאלית ישנן את כותרות המיילים בקטגוריה המוצגת, אם נלחץ על אחת מהן יוצג במסך הלבן תוכן המייל ופרטים נוספים עליו (אם צורפו למייל קבצים הם יופיעו בתיקיית ההורדות, תחת שם השולח ותאריך השליחה). בצד שמאל למעלה מופיע כפתור "Compose", עליו יש ללחוץ על מנת לשלוח מייל חדש, בלשוניות שלצידו מופיעות כל אפשרויות ההצגה השונות;

- all
- specific
- important
- saved for later
- by date
- deleted
- search
- work
- label2
- label3
- label4
- label5

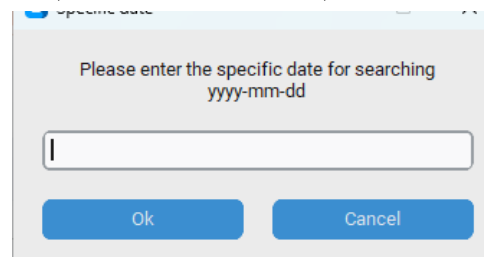
גיא משה טארנטו - DaVinci Mail

ניתן לראות כי רשום label2,label3 וכי, ניתן לשנות זאת לשמות ייחודים למשתמש (פירוט בהמשך), למשל למשתמש זה שיניתי את "label1" ל-"work". אם נרצה לראות את כל ההודעות לפי אפשרות הצגה מסוימת יש לסמנה ולאחר מכן לחוץ על כפתור "commit" בצד השמאלי התחתון של המסך, למשל בתמונה ניתן לראות את כל ההודעות שסומנו כחשובות ;



גיא משה טארנטו - DaVinci Mail

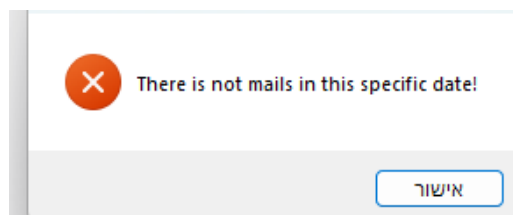
עבור בחירה באפשרויות ההצגה "by date", "search", "specific" ולחיצה על כפתור "commit" יפתחו חלוניות, למשל החלונית הבאה;



Please enter the specific date for searching
yyyy-mm-dd

Ok Cancel

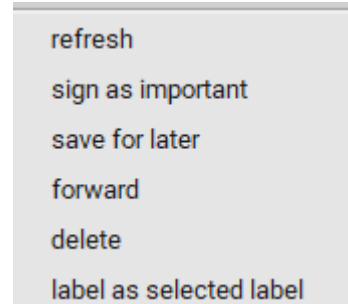
בחלונית הנפתחת על המשתמש להזין קלט בהתאם להודעה המוצגת, לאחר הזנת הקלט ובמידה ונמצאה התאמה יוצגו כל המיילים העונים על הסינון. אחרת תוצג הודעה מתאימה, למשל;



There is not mails in this specific date!

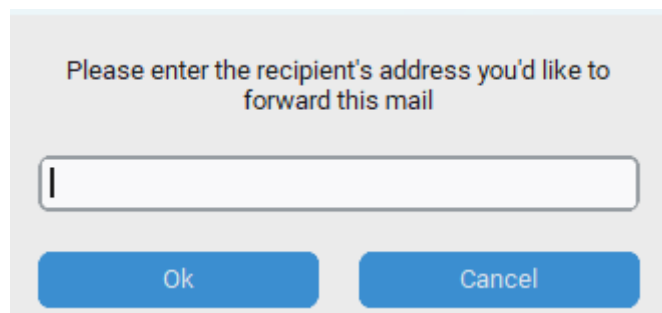
אישור

*אם יוזן קלט לא תקין תוצג הודעה מתאימה. לאחר מכן, על מנת לחזור להצגת כל ההודעות או לכל סינון אחר יש לסמן "all" וללחוץ שוב על "commit". משמאל לכפתור commit ישנה לשונית שניתן לראות כי מוצג בה "refresh", לשונית זו הינה לשונית פעולות, ניתן באמצעותה לבצע רענון לפי אפשרות ההצגה האחרונה שסומנה. ניתן גם באמצעותה לבקש מהשרת לבדוק האם יש מיילים חדשים באמצעות סימון אפשרות הצגה "all" ולחיצה על כפתור "commit" כאשר הפעולה שמסומנת הינה "refresh". כמו כן ישנן פעולות נוספות;



refresh
sign as important
save for later
forward
delete
label as selected label

אם נלחץ על כותרת מייל כלשהי, נסמן אחת מהפעולות הנ"ל (למעט מ-"refresh" ו-"forward") ונלחץ על כפתור "commit" תתבצע הפעולה, המייל יתווסף לקטגוריה הייחודית (למשל לאחד ה-labels) או שימחק (יש קטגוריית הצגה למיילים המוחקים). אם נלחץ על forward, תפתח חלונית ובה עלינו להזין את כתובת המייל אליה אנו רוצים להעביר את המייל.



Please enter the recipient's address you'd like to forward this mail

Ok Cancel

*מתחת ללשונית הפעולות ישנה את לשונית ה-labels ובה שמות כל ה-labels (או הגנריים, או הייחודיים של המשתמש, אם שינה אותם), אם המשתמש יבחר בפעולה "label as selected" ה-label "label" המייל יתווסף תחת ה-label שסומן בלשונית הנ"ל.

גיא משה טארנטו - DaVinci Mail

אם נלחץ על כפתור "compose" יפתח החלון הבא ;

If you wish to send to distribution list,
write; distribution_num, and enter as num 1,2 or 3

To:

Message:

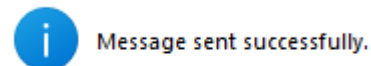
Title

Attach file - only txt file or PNG images

Send

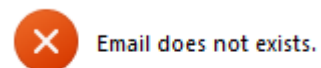
באמצעות חלון זה ניתן לשלוח מיילים חדשים, יש למלא את כתובת הנמען בלשונית הראשונה, לאחר מכן יש למלא את כותרת המייל ולאחר מכן יש לרשום את תוכנו. ניתן לצרף קובץ ואז תפתח אפשרות בחירת הקבצים מן המחשב. *ניתן לשלוח הודעות לכלל חברי אחת מרשימות התפוצה באמצעות רשימת "distribution_num" בלשונית הכתובת, כאשר num יהיה 1,2,3 כמספר רשימת התפוצה (לכל משתמש יש כ-3).

בעת לחיצה על send תשלח ההודעה לשרת, אם המשתמש הזין מייל **תקני** (לפי פורמט השירות, יש בדיקה לכך בצד הלקוח לשם חסיכה בהעברת מידע מיותרת) ואם הכתובת **קיימת** השרת יחזיר הודעה חיובית ללקוח והיא תוצג למשתמש ;



אישור

לאחר לחיצה על אישור יסגר חלון שליחת ההודעה.
אם המייל לא קיים תוצג הודעה מתאימה (ישנה גם הודעה לכך שפורמט הכתובת לא תקין) ;

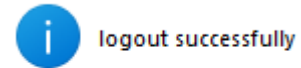


אישור

החלון לא יסגר, והמשתמש יוכל לערוך את המייל.

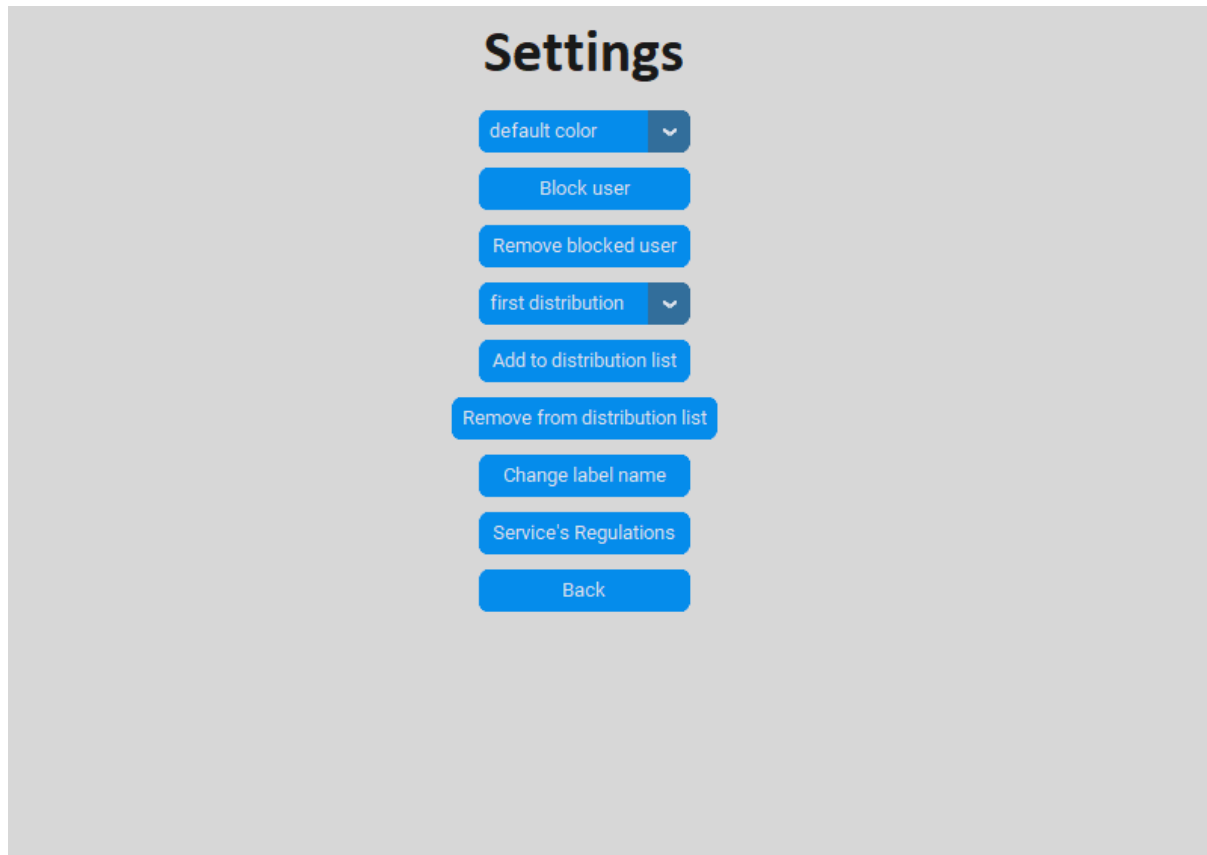
גיא משה טארנטו - DaVinci Mail

בלחיצה על כפתור "Logout" תתבצע התנתקות, תתקבל הודעה האם ההתנתקות צלחה או כשלה (כשל תקשורת או במסדי הנתונים) והמסך יעבור למסך הפתיחה.

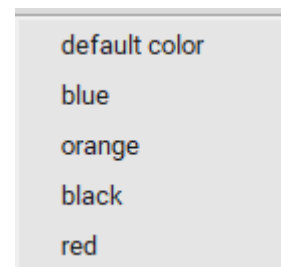


אישור

אם נלחץ על כפתור "settings", נעבור למסך ההגדרות;

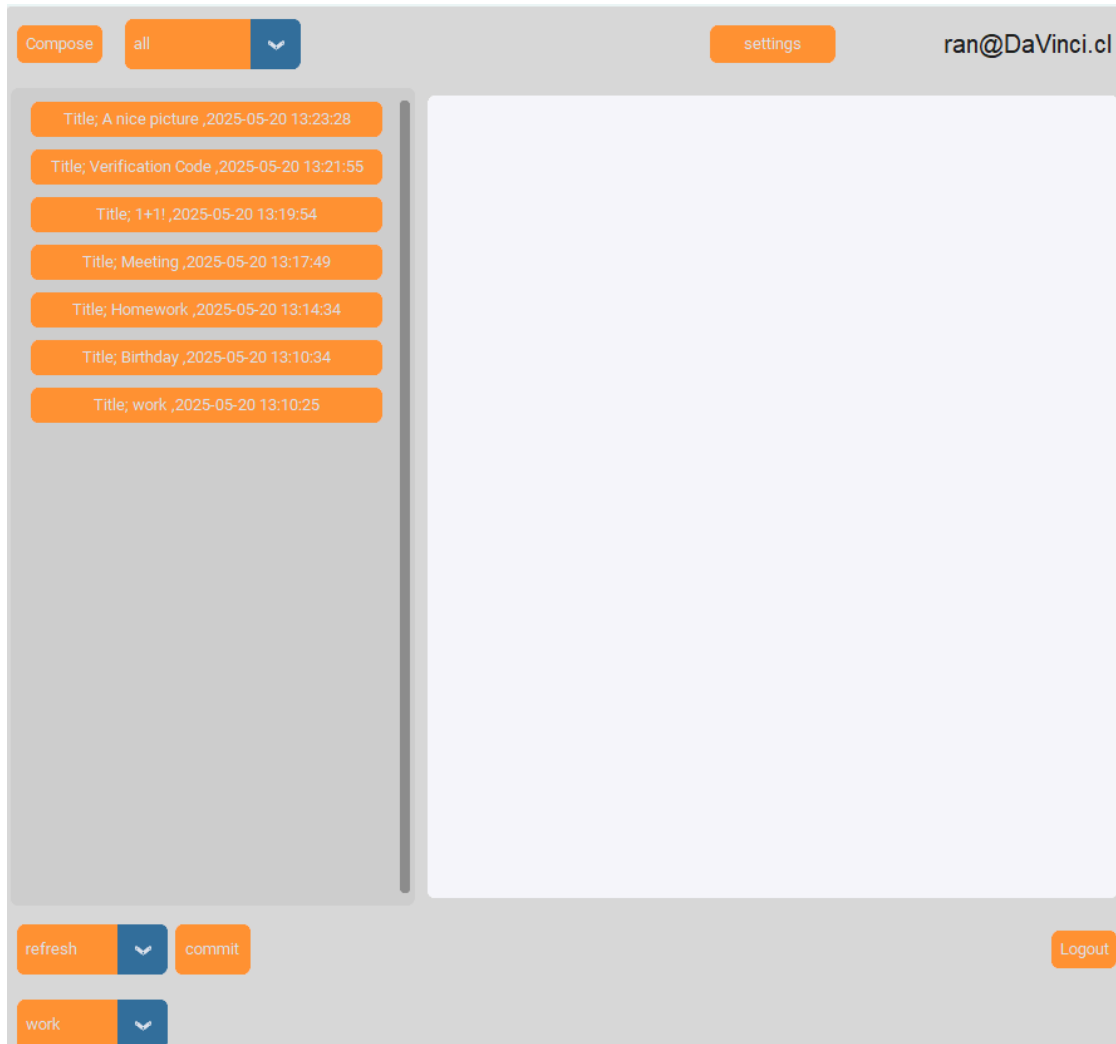


הלשונית הראשונה הינה לשונית צבע, אם נשנה את הצבע לאחת האפשרויות הנ"ל;

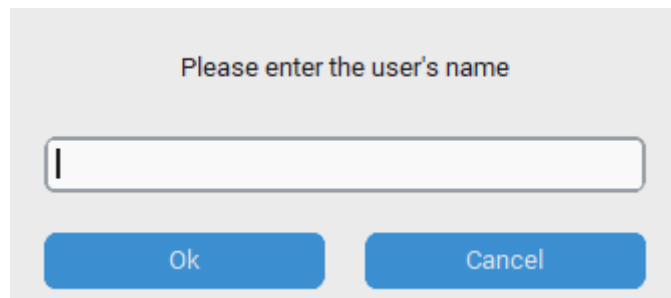


גיא משה טארנטו - DaVinci Mail

ולאחר מכן נחזור למסך הראשי (כפתור "Back"), צבע התצוגה הראשי ישתנה ;



אם נלחץ כל כפתור "Block user", תפתח חלונת מתאימה ;



יש להזין את שם המשתמש (כלומר השם לפני @DaVinci.cl) אותו רוצים לחסום (המייילים שישלח לא יוצגו עד להסרת החסימה). אם למשל הכתובת כבר חסומה / המשתמש ניסה לחסום את עצמו / המשתמש ניסה לחסום את הכתובת הרשמית של השירות, ניסיון החסימה יכשל ויקבל הודעה מתאימה ;

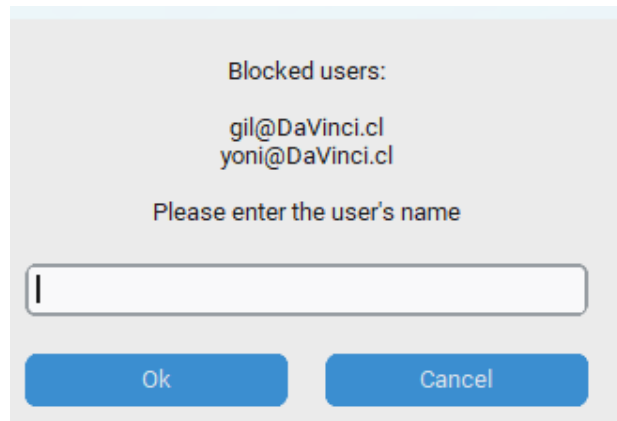


User already blocked.

אישור

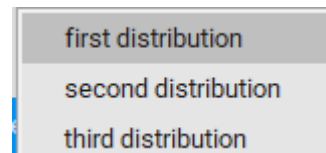
גיא משה טארנטו - DaVinci Mail

אם ברצוננו להסיר כתובת מרשימת החסומים נלחץ על כפתור "Remove blocked user", ואז תופיע לנו החלונית הבאה ;

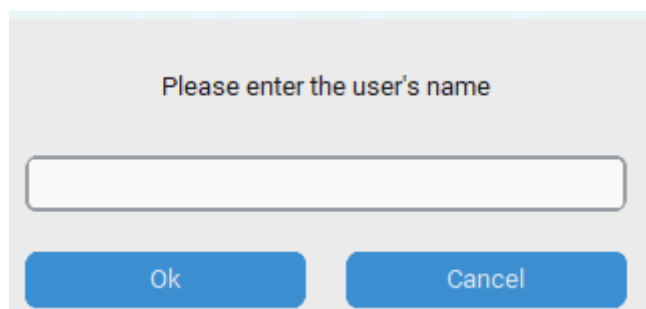


בחלונית ישנה את רשימת הכתובות החסומות, ניתן להכניס את שם המשתמש שרוצים להסיר את חסימתו, והוא יוסר, אם הכתובת לא קיימת ברשימה תופיע חלונית מתאימה.

בלשונית שמתחת לכפתור "Remove blocked user" ניתן לסמן אחת משלושת רשימות התפוצה ;



לאחר מכן בלחיצה על כפתור "Add to distribution list" ניתן יהיה להוסיף כתובת לרשימת התפוצה שסומנה.



אם למשל הכתובת כבר הוספה / המשתמש ניסה להוסיף את עצמו / המשתמש ניסה להוסיף את הכתובת הרשמית של השירות, ניסיון החסימה יכשל ויקבל הודעה מתאימה ;

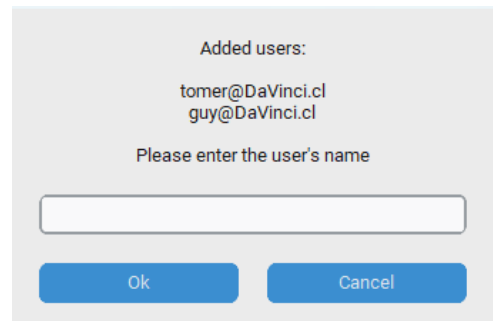


You can't add yourself

אישור

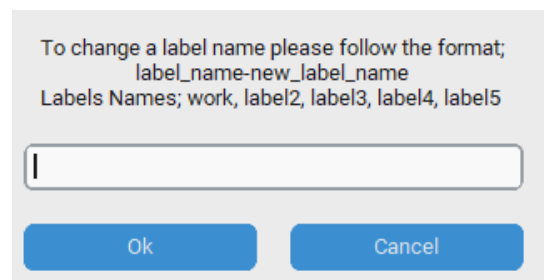
גיא משה טארנטו - DaVinci Mail

בלחיצה על כפתור "Remove from distribution list" תפתח חלונית ובה ישנה את רשימת הכתובות שהוספו לרשימת התפוצה ;

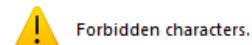


ניתן להכניס את שם המשתמש שרוצים להסיר מהרשימה, והוא יוסר. אם הכתובת לא מופיע ברשימה תופיע חלונית מתאימה.

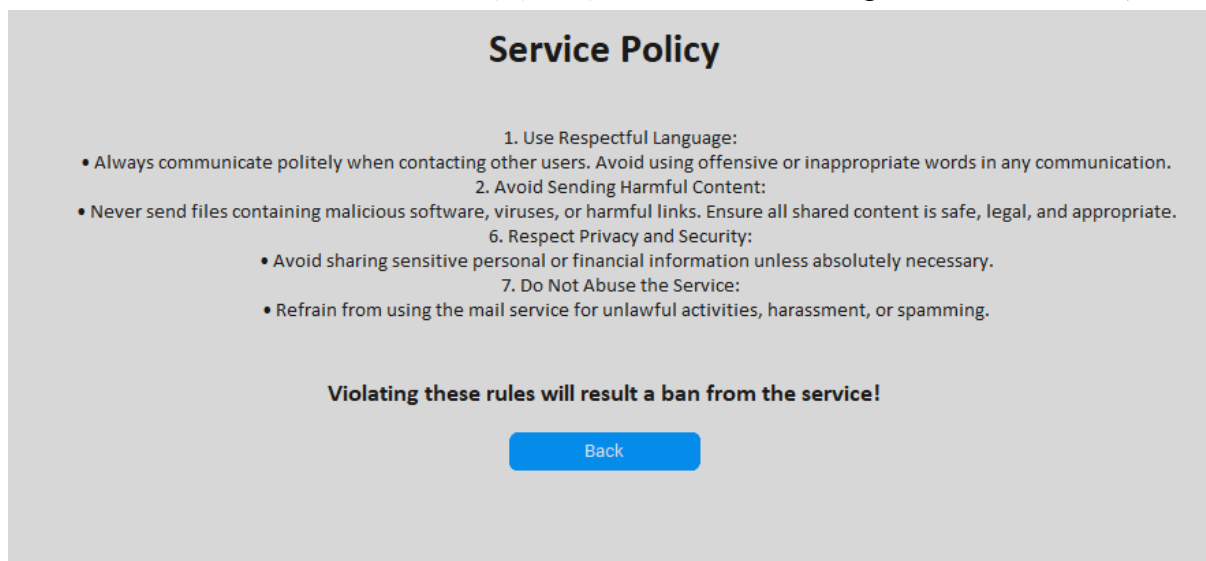
אם נלחץ על כפתור "Change label name" תפתח החלונית הבאה ;



בחלונית מוסבר כיצד יש לשנות שם של Label, יש לרשום בפורמט הבא ;
(שם_Label החדש)-(שם_Label הישן), כך כל משתמש יכול להגדיר את שמות ה-Labels כרצונו.
אם הקלט לא תקין / שם ה-Label שהמשתמש רוצה לשנות לא קיים / השם החדש מכיל תווים אסורים / יש כבר label עם השם הנ"ל, תוצג הודעה מתאימה, למשל ;



אם נלחץ על כפתור "service regulations" נעבור למסך התקנון של השירות ;



Service Policy

1. Use Respectful Language:
 - Always communicate politely when contacting other users. Avoid using offensive or inappropriate words in any communication.
2. Avoid Sending Harmful Content:
 - Never send files containing malicious software, viruses, or harmful links. Ensure all shared content is safe, legal, and appropriate.
6. Respect Privacy and Security:
 - Avoid sharing sensitive personal or financial information unless absolutely necessary.
7. Do Not Abuse the Service:
 - Refrain from using the mail service for unlawful activities, harassment, or spamming.

Violating these rules will result a ban from the service!

Back

ניתן לצפות בתקנון וללחוץ על כפתור "Back" לשם חזרה למסך ההגדרות.

לחיצה על כפתור "Back" במסך ההגדרות תחזיר את המשתמש למסך הראשי ותעדכן את הצבע שמסומן בתפריט הצבעים כצבע התצוגה הראשי, כפי שציינתי.

סיכום אישי / רפלקציה

תהליך העבודה על הפרויקט, ההצלחות, האתגרים, הקשיים ודרכי הפתרון

תהליך העבודה על הפרויקט היה ארוך, לא פשוט ומורכב. כמו כל תהליך, תהליך העבודה ידע עליות ומורדות, מההתרגשות ההתחלתית והתכנון הראשוני, עד לכך שהקוד ניהיה ארוך ומורכב, מה שהוביל לכך שפעמים רבות, תיקון באג, יכל לקחת זמן רב וממושך. כחלק מהתכנון הכנתי לי רשימה של דברים שיש להוסיף, ההצלחות היו שבכל פעם יכולתי להסיר נושאים מהרשימה וכך ידעתי שאני מתקרב לאט לאט אל התוצר הסופי. שתי ההצלחות המשמחות ביותר הן שהצלחתי להעביר בפעם הראשונה באופן תקין, מייל ממשתמש אחד לשני (כאשר שניהם הורצו על אותו מחשב) וההצלחה השנייה הייתה העברת מייל ממשתמש אחד לשני כאשר חיברתי שלושה מחשבים, שרת ושני משתמשים והמערכת עבדה כשורה. שני צעדים חשובים אלו נתנו לי ביטחון להמשיך ולהתקדם. האתגרים היו למידה עצמאית באופן נרחב של נושאים רבים שנלמדו באופן בסיסי בבית הספר, כמו התנהלות עם exceptions ומקרי קצה, מימוש תקשורת, הצפנה, gui ועוד...

לכן לכל אורך עשיית הפרויקט חוץ מהעשייה עצמה היה גם תהליך למידה נרחב. כמו כן, היו גם קשיים, לפעמים היו באגים שלא העלו תקלות (כפי שציינתי לדוגמה את התקלה בשאילתות למסד הנתונים) ולחפש תקלה שלא משאירה סימני מעקב או רמזים לפיתרון הוא קושי, לכן על מנת לפתור את התקלה, עברתי לאט לאט על חלקי הקוד המתקשרים לנושא התקלה, ביצעתי תהליך debug באמצעות הדפסת ערכים של משתנים שונים וביצעתי בדיקות נוספות עד להגעה למקור הבעיה. קושי נוסף שהיה הוא ניהול זמן, השנה אני בכיתה י"ב, שנת הלימודים הינה עמוסה ובנוסף השנה השתתפתי בפרויקט כספות במגמת פיזיקה מטעם מכון דוידסון (פרויקט בחירה), פרויקט הכספות היה בסדר גודל כמו הפרויקט הנ"ל. אני וחבריי קבוצתי עבדנו עליו חודשים רבים והיו תקופות שבשל שילוב עומס לימודי והשלים האחרונים של פרויקט הכספות, כמעט ולא היה לי זמן לעבוד על הפרויקט הנ"ל, לכן היו תקופות מסוימות שהיה פער של שבועיים מהפעם האחרונה שעבדתי על הפרויקט. בעקבות כך, כל פעם מחדש הייתי צריך לעשות לעצמי מעין רענון קצר לעבור על הדברים שרציתי להוסיף, לתקן ולשפר. לכן, תכננתי לעצמי לוח ניהול זמנים, מסודר שיאפשר לי לעמוד בכלל היעדים השונים בפרויקטים השונים ומבחינת הלימודים ואכן הצלחתי לעמוד בכלל היעדים.

תהליך למידה

כפי שציינתי בפסקה הקודמת כלל הפרויקט היה תהליך למידה אחד גדול ורציף, נתקלתי בבעיות שונות ומגוונות, מתחומים שונים ומגוונים. במרבית המקרים חיפשתי ברשת פיתרון לבעיה, בחלק מהמקרים נועצתי בבינה מלאכותית, או במוריי המגמה השונים ובחלק מהמקרים נעזרתי בחברים. יתר על כן, היו גם דברים רבים שלא ידעתי כיצד לממש, למשל gui מתקדם ולא בסיסי, לכן קראתי ברשת, ראיתי סרטונים, ונעזרתי בכלים שונים. זאת ועוד, למדתי גם כתיבה יותר נכונה של קוד ואני חושב שהשתפרתי בכך, דבר שנדרשנו ליישם באופן בסיסי במגמה. יתרה מכך, כתיבה נכונה הינה דבר חשוב מאוד לכל מתכנת. כל שלב ושלב בפרויקט כלל למידה המתבטאת בכיצד לממש דבר כזה או אחר, פתירת בעיה ומציאת הגורם אליה. לכן, כפי שציינתי כל הפרויקט היה תהליך למידה אחד גדול ורציף.

כלים להמשך

הכלים שאני לוקח אותם להמשך הם במרביתם כלים שכבר מימשתי בעבר ומימשתי גם בפרויקט זה, תכנון מוקדם לפני ביצוע, במקרה של בעיה לעבוד לאט לאט עד למציאת מקור ופיתרון, ניהול זמנים נכון, היועצות עם מוקדי ידע (מורים), בני משפחה (דודי) וחברים במידת הצורך. כלים חשובים שאקח מפרויקט זה הם ידע יותר טוב בביצוע debug וכתיבה נכונה של קוד.

תובנות מהתהליך, לרבות עזרה ממומחים, שיתוף מידע ולמידת עמיתים

התובנות שלי מהתהליך הם שהתהליך היה משתלם, העשרתי רבות את הידע שלי בתחומים שונים (בין היתר בזכות הבעיות השונות, מטעויות לומדים הכי הרבה), למדתי לבנות תוכנה וקוד בסדר גודל שמעולם לא עשיתי. חשוב לתכנן מראש, לא למהר ולפעול בקפידות. יתרה מכך, במידת הצורך נעזרתי בייעוץ מדודי, ממוריי המגמה השונים ובחברים או בבינה מלאכותית (כמה שפחות, רובוט לא יכול להחליף נקודת מבט של אדם), תמיד טוב שישנה נקודת מבט נוספת וחדשה.

בראייה לאחר, האם היית מיישם אחרת חלקים בפרויקט

באופן כללי אני מרוצה מההחלטות שעשיתי ודרכי המימוש והיישום שפעלתי. אני גאה בפרויקט שלי ומהתוצר הסופי, יחד עם זאת יש דבר אחד שהייתי מיישם אחרת בפרויקט; כפי שצינתי בספר, עבור כל לקוח חדש שמתחבר לשרת מופעל Thread אשר מטפל בו. שיטה זו הינה טובה לטיפול במספר בינוני של לקוחות, אך עבור מספר גדול של לקוחות היא יכולה להעמיס מאוד על המעבד של מחשב השרת. עקב כך, אם היה לי עוד זמן הייתי משנה את שיטת ניהול התקשורת עם הלקוחות לשיטה שונה, למשל שימוש ב-select. על קצה המזלג, כאשר ישנו שימוש ב-select לא נפתח עבור כל לקוח Thread אישי, אלא יש Thread אחד המשתמש בפונקציית select המטפל בכלל הלקוחות המחוברים לשרת. עם פונקציית select ניתן לנהל ב-Thread אחד כמה ערוצי תקשורת עם sockets שונים, על ידי כך שהיא מבצעת "סלקציה" לערוצים המוכנים לכתיבה וקריאה ברגע נתון. לכן גם כאשר מספר הלקוחות המחוברים הינו יחסית גבוה, אומנם יהיה עומס על השרת, אך נמוך משמעותית מאשר אם היה Thread עבור כל לקוח כפי שאני מימשתי. הבנתי את הדברים הללו לקראת מועד הגשת ספר הפרויקט הכולל את קוד הפרויקט, לשם ביצוע השינוי היה עליי ללמוד את השימוש ב-select, את ניהול התקשורת באמצעותו באופן עצמאי (במגמה ללימוד select הוקדש זמן קצר מאוד והצורך לשימוש ב-select ויתרונותיו לא הודגש בכלל, לכן לא ידעתי תחילה כי דרך הפעולה שלי איננה מיטבית) ולשנות את כלל ניהול התקשורת בפרויקט. דברים אלו מצריכים שעות עבודה רבות מאוד שבשילוב עם הבגרויות שהיו בזמן האחרון לא היו זמינות בעבורי.

במידה והיו ברשותך משאבים נוספים, האם וכיצד ניתן לשפר את הפרויקט

בהמשך לפסקה הקודמת, עם עוד משאבים - זמן נוסף, קודם כל הייתי לומד את ניהול התקשורת באמצעות select ומבצע שינוי לניהול התקשורת עם הלקוחות בפרויקט באמצעותה. יתר על כן, הגזרה שבה הפרויקט עוסק הינו שירות מייל, גזרה זו הינה גזרה רחבה מאוד, ניתן לשפר את הפרויקט ע"י הוספת עוד ועוד פיצ'רים, שונים ומגוונים, כמו שאחד מהמורים אמר לי - "אין גבול לפיצ'רים שאתה יכול להוסיף". בעולם של היום בזכות האינטרנט יש אינסוף מקורות מידע שונים, לכן מבחינת מקורות ידע אין בעיה כלל, באופן תאורטי יכולתי להוסיף כל פיצ'ר שרק אחשוב עליו. גם בזכות הידע העצום שיש ברשת יכולתי להעמיק מאוד את הידע שלי בתחומים השונים שיישמתי בפרויקט. אומנם, את התאוריה יש לשלב במציאות, ובשביל להוסיף פיצ'רים יש להשקיע זמן רב, לכן אני חושב שבזמן הנתון שהיה לי ביצעתי את המקסימום. עם עוד זמן יכולתי להוסיף עוד דברים, אבל שוב, אין גבול לכמות הדברים שניתן להוסיף.

תודות

אני רוצה להודות למורי המגמה השונים, עידן, רוס, עודי, קובי וליאת, על המענה לשאלות ונתינת נקודת מבט נוספת במידת הצורך. בנוסף, אני רוצה להודות לדודי, ניר ולחבריי על ייעוץ ומענה לשאלות. אני וחבריי עזרנו אחד לשני בייעוץ באופן הדדי, יתרון חשוב מאוד, אומנם הפרויקט הינו אישי, אך המעטפת הינה משלבת עזרה הדדית.

ביבליוגרפיה

להכנת הפרויקט נעזרתי באתרים רבים, התוכנות והמקורות;

- עיצוב תרשימים וגרפים, יצירת תמונות והסבר על **Class Diagram**;

★ [Class Diagram | Unified Modeling Language \(UML\)](#)

★ [Copilot](#)

★ [Diagrams](#)

★ MS PowerPoint (גרסת תוכנה)

- פיתרון בעיות, למידה והעמקת הידע (כלליים);

★ [Geeks for geeks](#)

★ [Gemini](#)

★ [GitHub](#)

★ [Stack overflow](#)

★ [NeuralNine](#)

- תוכנות איתן בדקתי את תקינות פעילות המערכת;

[DB Browser for SQLite](#)

[Wireshark](#)

- מקורות הידע מפרק "מבנה / ארכיטקטורה", כותרת "תיאור האלגוריתמים מרכזיים בפרויקט";

★ [אלגוריתם ראשון](#) - גיבוב סיסמאות במסד הנתונים - [מאמר מקיף בנושא גיבוב של האתר "vaadata"](#)

★ [אלגוריתם שני](#) - אבטחת תעבורת התקשורת בין השרת הלקוח ולהפך - [מאמר מקיף בנושא אבטחה סימטרית ואסימטרית של האתר "preyproject"](#), [מאמר kiteworks](#) אודות דרך הפעולה של מפתח AES בדגש על גרסת 256 ביטים בה נעשה שימוש בשירות.

★ [אלגוריתם שלישי](#) - שליחת ההודעה בדרך שתוכנה לא השתבש ובצורה יעילה - [מאמר מקיף בנושא "Serialize Your Data With Python"](#) מטעם אתר [realpython](#).

★ [אלגוריתם רביעי](#) - מניעת מתקפת DOS ומניעת מתקפת DDOS - [מאמר בנושא מתקפת DOS מטעם האתר CloudFlare](#), [מאמר בנושא מניעת מתקפת DDos מטעם האתר SecurityScorecard](#).

★ [אלגוריתם חמישי](#) - ניהול כלל נתוני השירות - עיקר המידע שבו נעזרתי הינו המודל של המורה (כמו מעין google classroom) שהינו מותנה גישה לתלמידים לכן לא אוכל לצרף קישור.

```
from datetime import datetime
import string
import json
import gzip
from EncryptionLib import HybridEncryption
from argon2 import PasswordHasher
from argon2.exceptions import VerifyMismatchError

class ServiceMainClass():
    '''
        This is the mail class of the Davinci Mail service, this class has
        attributes which the two classes;
        "MailClient" and "MailServer" should have in common. In addition, this
        class has "prepare_message" and "treat_message"
        which the "MailClient" and "MailServer" should have in common.
        Furthermore, this class have the following functions -
        "hash_password", "verify_password", which are not used in both
        "MailClient" and "MailServer, but the first one used in the first class
        and seconded
        in the seconded respectively, because these function are two parts of
        the same purpose - hashing the passwords it more efficient to define them
        in the same class, for easy changes that won't involve changes it each
        class but only in this class.
        It is important to notice that "MailClient" and "MailServer" inherit
        from this class and that this class has more functions for the different
        service's uses.
        Some of these classes are staticmethods and some are rgular ones, these
        functions are functions which changes in them shouldn't involve changes
        in the "MailClient" and "MailServer" because it not directly connected
        to them, these functions are part of the whole system.
    '''
    def __init__(self, host='127.0.0.1', port=5555):
        self.host=host # The ip address of the computer whom hosts the
server
        self.port=port # The port on the computers which will be used for
the service data flowing, i read that this port 5555 is rarely used
        self.ph=PasswordHasher() # creating instance of argon2 password
hasher in order to have a strong and reliable hasing cpability to my
service
        self.DaVinci_quotes=["Life without love, is no life at all.",
                             "As a well-spent day brings happy sleep, so
life well used brings happy death.",
                             "Life is pretty simple: You do some stuff.
Most fails. Some works. You do more of what works.",
                             "Painting is poetry that is seen rather than
felt, and poetry is painting that is felt rather than seen.",
                             "There is no object so large but that at a
great distance from the eye it does not appear smaller than a smaller
object near.",
                             "Learning never exhausts the mind.",
```

```

        "Study without desire spoils the memory, and
it retains nothing that it takes in.",
        "The noblest pleasure is the joy of
understanding.",

        "Nature never breaks her own laws."]
    self.forbidden_characters= ['!', '#', '$', '%', '&', '*', '+', '/',
'=', '{', '}', '[', ']', '|', '(', ')', '@', ',', '.', " "]
    def prepare_message(self, data, aes_key, client_iv,
shifting_bytes_num):
        """
        preparing the message; convert to json, encode the json data to
bytes, encrypting the bytes using EncryptionLib library.
        Finnally, compress the encrypted bytes.
        :param data:
        :param aes_key:
        :param client_iv:
        :param shifting_bytes_num:
        :return the ecnrypted and compressed data:
        """
        json_data = json.dumps(data) # convert dict to JSON string
        bytes_data = json_data.encode('utf-8') # encoding the JSON data to
bytes
        encrypted_data = HybridEncryption.encrypt_message_AES(bytes_data,
aes_key,client_iv) # encrypting the data
        shifted_bytes_data =
HybridEncryption.custom_encrypt_shift(encrypted_data, shifting_bytes_num)
# shifting the bits
        compressed_data = gzip.compress(shifted_bytes_data) # compress the
encrypted bytes
        return compressed_data
    def treat_message(self, data, aes_key, client_iv, shifting_bytes_num):
        """
        treating the message; decompress, decrypting the bytes using
EncryptionLib library, and then converts from json.
        :param data:
        :param aes_key:
        :param client_iv:
        :param shifting_bytes_num:
        :return the dictionary which the server / clients sent:
        """
        decompressed_data = gzip.decompress(data) # decompress the data
        decrypted_data =
HybridEncryption.custom_decrypt_shift(decompressed_data,shifting_bytes_num
) # reshifting the bits
        decrypted_data =
HybridEncryption.decrypt_message_AES(decrypted_data,aes_key, client_iv) #
decrypting the data
        response = json.loads(decrypted_data) # Convert JSON string back
to dictionary
        return response
    def hash_password(self,password):
        """

```

```

        hashing password using argon2 library, which is a recommended
        library for hashing passwords whom even won some contests in password
        hashing
        :param password:
        :return the hashed password as string:
        '''
        hashed_password = self.ph.hash(password)
        return hashed_password
    def verify_password(self, regular_password, hasehd_password):
        '''
        verify if password is the same as hashed password, which hashed
        with argon2 library.
        :param regular_password - the password itself:
        :param hasehd_passowrd:
        :return are the hashed and the regular passwords are the same?:
        '''
        try:
            self.ph.verify(hasehd_password[0], regular_password)
            return True
        except VerifyMismatchError:
            return False
    def is_birthday_today(self, birth_date_str):
        '''
        Checks if the birthday represented by the given date string is
        today.
        :param birth_date_str - a date in 'dd/mm/yyyy' format:
        :return True if the birthday is today, False otherwise:
        '''
        try:
            # creates datetime object with the given date
            birth_date = datetime.strptime(birth_date_str, "%d/%m/%Y")
        except ValueError:
            print("Error: Invalid date format. Please use 'dd/mm/yyyy'.")
            return False
        today = datetime.now()
        return (birth_date.day == today.day and birth_date.month ==
        today.month)
    @staticmethod
    def strong_password(password):
        '''
        chekcing if password is strong enough
        :param password:
        :return True is strong enough and if not return the resaon:
        '''
        if len(password)<2:
            return ("password should have at least six characters or
            above.")
        elif not any(char.isupper() for char in password):
            return ("You must have at least one capital letter in your
            password.")
        elif not any(char.islower() for char in password):
            return ("You must have at least one small letter in your
            password.")

```

```

        else:
            return True
    @staticmethod
    def check_content(content):
        """
        Checks if at least one word from the whole a list is in the given
        string content
        :param content:
        :return: True if at least one whole word from the list is found in
        the string,
            False otherwise.
        """
        bad_words = ["crap", "chit", "bitch", "bloody", "bollocks",
"bugger", "bullshit", "arse", "bastard", "damn",
                    "fuck", "motherfucker", "fucker", "pissed", "Cunt",
"Arsehole",
                    "stupid", "idiot"]
        # remove punctuation from the content and convert the latters to
        lowercase
        input_string_cleaned = content.translate(str.maketrans('', '',
string.punctuation)).lower()
        # split the 'cleaned' content to words and put them in a set for
        efficient matching
        input_words_set = set(input_string_cleaned.split())
        # convert the bad words list to a set of lowercase words
        word_list_set = set(word.lower() for word in bad_words)
        return len(input_words_set.intersection(word_list_set)) > 0

```

מחלקת HybridEncryption:

```

from cryptography.hazmat.backends import default_backend
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms,
modes
import os
class HybridEncryption():
    """
    The encryption class of the service, provides the functions to use RSA
    and AES encryptions
    To the service's needs.
    """
    def __init__(self):
        self.default_block_size = 128 # 128 bytes
        self.byte_size = 256 # One byte has 256 different values.
        # IMPORTANT: The block size MUST be less than or equal to the key
        size!
        # (Note: The block size is in bytes, the key size is in bits. There
        # are 8 bits in 1 byte.)
    def getBlocksFromText(self,message):
        """
        Converts a string message to a list of block integers. Each integer
        represents 128 string characters.
        :param message:
        :return the list of block integers:

```



```

'''
messageBytes = message.encode('ascii') # convert the string to
bytes
blockInts = []
for blockStart in range(0, len(messageBytes),
self.default_block_size):
    # Calculate the block integer for this block of text
    blockInt = 0
    for i in range(blockStart, min(blockStart +
self.default_block_size, len(messageBytes))):
        blockInt += messageBytes[i] * (self.byte_size ** (i %
self.default_block_size))
    blockInts.append(blockInt)
return blockInts
def getTextFromBlocks(self, blockInts, messageLength):
'''
    Converts a list of block integers to the original message string.
    The original message length is needed to properly convert the last
block integer.
:param blockInts:
:param messageLength:
:return the original message string:
'''
message = []
for blockInt in blockInts:
    blockMessage = []
    for i in range(self.default_block_size - 1, -1, -1):
        if len(message) + i < messageLength:
            # Decode the message string for the 128 (or whatever
            # blockSize is set to) characters from this block
integer.
            asciiNumber = blockInt // (self.byte_size ** i)
            blockInt = blockInt % (self.byte_size ** i)
            blockMessage.insert(0, chr(asciiNumber))
    message.extend(blockMessage)
return ''.join(message)
def readKeyFile(self, keyFilename):
'''
    Reads the public / private key
:param keyFilename:
:return the key size and the key as (n,e) for public key and (n,d)
for private key:
'''
fo = open(keyFilename)
content = fo.read()
fo.close()
keySize, n, EorD = content.split(',')
return (int(keySize), int(n), int(EorD))
def encrypt_blocks(self, message, key):
'''
    Converts the message string into a list of block integers, and then
encrypts each block integer.
:param message:

```

```

        :param key:
        :return encryptedBlocks:
        '''
        encryptedBlocks = []
        n, e = key
        for block in self.getBlocksFromText(message):
            # ciphertext = plaintext ^ e mod n
            encryptedBlocks.append(pow(block, e, n))
        return encryptedBlocks
    def encrypt_message_RSA(self, keyFilename, message):
        '''
        Using a key from a key file, encrypt the message
        :param keyFilename:
        :param message:
        :return a dictionary with the encrypted message and more important
        details:
        '''
        keySize, n, e = self.readKeyFile(keyFilename)
        # Encrypt the message
        encryptedBlocks = self.encrypt_blocks(message, (n, e))
        # Convert the large int values to one string value.
        for i in range(len(encryptedBlocks)):
            encryptedBlocks[i] = str(encryptedBlocks[i])
        encryptedContent = ','.join(encryptedBlocks)

    encryptedContent={'length':len(message),'block_size':self.default_block_size, 'encrypted_content':encryptedContent}
        return encryptedContent
    def decrypt_message_RSA(self, encryptedContent, keyFilename):
        '''
        Using a key from a key file, gets the encrypted message and other
        details and then decrypt it.
        :param encryptedContent:
        :param keyFilename:
        :return decrypted message string:
        '''
        keySize, n, d = self.readKeyFile(keyFilename)
        messageLength, blockSize,
    encryptedMessage=encryptedContent['length'],encryptedContent['block_size'],encryptedContent['encrypted_content']

        # Convert the encrypted message into large int values.
        encryptedBlocks = []
        for block in encryptedMessage.split(','):
            encryptedBlocks.append(int(block))

        # Decrypt the large int values.
        return self.decrypt_blocks(encryptedBlocks, messageLength, (n, d))
    def decrypt_blocks(self, encryptedBlocks, messageLength, key):
        '''
        Decrypts a list of encrypted block ints into the original message
        string.
        :param encryptedBlocks:

```

```

:param messageLength:
:param key:
:return :
'''

decryptedBlocks = []
n, d = key
for block in encryptedBlocks:
    # plaintext = ciphertext ^ d mod n
    decryptedBlocks.append(pow(block, d, n))
return self.getTextFromBlocks(decryptedBlocks, messageLength)
def write_file_content(self, file_path, content):
    '''
    Writes content to a file.
    :param file_path:
    :param content:
    :param mode:
    :return Nothing:
    '''
    try:
        with open(file_path, "w", encoding='utf-8') as file:
            file.write(content)
    except:
        print(f"Error writing to the key file")
def read_file(self, file_path):
    '''
    Reads the content of a text file expected to contain a single
integer
and returns the content as an integer.
:param file_path:
:return the content as int:
'''
    with open(file_path, 'r', encoding='utf-8') as file:
        content = file.read()
        return content
def save_to_downloads(self, content, filename="public_key.txt"):
    '''
    Saves content to a file with the given filename in the user's
Downloads folder.
:param content:
:param filename:
:return the file path:
'''
    try:
        home_dir = os.path.expanduser("~")
        downloads_dir = os.path.join(home_dir, "Downloads")
        os.makedirs(downloads_dir, exist_ok=True)
        full_file_path = os.path.join(downloads_dir, filename)
        print(f"Attempting to save file to: {full_file_path}")
        if self.write_file_content(full_file_path, content):
            print(f"Successfully saved '{filename}' to the Downloads
folder.")
            return full_file_path
        else:

```

```

        print(f"Failed to save '{filename}' to the Downloads
folder.")

        return full_file_path
    except Exception as e:
        print(f"An error occurred while trying to save to Downloads:
{e}")

        return False

    @staticmethod
    def generate_aes_key():
        """
        generates a random AES key (32 bytes for AES-256)
        :return the generated AES key:
        """
        key = os.urandom(32)
        return key

    @staticmethod
    def generate_aes_iv():
        """
        generates a random IV (16 bytes for AES in CBC mode)
        :return the generated IV:
        """
        iv = os.urandom(16)
        return iv

    @staticmethod
    def encrypt_message_AES(message_bytes, aes_key, iv):
        """
        Encrypts the message using AES in CBC mode.
        :param message_bytes:
        :param aes_key:
        :param iv:
        :return The encrypted message (ciphertext):
        """
        # create an AES cipher object in CBC mode
        cipher = Cipher(algorithms.AES(aes_key), modes.CBC(iv),
backend=default_backend())
        # create an encryptor object
        encryptor = cipher.encryptor()
        # calculate padding to make sure the message's length is a multiple
of the AES block size, which is 16 bytes
        n = len(message_bytes)
        block_size = 16 # AES block size is 16 bytes
        padding_length = block_size - (n % block_size)
        # creating the padding bytes
        padding = bytes([padding_length]) * padding_length
        # adding the padding to the message
        message_bytes_padded = message_bytes + padding
        # encrypting the padded message
        ciphertext = encryptor.update(message_bytes_padded) +
encryptor.finalize()
        # return the encrypted message
        return ciphertext

    @staticmethod
    def decrypt_message_AES(ciphertext, aes_key, iv):

```

```

'''
Decrypts the message using AES in CBC mode.
:param ciphertext:
:param aes_key:
:param iv:
:return The decrypted message:
'''

# create an AES cipher object in CBC mode
cipher = Cipher(algorithms.AES(aes_key), modes.CBC(iv),
backend=default_backend())
# create a decryptor object
decryptor = cipher.decryptor()
# decrypt the ciphertext, includingg the added padding
padded_message = decryptor.update(ciphertext) +
decryptor.finalize()
# removing the added the padding
padding_length = padded_message[-1]
message = padded_message[:-padding_length].decode('utf-8')
return message

@staticmethod
def custom_encrypt_shift(data_bytes, shift_amount):
'''
Encrypts bytes by circularly shifting bits left within each byte.
shift_amount should be between 1 and 7.
:param data_bytes:
:param shift_amount:
:return the binary data after the shifting:
'''

encrypted_bytes = bytes(((byte << shift_amount) & 255) | (byte >>
(8 - shift_amount)) for byte in data_bytes)
return encrypted_bytes

@staticmethod
def custom_decrypt_shift(encrypted_bytes, shift_amount):
'''
Decrypts bytes by circularly shifting bits right within each byte.
This reverses a circular left shift by shift_amount.
shift_amount should be between 1 and 7.
:param encrypted_bytes:
:param shift_amount:
:return the binary data after the reshifting:
'''

decrypted_bytes = bytes(
    (byte >> shift_amount) | ((byte << (8 - shift_amount)) & 255)
    for byte in encrypted_bytes
)
return decrypted_bytes

```

מחלקת MailClient ומחלקות ה-gui השונות:

```
import socket
import struct
from EncryptionLib import HybridEncryption
from ServiceMainClass import ServiceMainClass
import random
import time
import json
import pygame
import os
import base64
import gzip
from datetime import datetime
import customtkinter as ctk
from PIL import Image
import sqlite3
from tkinter import messagebox, filedialog

class MailClient(ServiceMainClass):
    """
    The class which is responsible for the all the aspects of the clients
    in the service;
    Networking - establishing communication chanel with the server via TCP
    protocol, sending and receiving data to and from the server,
    and managing the service's communication protocol with the server.
    Encryption, Data security & Cybersecurity - creating 256-bits AES key
    with the init of the class and creating new iv for each message before
    sending it, also sending the bist shifting num.
    Gui - creating an instance of all the different frames/top levels
    classes and managing the use of them.
    DB - storing and fetching mail and their unique parameters and filters
    """
    def __init__(self):
        """
        init the MailClient class and all its attributes, also call to a
        number of functions in order to connect the server and setup the app's
        gui.
        """
        super().__init__()
        self.shifting_bits_num=random.randint(1, 7)
        self.aes_key=HybridEncryption.generate_aes_key()
        self.iv=b""
        self.encoded_aes_key=base64.b64encode(self.aes_key).decode('utf-8')
        self.encryption_ins=HybridEncryption()
        self.socket = None
        self.db_error=False
        self.email = "None"
        self.connected = False
        self.conn=None
        self.cursor=None
        self.blocked_users_names=[]
        self.distribution_list1_names=[]
```

```

self.distribution_list2_names=[]
self.distribution_list3_names=[]
self.labels_names=["temp"]
self.new_blocked=False
self.violations=0
self.screen_height=835
self.screen_width=895
self.screen_size="default"
self.current_color="blue"
self.current_color_hex=0
self.app_colors_names = ["default color", # i used list because ctk
menu cannot use dictionaries
                        "blue",
                        "orange",
                        "black",
                        "red"]

self.app_colors_RGB=[(5, 144,
237), (9,12,255), (255,146,51), (0,0,0), (255,70,70)]
self.root = ctk.CTk()
self.root.title("DaVinci Mail Service")
self.setup_gui()
self.connect_to_server()
self.root.protocol("WM_DELETE_WINDOW", lambda:
self.close_while_running())
self.root.mainloop()

def connect_to_server(self):
    """
    Connect to the mail server using the specified host and port via
    TCP protocol, if the connection is not accepted; updates the client and
    close
    the app. If the connection is accepted; gets the sever's public
    key, save it and then encrypts the client's AES key and bit shifting num,
    after that -
    send them to the server. If the server isn't available there will
    be endless loop which every 15 seconds will try to connect to the server,
    unless the user will close the app.
    if the saving of the server's public key wasn't succeeded there
    will be a message to close the app.
    :return Nothing:
    """
    try: # service 'Handshake'
        # connect to server
        self.socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.socket.connect((self.host, self.port))
        # gets if the server can get more users
        length = self.socket.recv(4)
        length = struct.unpack('>I', length)[0]
        response = self.socket.recv(length)
        response = gzip.decompress(response).decode('utf-8')
        response = json.loads(response)
        if response['accept']==True: # the server can get more users
            # gets the server's public key
            length = self.socket.recv(4)

```

```

        length = struct.unpack('>I', length)[0]
        response=self.socket.recv(length)
        json_data = gzip.decompress(response).decode('utf-8')
        sever_public_key = json.loads(json_data)
        # saves the key in the downloads folder and gets the path
full_file_path=self.encryption_ins.save_to_downloads(sever_public_key)
        if full_file_path==False: # There was error in the save
process, the app can't keep running
            raise
            # encrypting the client's aes key and then sending it.

encrypted_aes_key=self.encryption_ins.encrypt_message_RSA(full_file_path,s
self.encoded_aes_key)
            # adding a number to the shifting bits in each byte num
according to the ip address of the host in order to encrypt the shifting
num
            ip_parts_str = self.host.split('.') # splits the ip address
of the host for four string in a list
            ip_tuple_int = tuple(int(part) for part in ip_parts_str) #
convert them to int
            sum=0
            for i in ip_tuple_int:
                sum+=((i*7)+13)*2
            # sending the client encrypted AES key and the encrypted
bits shifting num
            data = {'encrypted_key': encrypted_aes_key, 'num':
(self.shifting_bits_num + sum)}
            json_data = json.dumps(data) # Convert dict to JSON string
            compressed_data = gzip.compress(json_data.encode('utf-8'))
# Compress the JSON string
            length_bytes = struct.pack('>I',len(compressed_data)) #
Pack length as a big-endian unsigned integer (4 bytes)
            self.socket.sendall(length_bytes) # sends the len of the
message in bytes
            self.socket.sendall(compressed_data) # sends the message
itself

            self.connected = True
        else:
            messagebox.showinfo("Server Limit","We are sorry, but the
server has too many connections, it can't serve you right now.")
            self.root.destroy()
        except (ConnectionResetError, BrokenPipeError, socket.error):
            messagebox.showerror("Connection Error", "Unable to connect to
the server, Retrying in 15 seconds...")
            time.sleep(15)
            self.connect_to_server()
        except:
            messagebox.showerror("Error", "Error in saving necessary data,
please close the app and restart your device")
    def init_db(self):
        '''

```



```

        Initialize the SQLite database and create required table if doesn't
        exist.

        :return:
        '''
        try:
            self.conn = sqlite3.connect('user_messages.db')
            self.cursor = self.conn.cursor()
            self.cursor.execute('''CREATE TABLE IF NOT EXISTS messages (
                                id INTEGER PRIMARY KEY
                                AUTOINCREMENT,

                                sender TEXT,
                                title TEXT,
                                message TEXT,
                                timestamp TEXT,
                                important INT DEFAULT 0,
                                readlater INT DEFAULT 0,
                                deleted INT DEFAULT 0,
                                file_data TEXT,
                                image_data TEXT,
                                labels TEXT
                                )''')

            self.conn.commit()
        except:
            self.db_error=True
            messagebox.showerror("Data Base Error","There is problem in the
service's Data Base, please log out, and try to connect later")
        def empty_db(self):
            '''
            empty the user data base
            :return:
            '''
            try:
                self.cursor.execute(f'DELETE FROM messages')
                self.conn.commit()
            except:
                self.db_error=True
                messagebox.showerror("Data Base Error","There is problem in the
service's Data Base, please log out, and try to connect later")
        def send_action(self,data):
            '''
            Send a request to the server and receive a response; activates the
            delay mechanism, generates a new IV, call the function whom encrypt and
            compress the data,
            sends the generated iv, the encrypted and compressed data length,
            then receive the response length, after that the response itself, then
            call the function whom decrypts
            and decompress the data - getting the response form the server.
            :param data: The data to be sent to the server.
            :return: The response from the server.
            '''
            try:
                time.sleep(random.uniform(0.1, 0.4))

```

```

        self.iv = HybridEncryption.generate_aes_iv() # generated new
iv for the new message
        compressed_data = self.prepare_message(data, self.aes_key,
self.iv, self.shifting_bits_num)
        length_bytes = struct.pack('>I',
                                len(compressed_data)) # Pack length
as a big-endian unsigned integer (4 bytes)
        self.socket.sendall(self.iv) # generated new iv for the new
message
        self.socket.sendall(length_bytes) # sends the len of the
message in bytes
        self.socket.sendall(compressed_data) # sends the message
        # getting the response message iv
        self.iv = self.socket.recv(16)
        # getting the response's message len in bytes
        response = self.socket.recv(4)
        length = struct.unpack('>I', response)[0]
        # getting response in bytes
        response = self.socket.recv(length)
        # decompressing response
        response = self.treat_message(response, self.aes_key, self.iv,
self.shifting_bits_num)
        return response
    except (socket.error, ConnectionResetError, BrokenPipeError):
        self.connected=False
        return {'status': 'error', 'message': 'Not connected to the
server, please reopen the app.'}
    except:
        self.connected=False
        return {'status': 'error', 'message': 'Sudden error, please
reopen the app.'}
    def register_user(self, user_data):
        """
        Step in the process of register user, sets the action in the
assigned dictionary to "register" and pass the updated dictionary to the
"sent_action" function.
        :param user_data:
        :return if connected to the server - the server response
(response), else a corresponding message:
        """
        if self.connected:
            user_data['action'] = 'register'
            response = self.send_action(user_data)
            return response
        else:
            return {'status': 'error', 'message': 'Not connected to the
server, please reopen the app.'}
    def login_user(self, login_data):
        """
        Step in the process of login user, sets the action in the assigned
dictionary to "login" and pass the updated dictionary to the "sent_action"
function.
        :param login_data - The data containing user login details:

```

```

        :return if connected to the server - the server response
(response), else a corresponding message:
'''
    if self.connected:
        login_data['action'] = 'login'
        response=self.send_action(login_data)
        return response
    else:
        return {'status': 'error', 'message': 'Not connected to the
server, please reopen the app.'}
    def fetch_organization(self,rows):
        '''
        organizes all the mail whom have been fetched from the DB as list
of dictionaries
        :param rows:
        :return the mails as list of dictionaries, or Nothing:
        '''
        try:
            messages = []
            for row in rows:
                message = {
                    'id': row[0],
                    'sender': row[1],
                    'title': row[2],
                    'message': row[3],
                    'timestamp': row[4],
                    'label':row[10]
                }
                messages.append(message)
            return messages
        except sqlite3.OperationalError:
            self.init_db()
            self.db_error=True
            messagebox.showerror("Data Base Error","There is problem in the
service's Data Base, please log out, and try to connect later" )
            return [] # return an empty list to avoid errors
    def fetch_all_messages(self,including_deleted=False):
        '''
        if the parameter is False - fetch all the messages except the
deleted and blocked ones from the database, if the parameter is True ALL
of the messages will be fetched.
        :param including_deleted:
        :return fetched messages from the database and as a list of
dictionaries:
        '''
        try:
            if including_deleted:
                self.cursor.execute('SELECT * FROM messages')
                rows = self.cursor.fetchall()
            else:
                self.cursor.execute('SELECT * FROM messages WHERE
deleted=0')
                rows = self.cursor.fetchall()

```

```

        messages = []
        for row in rows:
            message = {
                'id': row[0],
                'sender': row[1],
                'title': row[2],
                'message': row[3],
                'timestamp': row[4],
                'important': row[5],
                'readlater': row[6],
                'deleted': row[7],
                'file_data': row[8],
                'image_data': row[9],
                'labels': row[10],
            }
            messages.append(message)
        for msg in messages:
            if msg['sender'] in self.blocked_users_names:
                messages.remove(msg)
        return messages
    except sqlite3.OperationalError:
        self.init_db()
        self.db_error = True
        messagebox.showerror("Data Base Error",
                               "There is problem in the service's Data
Base, please log out, and try to connect later")
        return [] # return an empty list to avoid errors
    def fetch_messages_by_category(self, category, serach_paramter=None):
        """
        Fetch all messages from the data base by a specific category.
        :param category:
        :param search_parameter, required only when the user want to search
specific text in content, when the user want the mails by specific sender
or by specific date:
        :return fetched messages as a list of dictionaries:
        """
        try:
            if category == "specific":
                self.cursor.execute('SELECT * FROM messages WHERE
sender=?', (serach_paramter,))
                rows = self.cursor.fetchall()
                return self.fetch_organization(rows)
            elif category == "important":
                self.cursor.execute('SELECT * FROM messages WHERE
important=1')
                rows = self.cursor.fetchall()
                return self.fetch_organization(rows)
            elif category == "saved for later":
                self.cursor.execute('SELECT * FROM messages WHERE
readlater=1')
                rows = self.cursor.fetchall()
                return self.fetch_organization(rows)
            elif category == "deleted":

```

```

        self.cursor.execute('SELECT * FROM messages WHERE
deleted=1')

        rows = self.cursor.fetchall()
        return self.fetch_organization(rows)
    elif category == "search":
        search_query = f"%{serach_paramter}%" # Create a search
pattern to match any text containing 'search_text'
        self.cursor.execute('SELECT * FROM messages WHERE message
LIKE ?', (search_query,))
        rows = self.cursor.fetchall()
        return self.fetch_organization(rows)
    elif category == "by date":
        print(serach_paramter)
        self.cursor.execute('SELECT * FROM messages WHERE
date(timestamp) = ?', (serach_paramter,))
        rows = self.cursor.fetchall()
        return self.fetch_organization(rows)
    elif category == "label":
        self.cursor.execute('SELECT * FROM messages')
        rows = self.cursor.fetchall()
        messages = self.fetch_organization(rows)
        message_that_labeled = [] # if we try to remove the
message which wasn't labeled through the loop iteration, the index will get
wrong, so we will miss some message in the checking
        for message in messages:
            if message['label'][serach_paramter] == 'Y':
                message_that_labeled.append(message)
        return message_that_labeled

    except sqlite3.OperationalError:
        self.init_db()
        self.db_error = True
        messagebox.showerror("Data Base Error",
                             "There is problem in the service's Data
Base, please log out, and try to connect later")
        return [] # return an empty list to avoid errors

    def setup_gui(self):
        """
        Set up the gui (graphical user interface) for the user and decided
the app's screen size.
        :return Nothing:
        """
        try:
            self.root.iconbitmap("app_icon.ico")
        except:
            print("The icon's file isn't available")
        if self.screen_height > self.root.wininfo_screenheight(): # checking
if the screen is small
            self.screen_height = self.root.wininfo_screenheight()-100
            self.screen_size = "small"
        self.root.geometry(f'{self.screen_width}x{self.screen_height}')
        self.frames = {}

```

```

        for F in (LoginFrame, RegisterFrame, LoadingFrame, MainFrame,
SettingsFrame, PolicyFrame):
            frame = F(parent=self.root, controller=self)
            self.frames[F] = frame
            frame.grid(row=0, column=0, sticky='nsew')
            self.frames[SettingsFrame].change_button_color() # changes the
color of all the necessary widgets to the default color
            self.show_frame(LoginFrame)
        def show_frame(self, frm):
            """
            Display the specified frame in the GUI.
            :param frm: The frame class to be shown:
            :return:
            """
            frame = self.frames[frm]
            frame.tkraise()
        def send_message(self, message_data):
            """
            Step in the process sending a new mail, sets the action in the
assigned dictionary to "send_message" and pass the updated dictionary to
the "sent_action" function.
            :param message_data - The data containing user login details:
            :return if connected to the server - the server response
(response), else a corresponding message:
            """
            if self.connected:
                message_data['action'] = 'send_message'
                print(message_data)
                response = self.send_action(message_data)
                return response
            else:
                return {'status': 'error', 'message': 'Not connected to the
server, please reopen the app.'}
        def get_messages(self):
            """
            Step in the process sending a request to retrieve new mails, sets
the action in the assigned dictionary to "retrieve_messages" and pass the
updated dictionary to the "sent_action" function.
            :param:
            :return if connected to the server - the server response
(response), else a corresponding message:
            """
            if self.connected:
                request_data = {'action': 'retrieve_messages', 'recipient':
self.email}
                response = self.send_action(request_data)
                return response
            else:
                return {'status': 'error', 'message': 'Not connected to the
server, please reopen the app.'}
        def logout(self):
            """
            Log out the current user, sending to the server necessary data,

```

```

        closing the current socket, initializes a number of the class
        attributes, reconnect to the socket by calling the "connect_to_server"
        function
        (as part of it generating a new AES key) and go back to the entry
        frame.
        :return Nothing:
        '''
        if self.connected:
            data = {'action': 'logout',
                    'mail': self.email,
                    'blocked_users': self.blocked_users_names,
                    'messages': [], # an empty list, if there is an error
in the user's db, at least the client will send the available data
                    'color': self.current_color_hex,
                    'violations': self.violations,
                    'dist1': self.distribution_list1_names,
                    'dist2': self.distribution_list2_names,
                    'dist3': self.distribution_list3_names,
                    'labels_names': self.labels_names
                    }

            if self.db_error == False:
                data['messages'] = self.fetch_all_messages(True)
                self.iv = HybridEncryption.generate_aes_iv() # generated new
iv for the new message
                compressed_data = self.prepare_message(data, self.aes_key,
self.iv, self.shifting_bits_num)
                length_bytes = struct.pack('>I', len(compressed_data)) # Pack
length as a big-endian unsigned integer (4 bytes)
                self.socket.sendall(self.iv) # generated new iv for the new
message
                self.socket.sendall(length_bytes) # sends the len of the
message in bytes
                self.socket.sendall(compressed_data)
                # getting the response message iv
                self.iv = self.socket.recv(16)
                # getting the response's message len in bytes
                response = self.socket.recv(4)
                length = struct.unpack('>I', response)[0]
                # getting response in bytes
                response = self.socket.recv(length)
                # treat response
                response = self.treat_message(response, self.aes_key, self.iv,
self.shifting_bits_num)
                if response['status'] == "success":
                    messagebox.showinfo("Logout", f"{response['message']}")
                    self.blocked_users_names.clear()
                    self.socket.close()
                    self.empty_db() # empty the db from the user data
                    self.email = "None"
                    self.connected = False
                    self.socket = None
                    self.shifting_bits_num = random.randint(1, 7) # geneartes a
new shifting bits num

```

```

        self.frames[LoginFrame].clear_entries() # clear the login
entries when logging out
        self.connect_to_server()
        self.show_frame(LoginFrame)
    else:
        messagebox.showerror("Logout",f"{response['message']}")
        self.socket.close()
        self.empty_db() # empty the db from the user data
        self.root.destroy()
    else:
        self.empty_db() # empty the user db
        messagebox.showerror("Connection error","Please close the app")
def close_while_running(self):
    """
    When closing the window while running by pressing on the "X"
button, we will activate some steps of the same process as
logout by sending the server necessary data and closing the
program.
    :return Nothing:
    """
    if self.connected:
        if self.email != "None": # if the mail is not none, the user
is not on the entry frame, meaning - the user have logged in
            data={'action': 'logout',
                  'mail': self.email,
                  'blocked_users': self.blocked_users_names,
                  'messages': [],
                  'color': self.current_color_hex,
                  'violations': self.violations,
                  'dist1': self.distribution_list1_names,
                  'dist2': self.distribution_list2_names,
                  'dist3': self.distribution_list3_names,
                  'labels_names': self.labels_names
                  }
            if self.db_error==False:
                data['messages']= self.fetch_all_messages(True)
                self.iv = HybridEncryption.generate_aes_iv() # generated
new iv for the new message
                compressed_data = self.prepare_message(data, self.aes_key,
self.iv,
self.shifting_bits_num) # Compress the encrypted string
                length_bytes = struct.pack('>I',
                                            len(compressed_data)) # Pack
length as a big-endian unsigned integer (4 bytes)
                self.socket.sendall(self.iv) # generated new iv for the
new message
                self.socket.sendall(length_bytes) # sends the len of the
message in bytes
                self.socket.sendall(compressed_data)
                self.empty_db() # empty the user db
            else:
                data = {'action': 'close before logged in'}

```



```

        self.iv = HybridEncryption.generate_aes_iv() # generated
new iv for the new message
        compressed_data = self.prepare_message(data, self.aes_key,
self.iv, self.shifting_bits_num) # Compress the encrypted string
        length_bytes = struct.pack('>I', len(compressed_data)) #
Pack length as a big-endian unsigned integer (4 bytes)
        self.socket.sendall(self.iv) # generated new iv for the
new message
        self.socket.sendall(length_bytes) # sends the len of the
message in bytes
        self.socket.sendall(compressed_data)
        self.socket.close()
        self.root.destroy()
    else:
        messagebox.showerror("Connection error", "The app will be
closed")
        self.socket.close()
        self.empty_db() # empty the user db
        self.root.destroy()
class LoadingFrame(ctk.CTkFrame):
    def __init__(self, parent, controller):
        '''
        init the SettingsFrame class.
        :param parent:
        :param controller:
        '''
        ctk.CTkFrame.__init__(self, parent)
        self.controller = controller
        label = ctk.CTkLabel(self, text="Loading Your Data (:)",
font=("Calibri", 40, "bold"))
        label.pack(pady=10)

        self.progress_bar = ctk.CTkProgressBar(self)
        self.progress_bar.pack(pady=20)
        # Initialize the progress bar value
        self.progress_bar.set(0)

        try:
            if self.controller.screen_size == "small":
                width, height = 380, 220
            else:
                width, height = 500, 430
            image = Image.open(r"logo.jpeg")
            ctk_image = ctk.CTkImage(light_image=image, size=(width,
height))
            label_image = ctk.CTkLabel(self, image=ctk_image, text="")
            label_image.image = ctk_image # Keep a reference to avoid
garbage collection
            label_image.pack(pady=10)
        except FileNotFoundError:
            self.image_label = ctk.CTkLabel(self.master, text="[Image Not
Found]", font=("Calibri", 12))
            self.image_label.grid(row=1, column=0, pady=20)

```

```

        try:
            # Initialize pygame mixer and play sound
            pygame.mixer.init()
            pygame.mixer.music.load(r"loading_melody.mp3") # Replace with
your sound file path
        except:
            print("The mp3 file is not available")
    def start_sound(self):
        """
        starts the loading sound and calling the function which creates the
animation for the progress bar
        :return:
        """
        try:
            pygame.mixer.music.play()
        except:
            print("The mp3 file is not available")
        self.update_progress()
        # a method used to schedule a function to run after a specified amount
of time, without blocking the main UI thread.
    def update_progress(self, step=0):
        """
        Create the animation of the progress bar, when it is finished, the
main page will appear
        :param step:
        :return:
        """
        if step <= 100:
            self.progress_bar.set(step / 100)
            self.after(15, self.update_progress, step + 1) # Update every
15ms for smooth animation
        else:
            self.controller.show_frame(MainFrame)
            self.controller.frames[MainFrame].update_email() # updates the
mail of the user for the interface
            self.controller.frames[MainFrame].refresh_messages()
            try:
                pygame.mixer.music.stop()
            except:
                print("The mp3 file is not available")
class RegisterFrame(ctk.CTkFrame):
    def __init__(self, parent, controller):
        """
        init the RegisterFrame class, setting up the registration
interface.
        :param parent:
        :param controller:
        """
        ctk.CTkFrame.__init__(self, parent)
        self.controller = controller
        password_policy="Password requirements;\nAt least six
characters, one capital letter and one small letter.\nPassword

```

```

recommendations;\nFor a strong password, use at least 12-16 characters
with a mix of uppercase and lowercase letters,\nnnumbers, and special
symbols (#,%,!), avoiding personal info, common words, and password
reuse."

    label = ctk.CTkLabel(self, text="Register", font=("Calibri",
40,"bold"))
    label.pack(pady=10)

    self.name_entry = ctk.CTkEntry(self, width=300,
placeholder_text="Name")
    self.name_entry.pack(pady=5)

    self.password_entry = ctk.CTkEntry(self, width=300,
placeholder_text="Password", show="*")
    self.password_entry.pack(pady=5)

    self.second_password_entry = ctk.CTkEntry(self, width=300,
placeholder_text="Enter password again", show="*")
    self.second_password_entry.pack(pady=5)

    self.birthday_date_entry = ctk.CTkEntry(self, width=300,
placeholder_text="Enter your date of birth, dd/mm/yyyy")
    self.birthday_date_entry.pack(pady=5)

    self.security_qs = [
        "What was your childhood nickname?",
        "What is your favorite pet?",
        "What is your mother's maiden name?",
        "What was your favorite subject in high school?"
    ]

    self.security_q1_var = ctk.StringVar(value=self.security_qs[0])
    self.security_q2_var = ctk.StringVar(value=self.security_qs[1])

    self.security_q1_menu = ctk.CTkOptionMenu(self,
variable=self.security_q1_var, values=self.security_qs)
    self.security_q1_menu.pack(pady=5)

    self.security_a1_entry = ctk.CTkEntry(self, width=300,
placeholder_text="Answer 1")
    self.security_a1_entry.pack(pady=5)

    self.security_q2_menu = ctk.CTkOptionMenu(self,
variable=self.security_q2_var, values=self.security_qs)
    self.security_q2_menu.pack(pady=5)

    self.security_a2_entry = ctk.CTkEntry(self, width=300,
placeholder_text="Answer 2")
    self.security_a2_entry.pack(pady=5)

    register_button = ctk.CTkButton(self, text="Register",
command=self.register)
    register_button.pack(pady=10)

```

```

        back_button = ctk.CTkButton(self, text="Back to Login",
command=self.back_button)
        back_button.pack(pady=5)

        welcome_label = ctk.CTkLabel(self, text=password_policy,
font=("Calibri", 20))
        welcome_label.pack(pady=5)
    def clear_entries(self):
        """
        clears all the frame's entries
        :return:
        """
        self.name_entry.delete(0, ctk.END)
        self.password_entry.delete(0, ctk.END)
        self.second_password_entry.delete(0, ctk.END)
        self.birthday_date_entry.delete(0, ctk.END)
        self.security_a1_entry.delete(0, ctk.END)
        self.security_a2_entry.delete(0, ctk.END)
    def back_button(self):
        """
        Clears the entries in the login, register frames and returns to the
login frame
        :return:
        """
        self.clear_entries()
        self.controller.frames[LoginFrame].clear_entries() # clear the
login entries when logging out
        self.controller.show_frame(LoginFrame)
    def is_valid_date(self):
        try:
            # Try to parse the date string with the given format
            datetime.strptime(self.birthday_date_entry.get(), "%d/%m/%Y")
            return True
        except ValueError:
            # If parsing fails, the date is not valid
            return False
    def register(self):
        """
        handles the user registration and pass the registration data to the
next func if the details are according to the requirments.
        :return:
        """
        name = self.name_entry.get()
        password = self.password_entry.get()
        password_2 = self.second_password_entry.get()
        security_q1 = self.security_q1_var.get()
        security_a1 = self.security_a1_entry.get()
        security_q2 = self.security_q2_var.get()
        security_a2 = self.security_a2_entry.get()
        birthday_date = self.birthday_date_entry.get()
        strong_password_check_result =
ServiceMainClass.strong_password(password) # Rename for clarity

```

```

        if not (name and password and password_2 and security_q1 and
security_a1 and security_q2 and security_a2 and birthday_date):
            messagebox.showwarning("Input Error", "Please fill all
fields.")
            return # stop the func
        if security_q1 == security_q2:
            messagebox.showerror(title="Attention", message="You chose the
same two security questions!")
            return
        if name == "Service_Official":
            messagebox.showerror(title="Attention", message="You can't use
this mail address!")
            return
        if not all(char not in name for char in
self.controller.forbidden_characters):
            messagebox.showerror(title="Attention", message="Forbidden
characters in mail address name!")
            return
        if name.isdigit():
            messagebox.showerror(title="Attention", message="Mail address
name cannot be only numbers!")
            return
        if ServiceMainClass.check_content(name):
            messagebox.showerror(title="Attention",
                                message="Mail address name shouldn't
contain inappropriate words!")
            return
        if len(name) < 3:
            messagebox.showerror(title="Attention", message="Mail address
name should be three characters or above.")
            return
        if strong_password_check_result is not True: # strong_password
returns a message if not True
            messagebox.showerror(title="Attention",
                                message=strong_password_check_result)
            return
        if password != password_2:
            messagebox.showerror(title="Attention", message="Passwords do
not match. Please check for mistakes.")
            return
        if not self.is_valid_date():
            messagebox.showerror(title="Attention", message="Invalid date
format. Please check your input.")
            return
        hashed_password = self.controller.hash_password(password)
        email = f"{name}@DaVinci.cl" # Generate email
        # Collect user registration data
        user_data = {
            'email': email,
            'password': hashed_password, # Send the hashed password
            'security_q1': security_q1,
            'security_a1': security_a1,
            'security_q2': security_q2,

```

```

        'security_a2': security_a2,
        'birthday_date': birthday_date
    }
    response = self.controller.register_user(user_data)
    if response['status'] == 'success':
        messagebox.showinfo("Registration Successful", f"Your email is {email}")
        self.clear_entries()
        self.controller.show_frame(LoginFrame)
    else:
        messagebox.showerror("Registration Failed",
response['message'])
class LoginFrame(ctk.CTkFrame):
    def __init__(self, parent, controller):
        ctk.CTkFrame.__init__(self, parent)
        self.controller = controller
        label = ctk.CTkLabel(self, text="DaVinci Mail", font=("Calibri",
40, "bold"))
        label.pack(pady=5)

sentence=self.controller.DaVinci_quotes[random.randint(0,len(self.controller.DaVinci_quotes)-1)]
        if len(sentence)<=100:
            label = ctk.CTkLabel(self, text=f"The daily quote: {sentence}",
font=("Calibri", 15, "bold"))
            label.pack(pady=5)
        else:
            label = ctk.CTkLabel(self, text=f"The daily quote: {sentence}",
font=("Calibri", 13, "bold"))
            label.pack(pady=5)

        label = ctk.CTkLabel(self, text="Login", font=("Calibri", 25,
"bold"))
        label.pack(pady=5)

        self.email_entry = ctk.CTkEntry(self, width=300,
placeholder_text="Email (enter only your name)")
        self.email_entry.pack(pady=5)

        self.password_entry = ctk.CTkEntry(self, width=300,
placeholder_text="Password", show="*")
        self.password_entry.pack(pady=5)

        login_button = ctk.CTkButton(self, text="Login",
command=self.login)
        login_button.pack(pady=5)

        register_button = ctk.CTkButton(self, text="Register",
command=lambda: controller.show_frame(RegisterFrame))
        register_button.pack(pady=5)

        # Forgot Password button

```

```

        self.forgot_password_button = ctk.CTkButton(self, text="Forgot
Password?", command=self.open_password_recovery_window)
        self.forgot_password_button.pack(pady=5)

    try:
        if self.controller.screen_size == "small":
            width, height = 380, 220
        else:
            width, height = 500, 430
        image = Image.open("login_page_image.png")
        ctk_image = ctk.CTkImage(light_image=image, size=(width,
height))

        label_image = ctk.CTkLabel(self, image=ctk_image, text="")
        label_image.image = ctk_image # Keep a reference to avoid
garbage collection
        label_image.pack(pady=10)
    except FileNotFoundError:
        self.image_label = ctk.CTkLabel(self.master, text="[Image Not
Found]", font=("Calibri", 12))
        self.image_label.grid(row=1, column=0, pady=20)

    def open_password_recovery_window(self):
        password_recovery_window = PasswordRecoveryWindow(self.controller)
        password_recovery_window.grab_set() # the user can't use the login
frame until the password recovery top level is closed

    def clear_entries(self):
        self.email_entry.delete(0, ctk.END)
        self.password_entry.delete(0, ctk.END)

    def change_button_color(self):
        '''
        Change the color of all the buttons to the last color.
        :return:
        '''
        color=self.controller.current_color_hex
        all_widgets = self.controller.root.winfo_children()
        for widget in all_widgets:
            if isinstance(widget, ctk.CTkButton) or isinstance(widget,
ctk.CTkOptionMenu):
                widget.configure(fg_color=color)
                all_widgets.extend(widget.winfo_children())

    def login(self):
        '''
        handle user login by sending login data to the server, gets a
number of data strings and convert them to lists,
        if login is accepted showing the main frame, else show why the
login failed.
        '''
        email = self.email_entry.get()
        password = self.password_entry.get()
        print(password)
        if email and password and not all(char not in email for char in
self.controller.forbidden_characters):
            messagebox.showerror(title="Attention", message="Forbidden
characters")

```

```

elif email and password:
    # Collect login data and send it to the server, we send the
    regular password and not the hashed one on intention, we will verify if
    they are the same in the sever
    login_data = {'email': (email + "@DaVinci.cl"), 'password':
password}
    response = self.controller.login_user(login_data)
    if response['status'] == 'success':
        self.controller.current_color_hex=response['last_color']
#updates the user last color
        try:
            self.change_button_color()
        except:
            print("the color is default")
            self.controller.labels_names = (
                response['labels_names'].split("/")) # updates the
user's labels name list
            print(self.controller.labels_names)
        try:
            self.controller.blocked_users_names = (
                response['blocked_users'].split("/")) # updates
the user's blocked users list
        except:
            print("the blocked user list is empty")
        try:
            self.controller.distribution_list1_names = (
                response['dist1'].split("/")) # updates the user's
first distribution list
        except:
            print("the dist1 list is empty")
        try:
            self.controller.distribution_list2_names = (
                response['dist2'].split("/")) # updates the user's
second distribution list
        except:
            print("the dist2 list is empty")
        try:
            self.controller.distribution_list3_names = (
                response['dist3'].split("/")) # updates the user's
third distribution list
        except:
            print("the dist3 list is empty")
            self.controller.init_db() # init the db for the user
            self.controller.email=email+ "@DaVinci.cl" # sets the email
to the client instance
            self.controller.show_frame>LoadingFrame) # shows the
loading frame
            self.controller.frames>LoadingFrame).start_sound() #
starts the sound and the progress bar
            self.controller.frames>MainFrame).update_label_menu() #
updates the label menu according to user's assignd names
    else:
        messagebox.showerror("Login Failed", response['message'])

```



```

        else:
            messagebox.showwarning("Input Error", "Please enter email and
password.")
class PasswordRecoveryWindow(ctk.CTkToplevel):
    def __init__(self, controller):
        '''
        init PasswordRecoveryWindow frame class
        :param controller:
        '''
        ctk.CTkToplevel.__init__(self)
        self.controller=controller
        self.title("Password Recovery")
        self.geometry("400x500")
        self.response=""
        self.q1=""
        self.q2=""

        # Step 1: Enter Email
        self.email_label = ctk.CTkLabel(self, text="Enter your email:",
font=("Calibri", 25))
        self.email_label.pack(pady=10)
        self.email_entry = ctk.CTkEntry(self)
        self.email_entry.pack(pady=5)
        self.submit_email_button = ctk.CTkButton(self, text="Submit",
command=self.submit_email)
        self.submit_email_button.pack(pady=10)

        # Security Questions Frame (initially hidden)
        self.security_questions_frame = ctk.CTkFrame(self)
    def submit_email(self):
        '''
        Submit the email to the server to retrieve security questions.
        :return:
        '''
        if self.controller.connected:
            email = self.email_entry.get()
            if email and "@DaVinci.cl" in email:
                request = {
                    "action": "forgot_password",
                    "email": email
                }
                response = self.controller.send_action(request)
                if response['status'] == 'success':
                    self.q1, self.q2 = response['security_q1'],
response['security_q2']
                    if self.response != "success":
                        self.display_security_questions()
                    else: # the user got mistkae with the mail or want to
reset passowrd of anotehr mail, so we need to chnage the questions
according the new account

self.security_a1_entry.configure(placeholder_text=self.q1)

```

```

self.security_a2_entry.configure(placeholder_text=self.q2)
    else:
        messagebox.showerror("Error", response['message'])
    else:
        messagebox.showerror("Input error", "You didn't enter any
address, or the mail address isn't in the service's format")
    else:
        messagebox.showerror("Error", 'Not connected to the server,
please reopen the app.')
def display_security_questions(self):
    """
    display the security questions that was retrieved from the server.
    :return:
    """
    self.response="success"
    self.security_questions_frame.pack(pady=10)
    # Step 2: Display Security Questions

    self.security_a1_entry =
ctk.CTkEntry(self.security_questions_frame,width=300,placeholder_text=self
.q1)
    self.security_a1_entry.pack(pady=5)

    self.security_a2_entry =
ctk.CTkEntry(self.security_questions_frame,width=300,placeholder_text=self
.q2)
    self.security_a2_entry.pack(pady=5)

    self.new_password_entry =
ctk.CTkEntry(self.security_questions_frame,
show='*',width=300,placeholder_text="Enter new password")
    self.new_password_entry.pack(pady=5)

    self.new_password_entry_confirmation =
ctk.CTkEntry(self.security_questions_frame,
show='*',width=300,placeholder_text="Enter password again")
    self.new_password_entry_confirmation.pack(pady=5)

    self.reset_password_button =
ctk.CTkButton(self.security_questions_frame, text="Reset
Password",command=self.reset_password)
    self.reset_password_button.pack(pady=10)
    def reset_password(self):
        """
        getting the user's answers for the the security question and his
new password
        :return:
        """
        if self.controller.connected:
            email = self.email_entry.get()
            answer1 = self.security_a1_entry.get()
            answer2 = self.security_a2_entry.get()

```

```

        new_password = self.new_password_entry.get()
        new_password_confirmation =
self.new_password_entry_confirmation.get()
        if not (email and answer1 and answer2 and new_password and
new_password_confirmation):
            messagebox.showerror("Input error", "You didn't filled all
the entries")
            return
        if new_password_confirmation != new_password:
            messagebox.showerror("Error", "The confirmation password
and the first one are not the same")
            return
        strong_password =
ServiceMainClass.strong_password(new_password)
        if strong_password != True:
            messagebox.showerror("Error", message=strong_password)
            return
        if "@DaVinci.cl" not in email:
            messagebox.showerror("Input error", "The mail address isn't
in the service's format")
            return

        request = {
            "action": "reset_password",
            "email": email,
            "security_a1": answer1,
            "security_a2": answer2,
            "new_password": self.controller.hash_password(new_password)
        }
        response = self.controller.send_action(request)
        if response['status'] == 'success':
            messagebox.showinfo("Success", response['message'])
            self.destroy()
        else:
            messagebox.showerror("Error", response['message'])
    else:
        messagebox.showerror("Error", 'Not connected to the server,
please reopen the app.')
class SettingsFrame(ctk.CTkFrame):
    def __init__(self, parent, controller):
        '''
        init the SettingsFrame class
        :param parent:
        :param controller:
        '''
        ctk.CTkFrame.__init__(self, parent)
        self.controller = controller

        label = ctk.CTkLabel(self, text="Settings", font=("Calibri",
40, "bold"))
        label.pack(pady=10)

```

```

        self.color_var =
ctk.StringVar(value=self.controller.app_colors_names[0])
        self.color_menu = ctk.CTkOptionMenu(self, variable=self.color_var,
values=self.controller.app_colors_names)
        self.color_menu.pack(pady=5,padx=5)

        self.add_blocked_user_button=ctk.CTkButton(self, text="Block user",
command=self.add_blocked_user)
        self.add_blocked_user_button.pack(pady=5,)

        self.remove_blocked_user_button=ctk.CTkButton(self, text="Remove
blocked user", command=self.remove_blocked_user)
        self.remove_blocked_user_button.pack(pady=5,)

        self.dist_menu_possibilities=['first distribution','second
distribution','third distribution']
        self.dist_var =
ctk.StringVar(value=self.dist_menu_possibilities[0])
        self.dist_menu = ctk.CTkOptionMenu(self, variable=self.dist_var,
values=self.dist_menu_possibilities)
        self.dist_menu.pack(pady=5, padx=5)

        self.add_user_to_dist_list_button = ctk.CTkButton(self, text="Add
to distribution list", command=self.add_to_dist_list)
        self.add_user_to_dist_list_button.pack(pady=5, )

        self.remove_user_from_dist_list_button = ctk.CTkButton(self,
text="Remove from distribution list",command=self.remove_added_user)
        self.remove_user_from_dist_list_button.pack(pady=5, )

        self.change_label_name_button = ctk.CTkButton(self, text="Change
label name", command=self.change_label_name)
        self.change_label_name_button.pack(pady=5, )

        self.regulation_frame_button = ctk.CTkButton(self, text="Service's
Regulations", command=self.regulation)
        self.regulation_frame_button.pack(pady=5, )

        self.back_button = ctk.CTkButton(self, text="Back",
command=self.implement_changes)
        self.back_button.pack(pady=5,)
    def regulation(self):
        self.controller.show_frame(PolicyFrame)
    def change_label_name(self):
        labels_name=", ".join(self.controller.labels_names)
        label_name = ctk.CTkInputDialog(title="Change label name",
text=f"To change a label name please follow the
format;\nlabel_name-new_label_name\nLabels Names; {labels_name} ")
        label_name = label_name.get_input()
        if not label_name:
            messagebox.showwarning("Warning","You didn't followed the
format")
        else:

```

```

        names_list = label_name.split('-')
        if names_list[0] not in self.controller.labels_names:
            messagebox.showwarning("Warning", "The label's name you want
to change is incorrect or doesn't exists")
            elif len(names_list) > 2 or len(names_list[1]) >= 20:
                messagebox.showwarning("Warning", "You didn't followed the
format, or the label name must be 20 characters or less")
            elif not all(char not in names_list[1] for char in
self.controller.forbidden_characters):
                messagebox.showwarning("Input Error", "Forbidden
characters.")
            elif names_list[1] in self.controller.labels_names:
                messagebox.showwarning("Input Error", "There is already
label with this name.")
            else:
                index = self.controller.labels_names.index(names_list[0])
# get the index of the name to chnage
                self.controller.labels_names[index] = names_list[1] #
change the name
                self.controller.frames[MainFrame].update_label_menu()
    def implement_changes(self):
        '''
        Apply the changes and navigate back to the MainFrame.
        :return:
        '''
        if self.color_var!=self.controller.current_color:
            self.change_button_color()
        self.controller.frames[MainFrame].refresh_messages() # refresh the
message in case user got blocked, or is not blocked any more
        self.controller.show_frame(MainFrame)
    def add_to_dist_list(self):
        dist_list_num =
self.dist_menu_possibilities.index(self.dist_var.get())+1
        if dist_list_num==1:
            dist_list=self.controller.distribution_list1_names
        elif dist_list_num==2:
            dist_list=self.controller.distribution_list2_names
        else:
            dist_list=self.controller.distribution_list3_names
        user_name = ctk.CTkInputDialog(title="Add user to the selected
distribution list", text="Please enter the user's name")
        user_name=user_name.get_input()
        if not user_name:
            messagebox.showwarning("Input Error", "you didn't enter any
input")
        elif (user_name + "@DaVinci.cl")==self.controller.email:
            messagebox.showwarning("Input Error", "You can't add yourself")
        elif (user_name+"@DaVinci.cl")==self.controller.service_official_email:
            messagebox.showwarning("Input Error", "You can't add the
service official mail!")
        elif (user_name+"@DaVinci.cl") in dist_list:
            messagebox.showwarning("Input Error", "User already added.")

```

```

        elif not all(char not in user_name for char in
self.controller.forbidden_characters) :
            messagebox.showwarning("Input Error", "Forbidden characters.")
        else:
            dist_list.append(user_name + "@DaVinci.cl")
            messagebox.showinfo("Success", "The user was added.")
    def remove_added_user(self):
        dist_list_num =
self.dist_menu_possibilities.index(self.dist_var.get())+1
        if dist_list_num == 1:
            dist_list = self.controller.distribution_list1_names
        elif dist_list_num == 2:
            dist_list = self.controller.distribution_list2_names
        else:
            dist_list = self.controller.distribution_list3_names
        added_user_string = "\n".join(dist_list)
        user_name = ctk.CTkInputDialog(title="Remove added user",
text=f"Added users:\n {added_user_string}\n\nPlease enter the user's
name")
        user_name=user_name.get_input()
        if not user_name:
            messagebox.showwarning("Input Error", "you didn't enter any
input")
        else:
            try:
                dist_list.remove(user_name+ "@DaVinci.cl")
            except:
                messagebox.showwarning("Input Error", "These user doesn't
added or doesn't exist.")
    def add_blocked_user(self):
        user_name = ctk.CTkInputDialog(title="Add blocked user",
text="Please enter the user's name")
        user_name=user_name.get_input()
        if not user_name:
            messagebox.showwarning("Input Error", "you didn't enter any
input")
        elif (user_name + "@DaVinci.cl")==self.controller.email:
            messagebox.showwarning("Input Error", "You can't block
yourself")
        elif (user_name+"@DaVinci.cl")== "Service_Official@DaVinci.cl":
            messagebox.showwarning("Input Error", "You can't block the
service official mail!")
        elif (user_name+"@DaVinci.cl") in
self.controller.blocked_users_names:
            messagebox.showwarning("Input Error", "User already blocked.")
        elif not all(char not in user_name for char in
self.controller.forbidden_characters) :
            messagebox.showwarning("Input Error", "Forbidden characters.")
        else:
            self.controller.blocked_users_names.append(user_name +
"@DaVinci.cl")
            self.controller.new_blocked = True
    def remove_blocked_user(self):

```

```

        blocked_user_string =
"\n".join(self.controller.blocked_users_names)
        user_name = ctk.CTkInputDialog(title="Remove blocked user",
text=f"Blocked users:\n {blocked_user_string}\n\nPlease enter the user's
name")
        user_name=user_name.get_input()
        if not user_name:
            messagebox.showwarning("Input Error", "you didn't enter any
input")
        else:
            try:
                self.controller.blocked_users_names.remove(user_name+
"@DaVinci.cl")
            except:
                messagebox.showwarning("Input Error", "These user doesn't
blocked or doesn't exist.")
        def rgb_to_hex(self,rgb):
            '''
            Convert an RGB color value to a hex color string, which
costumtkinter accepts
            :param rgb:
            :return:
            '''
            return '#%02x%02x%02x' % rgb
        def change_button_color(self):
            '''
            Change the color of all the buttons to the selected color.
            :return:
            '''
            color =
self.controller.app_colors_names.index(self.color_var.get())
            color = self.rgb_to_hex(self.controller.app_colors_RGB[color])
            self.controller.current_color_hex=color
            if color!="default":
                all_widgets = self.controller.root.wininfo_children()
                for widget in all_widgets:
                    if isinstance(widget, ctk.CTkButton) or isinstance(widget,
ctk.CTkOptionMenu):
                        widget.configure(fg_color=color)
                all_widgets.extend(widget.wininfo_children())
class PolicyFrame(ctk.CTkFrame):
    def __init__(self, parent, controller):
        '''
        init the PolicyFrame class,
        :param parent:
        :param controller:
        '''
        ctk.CTkFrame.__init__(self, parent)
        self.controller = controller
        self.title="Service Policy"
        self.policy_text = ""
        1. Use Respectful Language:

```

```

        • Always communicate politely when contacting other users. Avoid
        using offensive or inappropriate words in any communication.
    2. Avoid Sending Harmful Content:
        • Never send files containing malicious software, viruses, or
        harmful links. Ensure all shared content is safe, legal, and appropriate.
    6. Respect Privacy and Security:
        • Avoid sharing sensitive personal or financial information
        unless absolutely necessary.
    7. Do Not Abuse the Service:
        • Refrain from using the mail service for unlawful activities,
        harassment, or spamming.
    """
    label = ctk.CTkLabel(self, text=self.title, font=("Calibri",
30,"bold"))
    label.pack(pady=10)
    label = ctk.CTkLabel(self, text=self.policy_text, font=("Calibri",
15))
    label.pack(pady=10)
    label = ctk.CTkLabel(self, text="Violating these rules will result
a ban from the service!", font=("Calibri", 18,"bold"))
    label.pack(pady=10)
    self.back_button = ctk.CTkButton(self, text="Back",
command=self.go_back)
    self.back_button.pack(pady=5,)
    def go_back(self):
        """
        navigate back to the MainFrame.
        :return:
        """
        self.controller.show_frame(SettingsFrame)
class MainFrame(ctk.CTkFrame):
    def __init__(self, parent, controller):
        """
        init the MainFrame class
        :param parent:
        :param controller:
        """
        ctk.CTkFrame.__init__(self, parent)
        self.controller = controller
        self.current_mail_title=""
        self.current_mail_timestamp=""
        if self.controller.screen_size=="small":
            button_height,box_height,menu_height=20,400,20
        else:
            button_height,box_height,menu_height=30,640,40

        self.welcome_label = ctk.CTkLabel(self, text=self.controller.email,
font=("Arial", 20))
        self.welcome_label.grid(row=0, column=1, pady=10,
padx=10,sticky="e")

        self.display_options = [

```



```

        "all", "specific", "important", "saved for later", "by
date", "deleted", "search"
    ]# "all" - all the mails, "specific" - showing only the mails from
specific sender

    self.display_option_var =
ctk.StringVar(value=self.display_options[0])
    self.display_option_menu = ctk.CTkOptionMenu(self,
variable=self.display_option_var, values=self.display_options,
height=menu_height)
    self.display_option_menu.grid(row=0, column=0, pady=10, padx=10)

    self.action_option=[
        "refresh", "sign as important", "save for
later", "forward", "delete", "label as selected label"
    ]

    self.mails_titles_listbox = ctk.CTkScrollableFrame(self, width=300,
height=box_height)
    self.mails_titles_listbox.grid(row=1, column=0, pady=5, padx=5)

    self.mail_content_box = ctk.CTkTextbox(self, width=550,
height=box_height)
    self.mail_content_box.grid(row=1, column=1, pady=5, padx=5)

    self.actions_options_var =
ctk.StringVar(value=self.action_option[0])
    self.actions_options_menu = ctk.CTkOptionMenu(self,
variable=self.actions_options_var,
values=self.action_option, height=menu_height,width=120)
    self.actions_options_menu.grid(row=2, column=0, pady=10,
padx=(10,5), sticky="w")

    self.adding_to_labels_var =
ctk.StringVar(value=self.controller.labels_names[0])
    self.adding_to_labels_menu = ctk.CTkOptionMenu(self,
variable=self.adding_to_labels_var,
values=self.controller.labels_names, height=menu_height, width=120)
    self.adding_to_labels_menu.grid(row=3, column=0, pady=10, padx=(10,
5), sticky="w")

    commit_button = ctk.CTkButton(self, text="commit",
command=self.commit, width=60,height=button_height+10)
    commit_button.grid(row=2, column=0, pady=10, padx=30)

    settings_button = ctk.CTkButton(self, text="settings",
command=lambda: controller.show_frame(SettingsFrame),
width=100,height=button_height)
    settings_button.grid(row=0, column=1, pady=10, padx=10)

```

```

        compose_button = ctk.CTkButton(self,
text="Compose",width=30,height=button_height,
command=self.open_compose_window)
        compose_button.grid(row=0, column=0, pady=10, padx=(10, 5),
sticky="w")

        logout_button = ctk.CTkButton(self, text="Logout",
command=self.logout,width=50,height=button_height)
        logout_button.grid(row=2, column=1, pady=5, padx=5,sticky="e")
        def update_label_menu(self):

self.adding_to_labels_menu.configure(values=self.controller.labels_names)
        self.adding_to_labels_var.set(self.controller.labels_names[0])

self.display_option_menu.configure(values=self.display_options+self.controller.labels_names)
        def delete_message(self):
            if self.display_option_var.get()=="all":
                try:
                    self.controller.cursor.execute('UPDATE messages SET
deleted=1 WHERE title=? AND timestamp=?',
                                                    (self.current_mail_title,
self.current_mail_timestamp,))
                    self.controller.conn.commit()
                    for widget in self.mails_titles_listbox.winfo_children():
                        if self.current_mail_title in widget.cget("text") and
self.current_mail_timestamp in widget.cget(
                            "text"):
                            widget.destroy()
                except sqlite3.OperationalError as e:
                    self.controller.db_error = True
                    self.controller.init_db() # creates a new table
                    messagebox.showerror("Data Base Error",'There is error in
the app data base.\n Please log out and login again, we are sorry!')
                    elif self.display_option_var.get()=="important":
                        try:
                            self.controller.cursor.execute('UPDATE messages SET
important=0 WHERE title=? AND timestamp=?',
                                                            (self.current_mail_title,
self.current_mail_timestamp,))
                            self.controller.conn.commit()
                            self.display_option_var.set("important")
                            self.refresh_messages()
                        except sqlite3.OperationalError as e:
                            self.controller.db_error = True
                            self.controller.init_db() # creates a new table
                            messagebox.showerror("Data Base Error",'There is error in
the app data base.\n Please log out and login again, we are sorry!')
                            elif self.display_option_var.get()=="saved for later":
                                try:
                                    self.controller.cursor.execute('UPDATE messages SET
readlater=0 WHERE title=? AND timestamp=?',

```

```

                                                                    (self.current_mail_title,
self.current_mail_timestamp,))
        self.controller.conn.commit()
        self.display_option_var.set("saved for later")
        self.refresh_messages()
    except sqlite3.OperationalError as e:
        self.controller.db_error = True
        messagebox.showerror()
        self.controller.init_db() # creates a new table
        messagebox.showerror("Data Base Error", 'There is error in
the app data base.\n Please log out and login again, we are sorry!')
    elif self.adding_to_labels_var.get() in
self.controller.labels_names: # the user see all the messages in label
        self.remove_from_label()
        self.display_option_var.set(self.adding_to_labels_var.get())
        self.refresh_messages()
    def undelete_message(self):
        try:
            self.controller.cursor.execute('UPDATE messages SET deleted=0
WHERE title=? AND timestamp=?',
                                                                    (self.current_mail_title,
self.current_mail_timestamp,))
            self.controller.conn.commit()
            for widget in self.mails_titles_listbox.wininfo_children():
                if self.current_mail_title in widget.cget("text") and
self.current_mail_timestamp in widget.cget(
                    "text"):
                    widget.destroy()
        except sqlite3.OperationalError as e:
            self.controller.db_error = True
            self.controller.init_db() # creates a new table
            messagebox.showerror("Data Base Error",
                                'There is error in the app data base.\n
Please log out and login again, we are sorry!')
    def commit(self):
        if self.actions_options_var.get()=="refresh":
            self.refresh_messages()
        else:
            if self.current_mail_title != "":
                if self.actions_options_var.get() == "sign as important":
                    self.sign_importance()
                    self.actions_options_menu.set("refresh")
                elif self.actions_options_var.get() == "save for later":
                    self.save_for_later()
                    self.actions_options_menu.set("refresh")
                elif self.actions_options_var.get() == "delete":
                    self.delete_message()
                    self.actions_options_menu.set("refresh")
                elif self.actions_options_var.get() == "forward":
                    self.forward()
                    self.actions_options_menu.set("refresh")
                elif self.actions_options_var.get() == "undelete":
                    self.undelete_message()

```

```

        elif self.actions_options_var.get() == "label as selected
label":
            self.add_to_label()
            self.actions_options_menu.set("refresh")
        else:
            messagebox.showwarning("Warning", "You didn't clicked on
any mail!")
    def add_to_label(self):
        selected_label=self.adding_to_labels_var.get()
        index=self.controller.labels_names.index(selected_label)
        try:
            self.controller.cursor.execute('SELECT labels FROM messages
WHERE id=?',
                                           (self.current_mail_id,))

labels_sign_as_list=list((self.controller.cursor.fetchone()[0]))
            labels_sign_as_list[index]="Y"
            labels_sign="".join(labels_sign_as_list)
            self.controller.cursor.execute('UPDATE messages SET labels=?
WHERE id=?',
                                           (labels_sign,self.current_mail_id,))
            self.controller.conn.commit()
        except sqlite3.OperationalError as e:
            self.controller.db_error = True
            self.controller.init_db() # creates a new table
            messagebox.showerror("Data Base Error",
                                'There is error in the app data base.\n
Please log out and login again, we are sorry!')
    def remove_from_label(self):
        selected_label=self.adding_to_labels_var.get()
        index=self.controller.labels_names.index(selected_label)
        try:
            # udapde the string that represents in each label mail is
labeld
            self.controller.cursor.execute('SELECT labels FROM messages
WHERE id=?',
                                           (self.current_mail_id,))

labels_sign_as_list=list((self.controller.cursor.fetchone()[0]))
            labels_sign_as_list[index]="N"
            labels_sign="".join(labels_sign_as_list)
            self.controller.cursor.execute('UPDATE messages SET labels=?
WHERE id=?',
                                           (labels_sign,self.current_mail_id,))
            self.controller.conn.commit()
        except sqlite3.OperationalError as e:
            self.controller.db_error = True
            self.controller.init_db() # creates a new table
            messagebox.showerror("Data Base Error",
                                'There is error in the app data base.\n
Please log out and login again, we are sorry!')

```

```

def forward(self):
    try:
        self.controller.cursor.execute('SELECT title, message FROM
messages WHERE id=?',
                                        (self.current_mail_id,))
        message = self.controller.cursor.fetchall()
        specific_user = ctk.CTkInputDialog(title="Forward",
                                           text="Please enter the
recipient's address you'd like to forward this mail")
        if specific_user and message:
            if not specific_user.get_input() is None and
specific_user.get_input() != "" and "@DaVinci.cl" in
specific_user.get_input():
                message_data = {
                    'sender': self.controller.email,
                    'recipient': specific_user.get_input(),
                    'title': message[0][0],
                    'message': message[0][1]
                }
                response = self.controller.send_message(message_data)
                if response['status'] == 'success':
                    messagebox.showinfo("Success", "Message sent
successfully.")
                else:
                    messagebox.showerror("Error", response['message'])
            else:
                messagebox.showerror("Error", "The recipient mail isn't
in the service format or you didn't enter any address")
        else:
            messagebox.showwarning("Sudden Error", "Please try again.")
    except sqlite3.OperationalError as e:
        self.controller.db_error = True
        self.controller.init_db() # creates a new table
        messagebox.showerror("Data Base Error",
                              'There is error in the app data base.\n
Please log out and login again, we are sorry!')
    def sign_importance(self):
        try:
            self.controller.cursor.execute('UPDATE messages SET important=1
WHERE id=?',
                                            (self.current_mail_id,))
            self.controller.conn.commit()
        except sqlite3.OperationalError:
            self.controller.db_error = True
            self.controller.init_db() # creates a new table
            messagebox.showerror("Data Base Error",
                                  'There is error in the app data base.\n
Please log out and login again, we are sorry!')
    def save_for_later(self):
        try:
            self.controller.cursor.execute('UPDATE messages SET readlater=1
WHERE id=?',
                                            (self.current_mail_id,))

```

```

        self.controller.conn.commit()
    except sqlite3.OperationalError as e:
        self.controller.db_error = True
        self.controller.init_db() # creates a new table
        messagebox.showerror("Data Base Error",
                              'There is error in the app data base.\n
Please log out and login again, we are sorry!')
    def show_mail_content(self, title, timestamp,id):
        '''
        When a title button is pressed the sender, sending time and the
        content will be shown on the txt message
        :return:
        '''
        try:
            self.current_mail_title,
self.current_mail_timestamp,self.current_mail_id = title, timestamp,id
            self.mail_content_box.delete("1.0", ctk.END) # clears the
previous text
            self.controller.cursor.execute(
                'SELECT sender, message,file_data,image_data FROM messages
WHERE id=?',
                (id,))
            data = self.controller.cursor.fetchone()
            sender, content, file_data, image_data = data[0], data[1],
data[2], data[3]
            timestamp = timestamp.replace(':', "-")
            if len(file_data) > 15:
                # saving the txt data as file in the downloads folder
                downloads_folder = os.path.join(os.path.expanduser("~"),
                                                "Downloads") # Get the
path to the Downloads folder
                file_path = os.path.join(downloads_folder,
f"{{(sender.split('@')[0])}},{{timestamp}}.txt") # Create the file path
                with open(file_path, "w") as file: # Save the string to
the file
                    file.write(file_data)
                file_data = "A txt file was attached, see in the downloads
folder"
            else:
                file_data = "No txt attached"
            if len(image_data) > 15:
                print(image_data)
                image_data = base64.b64decode(image_data)
                # saving the image/docx or any other file's data as file in
the downloads folder
                downloads_folder = os.path.join(os.path.expanduser("~"),
                                                "Downloads") # Get the
path to the Downloads folder
                file_path = os.path.join(downloads_folder,
f"{{(sender.split('@')[0])}},{{timestamp}}.PNG") # Create the file path

```

```

        with open(file_path, "wb") as file: # Save the string to
the file
            file.write(image_data)
            image_data = "A none txt file was attached, see in the
downloads folder"
        else:
            image_data = "No none txt file attached"
            if sender != self.controller.email: # if the sender is not the
user himself
                self.mail_content_box.insert(ctk.END,
                                                f"Sender: {sender}\nSending
Time: {timestamp}\n{file_data}\n{image_data}\nMail content: {content}")
            else:
                self.mail_content_box.insert(ctk.END,
                                                f"Sender: {sender}
(you)\nSending Time: {timestamp}\n{file_data}\n{image_data}\nMail content:
{content}")
        except sqlite3.OperationalError:
            self.controller.db_error = True
            self.controller.init_db() # creates a new table
            messagebox.showerror("Data Base Error",
                                'There is error in the app data base.\n
Please log out and login again, we are sorry!')
    def update_email(self):
        '''
        updates the email from 'None'
        :return:
        '''
        self.welcome_label.configure(text=self.controller.email)
    def refresh_messages(self):
        '''
        Retrieve and display messages from the server for the user by the
different categories.
        :return:
        '''
        if self.display_option_var.get()=="all":
            try:
                response = self.controller.get_messages()
                print(response)
                if response['status'] == 'success':
                    messages = response.get('messages', [])
                    if len(messages) != 0 or self.controller.new_blocked: #
if we blocked we need to reset the message
                        self.controller.new_blocked = False
                        for msg in messages:
                            sender = msg['sender']
                            title = msg['title']
                            content = msg['message']
                            timestamp = msg['timestamp']
                            important = msg['important']
                            readlater = msg['readlater']
                            deleted = msg['deleted']
                            file_data = msg['file_data']

```

```

        image_data = msg['image_data']
        labels = msg['labels']
        print(image_data)
        print(labels)
        self.controller.cursor.execute(
            'INSERT INTO messages (sender,
title,message,
timestamp,important,readlater,deleted,file_data,image_data,labels) VALUES
(?, ?, ?, ?,?,?,?,?,,?)',
            (sender, title, content, timestamp,
important, readlater, deleted, file_data, image_data,
            labels,))
        self.controller.conn.commit()
        for widget in
self.mails_titles_listbox.wininfo_children():
            widget.destroy() # Clear existing items
        msg_lst = self.controller.fetch_all_messages() #
gets all the mails
        msg_lst = list(reversed(msg_lst)) # rerverse the
list so we add the new mails first
        print('msg_lst', msg_lst)
        for msg in msg_lst:
            item = ctk.CTkButton(self.mails_titles_listbox,
                                text=f"Title;
{msg['title']} ,{msg['timestamp']}",
                                command=lambda m=msg:
self.show_mail_content(m['title'], m['timestamp'],
m['id']), width=290)
            item.pack(pady=5, padx=10)
        elif len(self.controller.fetch_all_messages()) > len(
            self.mails_titles_listbox.wininfo_children()) or
self.display_option_var == "deleted": # if there are more message on
general than the ones whom being display, we were on specific and should
get all the msseages
            for widget in
self.mails_titles_listbox.wininfo_children():
                widget.destroy() # Clear existing items
            msg_lst = self.controller.fetch_all_messages() #
gets all the mails
            msg_lst = list(reversed(msg_lst)) # rerverse the
list so we add the new mails first
            for msg in msg_lst:
                item = ctk.CTkButton(self.mails_titles_listbox,
                                    text=f"Title;
{msg['title']} ,{msg['timestamp']}",
                                    command=lambda m=msg:
self.show_mail_content(m['title'], m['timestamp'], m['id']), width=290)
                item.pack(pady=5, padx=10)
            elif len(messages) == 0:
                messagebox.showinfo("Attention", 'There is not new
mails!')
        else:

```



```

        messagebox.showerror("Error", response['message'])
    except sqlite3.OperationalError as e:
        self.controller.db_error = True
        self.controller.init_db() # creates a new table
        messagebox.showerror("Data Base Error", 'There is error in
the app data base.\n Please log out and login again, we are sorry!')
    elif self.display_option_var.get()=="specific":
        specific_user=ctk.CTkInputDialog(title="Specific
user",text="Please enter the specific's user mail")
        if specific_user:
            for widget in self.mails_titles_listbox.winfo_children():
                widget.destroy() # Clear existing items
            msg_lst =
self.controller.fetch_messages_by_category("specific",specific_user.get_in
put()) # gets all the mails of the speicific user
            if len(msg_lst)!=0:
                msg_lst = list(reversed(msg_lst)) # rerverse the list
so we add the new mails first
                for msg in msg_lst:
                    item = ctk.CTkButton(self.mails_titles_listbox,
                                         text=f"Title; {msg['title']}
,{msg['timestamp']}", command=lambda m=msg:
self.show_mail_content(m['title'],
m['timestamp'],m['id']), width=290)
                    item.pack(pady=5, padx=10)
            else:
                messagebox.showerror("Attention", 'There is not mails
from this specific sender!')
        elif self.display_option_var.get()=="important":
            for widget in self.mails_titles_listbox.winfo_children():
                widget.destroy() # Clear existing items
            msg_lst =
self.controller.fetch_messages_by_category("important") # gets all the
mails of the speicific user
            if len(msg_lst) != 0:
                msg_lst = list(reversed(msg_lst)) # rerverse the list so
we add the new mails first
                for msg in msg_lst:
                    item = ctk.CTkButton(self.mails_titles_listbox,
                                         text=f"Title; {msg['title']}
,{msg['timestamp']}",
                                         command=lambda m=msg:
self.show_mail_content(m['title'], m['timestamp'],m['id']),width=290)
                    item.pack(pady=5, padx=10)
            else:
                messagebox.showerror("Attention", 'There is not important
mails yet!')
        elif self.display_option_var.get()=="saved for later":
            for widget in self.mails_titles_listbox.winfo_children():
                widget.destroy() # Clear existing items
            msg_lst = self.controller.fetch_messages_by_category("saved for
later") # gets all the mails of the speicific user

```

```

        if len(msg_lst) != 0:
            msg_lst = list(reversed(msg_lst)) # rerverse the list so
we add the new mails first
            for msg in msg_lst:
                item = ctk.CTkButton(self.mails_titles_listbox,
                                    text=f"Title; {msg['title']}
,{msg['timestamp']}", command=lambda m=msg:
self.show_mail_content(m['title'], m['timestamp'],m['id']),width=290)
                item.pack(pady=5, padx=10)
            else:
                messagebox.showerror("Attention", 'There is not saved for
later mails yet!')
            elif self.display_option_var.get()=="deleted":
                for widget in self.mails_titles_listbox.winfo_children():
                    widget.destroy() # Clear existing items
                self.actions_options_var.set("undelete")
                msg_lst = self.controller.fetch_messages_by_category("deleted")
# gets all the mails of the speicific user
                if len(msg_lst) != 0:
                    msg_lst = list(reversed(msg_lst)) # rerverse the list so
we add the new mails first
                    for msg in msg_lst:
                        item = ctk.CTkButton(self.mails_titles_listbox,
                                            text=f"Title; {msg['title']}
,{msg['timestamp']}", command=lambda m=msg:
self.show_mail_content(m['title'], m['timestamp'],m['id']), width=290)
                        item.pack(pady=5, padx=10)
                    else:
                        messagebox.showerror("Attention", 'There is not deleted
mails yet!')
                    elif self.display_option_var.get()=="search":
                        specific_date=ctk.CTkInputDialog(title="Specific
text",text="Please enter the specific text for searching")
                        if specific_date:
                            for widget in self.mails_titles_listbox.winfo_children():
                                widget.destroy() # Clear existing items
                            msg_lst =
self.controller.fetch_messages_by_category("search",specific_date.get_inpu
t()) # gets all the mails of the speicific user
                            if len(msg_lst)!=0:
                                msg_lst = list(reversed(msg_lst)) # rerverse the list
so we add the new mails first
                                for msg in msg_lst:
                                    item = ctk.CTkButton(self.mails_titles_listbox,
                                                        text=f"Title; {msg['title']}
,{msg['timestamp']}", command=lambda m=msg:
self.show_mail_content(m['title'], m['timestamp'],m['id']), width=290)
                                    item.pack(pady=5, padx=10)
                                else:
                                    messagebox.showerror("Attention", 'There is not mails
with this specific text!')
                            elif self.display_option_var.get()=="by date":

```

```

        specific_date=ctk.CTkInputDialog(title="Specific
date",text="Please enter the specific date for searching yyyy-mm-dd")
        if specific_date:
            for widget in self.mails_titles_listbox.winfo_children():
                widget.destroy() # Clear existing items
            msg_lst = self.controller.fetch_messages_by_category("by
date",specific_date.get_input()) # gets all the mails of the speicific
date

            if len(msg_lst)!=0:
                msg_lst = list(reversed(msg_lst)) # rerverse the list
so we add the new mails first
                for msg in msg_lst:
                    item = ctk.CTkButton(self.mails_titles_listbox,
                                         text=f"Title; {msg['title']}
,{msg['timestamp']}", command=lambda m=msg:
self.show_mail_content(m['title'], m['timestamp'],m['id']), width=290)
                    item.pack(pady=5, padx=10)
                else:
                    messagebox.showerror("Attention", 'There is not mails
in this specific date!')
            elif self.display_option_var.get() in self.controller.labels_names:
                for widget in self.mails_titles_listbox.winfo_children():
                    widget.destroy() # Clear existing items

index=self.controller.labels_names.index(self.display_option_var.get())
msg_lst =
self.controller.fetch_messages_by_category("label",index) # gets all the
mails of the speicific user
            if len(msg_lst) != 0:
                msg_lst = list(reversed(msg_lst)) # rerverse the list so
we add the new mails first
                for msg in msg_lst:
                    item = ctk.CTkButton(self.mails_titles_listbox,
                                         text=f"Title; {msg['title']}
,{msg['timestamp']}",
                                         command=lambda m=msg:
self.show_mail_content(m['title'], m['timestamp'],m['id']),width=290)
                    item.pack(pady=5, padx=10)
                else:
                    messagebox.showerror("Attention", 'You did not labeled
message to this label!')
            def open_compose_window(self):
                '''
                open a new window to in order to send a new mail
                :return:
                '''
                compose_window = ComposeWindow(self.controller)
                compose_window.grab_set()
            def logout(self):
                '''
                Log out the current user and show the login frame & deletes all the
last user data.
                :return:

```

```

'''
    result = messagebox.askyesno("Logout Confirmation", "Are you sure
you want to logout?")
    if result:
        self.mail_content_box.delete("1.0", ctk.END) # clears the
previous text
        for widget in self.mails_titles_listbox.winfo_children():
            widget.destroy() # Clear existing items
        self.actions_options_var.set("refresh")
        self.display_option_var.set("all") # sets the display options
to "all"
        self.controller.logout()
class ComposeWindow(ctk.CTkToplevel):
    def __init__(self, controller):
        '''
        init the ComposeWindow class
        :param controller:
        '''
        ctk.CTkToplevel.__init__(self)
        self.controller = controller
        self.message_limit=3000 #any message should be equal or under 3000
letters
        self.title("Compose Message")
        self.geometry('400x490')
        self.attachment_path=None
        self.file_data="WASNOT_ATTACHED"
        self.image_data="WASNOT_ATTACHED"
        self.image_data_binary= b""

        distribution_label = ctk.CTkLabel(self, text="If you wish to send
to distribution list,\n write; distribution_num, and enter as num 1,2 or
3")
        distribution_label.pack(pady=5)

        recipient_label = ctk.CTkLabel(self, text="To:")
        recipient_label.pack(pady=5)

        self.recipient_entry = ctk.CTkEntry(self, width=350)
        self.recipient_entry.pack(pady=5)

        message_label = ctk.CTkLabel(self, text="Message:")
        message_label.pack(pady=5)

        self.title_entry = ctk.CTkEntry(self,
width=150,placeholder_text="Title")
        self.title_entry.pack(pady=5)

        self.message_textbox = ctk.CTkTextbox(self, width=350, height=150)
        self.message_textbox.pack(pady=5)

        attach_button = ctk.CTkButton(self, text="Attach file - only txt
file or PNG images", command=self.attach_file)
        attach_button.pack(pady=5)

```

```

        send_button = ctk.CTkButton(self, text="Send",
command=self.send_message)
        send_button.pack(pady=5)
    def send_message(self):
        '''
        Send the composed message to the server.
        :return:
        '''
        title=self.title_entry.get()
        recipient = self.recipient_entry.get()
        message = self.message_textbox.get("1.0", ctk.END).strip()
        if len(message) > self.message_limit:
            messagebox.showwarning("Input Error",
                                f"Message is too long. Please limit to
{self.message_limit} characters,\nyou sre now on {len(message)}
characters.")
            elif ServiceMainClass.check_content(message)==True or
ServiceMainClass.check_content(title)==True:
                self.controller.violations+=1
                messagebox.showwarning("Warning","There is profanity in your
mail's title/content, if you continue in this way you will get ban from
the service!\nA user which violates the service's rules over 5 times will
get banned!\nIn addition, the user will have pay 100$ fine to get is
account back.")
                if self.controller.violations==5:
                    messagebox.showerror("Error","You have violated the srvice
rules 5 times, you got baned!")
                    self.controller.close_while_running()
                    self.destroy()
            else:
                if recipient and message:
                    if len(self.file_data)>15:
                        self.file_data=self.file_data[15:]
                    if recipient in ['distribution_1', 'distribution_2',
'distribution_3'] and message:
                        if recipient == 'distribution_1':
                            dist_list =
self.controller.distribution_list1_names
                        elif recipient == 'distribution_2':
                            dist_list =
self.controller.distribution_list2_names
                        else:
                            dist_list =
self.controller.distribution_list3_names
                        if len(dist_list)!=1:
                            if len(dist_list) != 1: # if there is only one
user
                                for i in range(len(dist_list) - 1): # goes all
over the dist list except the last one
                                    if dist_list[i]!="":
                                        message_data = {
                                            'sender': self.controller.email,

```

```

        'recipient': dist_list[i],
        'title': title,
        'message': message,
        'file_data': self.file_data,
        'image_data': self.image_data,
    }

self.controller.send_message(message_data)

    message_data = {
        'sender': self.controller.email,
        'recipient': dist_list[-1],
        'title': title,
        'message': message,
        'file_data': self.file_data,
        'image_data': self.image_data,
    }
    response = self.controller.send_message(
        message_data) # now we get the response from
the server to know if was sent succeseckfly
    if response['status'] == 'success':
        messagebox.showinfo("Success", "Message sent
successfully.")

        self.destroy()
    else:
        messagebox.showerror("Error",
response['message'])
    else:
        messagebox.showinfo("Please notice", "The selected
distribution list is empty.")
    else:
        if "DaVinci.cl" in recipient:
            message_data = {
                'sender': self.controller.email,
                'recipient': recipient,
                'title': title,
                'message': message,
                'file_data': self.file_data,
                'image_data': self.image_data,
            }
            response =
self.controller.send_message(message_data)
            if response['status'] == 'success':
                messagebox.showinfo("Success", "Message sent
successfully.")

                self.destroy()
            else:
                messagebox.showerror("Error",
response['message'])
        else:
            messagebox.showerror("Mail Input Error", "The
address you entered in not in the service format - name@DaVinci.cl")
    else:

```

```

        messagebox.showwarning("Input Error", "Please enter a
recipient and message.")
    def attach_file(self):
        '''
        If the client wishes to attach
        :return:
        '''
        if len(self.file_data)>15 or len(self.image_data)>15:
            messagebox.showwarning("Notice","You have already attached txt
or image file, adding another one instead will erase the previous one (in
it's section - txt or image)!")
            filename = filedialog.askopenfilename()
            self.attachment_path = filename
            if self.attachment_path and (any(ext in self.attachment_path for
ext in ["jpg", "png", "jpeg", "image", "Image","docx"])): # images
                self.image_data = "WASNOT_ATTACHED"
                with open(self.attachment_path, 'rb') as f:
                    self.image_data_binary=b""
                    while True:
                        data = f.read(1024)
                        if not data:
                            break
                        self.image_data_binary += data
                    if (len(self.file_data)+len(self.image_data_binary))>25600:
#24kb
                        self.image_data_binary=b''
                        messagebox.showwarning("Size","The size of this file is
too big,\n or the size all the files you have attached are too big")
                    else:
                        self.image_data =
base64.b64encode(self.image_data_binary).decode('utf-8')
            elif self.attachment_path:
                self.file_data = "WASNOT_ATTACHED"
                with open(self.attachment_path, 'r') as f: # txt files
                    while True:
                        data = f.read(1024)
                        if not data:
                            break
                        self.file_data += data
                    if (len(self.file_data)+len(self.image_data_binary))>25600:
#kb
                        messagebox.showwarning("Size","The size of this file is
too big,\n or the size all the files you have attached are too big")
                        self.file_data=""
                    else:
                        messagebox.showwarning("Attention", "You didn't choose a
file!")

if __name__ == "__main__":
    ServiceMainClass()
    MailClient()

```

```
import socket
import struct
import threading
import sqlite3
import json
import gzip
import sys
import base64
from datetime import datetime
from EncryptionLib import HybridEncryption
from ServiceMainClass import ServiceMainClass

class MailServer(ServiceMainClass):
    def __init__(self):
        """
        init the MailServer class with the given host and port from the
        ServiceMainClass.
        This class implements the service's server with all of its
        capabilities, functions and data base usage.
        """
        super().__init__()
        self.encryption_ins = HybridEncryption()
        self.private_key, self.public_key =
self.encryption_ins.read_file("private_key.txt"),
self.encryption_ins.read_file("public_key.txt") # stores the private and
public key in two vars
        self.server_socket=None
        self.max_connections = 20 # max amount of clients which can
connect to this server
        self.db_lock=threading.Lock()
        self.init_db() # init the server db, or creating it if needed
        self.start_server() # class the "start_server" func which as it's
name starts the server and watis for connection of clients.
    def init_db(self):
        """
        Initialize the server's SQLite database and create required tables
        if they don't exist.
        :return:
        """
        try:
            self.conn = sqlite3.connect('mail_server.db',
check_same_thread=False) # the conn can be used in various threads
            self.cursor = self.conn.cursor()
            self.cursor.execute('''CREATE TABLE IF NOT EXISTS users (
                                email TEXT PRIMARY KEY UNIQUE,
                                password TEXT,
                                security_q1 TEXT,
                                security_a1 TEXT,
                                security_q2 TEXT,
                                security_a2 TEXT,
                                blocked_users TEXT,
```



```

        color TEXT,
        violations INT DEFAULT 0,
        ban INT DEFAULT 0,
        birthday_date TEXT,
        distribution_list1 TEXT,
        distribution_list2 TEXT,
        distribution_list3 TEXT,
        labels_names TEXT DEFAULT
'label11/label12/label13/label14/label15'
    )'''
    self.cursor.execute('''CREATE TABLE IF NOT EXISTS messages (
AUTOINCREMENT,

        sender TEXT,
        recipient TEXT,
        title TEXT,
        message TEXT,
        timestamp DATETIME DEFAULT
CURRENT_TIMESTAMP,

        read INTEGER DEFAULT 0,
        important INT DEFAULT 0,
        readlater INT DEFAULT 0,
        deleted INT DEFAULT 0,
        image_data TEXT,
        file_data TEXT,
        labels TEXT DEFAULT 'NNNNN',
        FOREIGN KEY (sender) REFERENCES
users(email)

        FOREIGN KEY (recipient)
REFERENCES users(email)
    )''')

    self.conn.commit()
except:
    print("NOTICE - error in the server DB")
    sys.exit(1) # stop the serve from running
def start_server(self):
    '''
    Start the server, listen for incoming connections, and handle each
client connection in a new thread.
    :return:
    '''
    self.server_socket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
    self.server_socket.bind((self.host, self.port))
    self.server_socket.listen()
    print("[STARTED] Server is running and waiting for connections...")
    while True:
        if (threading.active_count() - 1) < self.max_connections:
            client_socket, addr = self.server_socket.accept()
            thread = threading.Thread(target=self.handle_client,
args=(client_socket, addr))
            thread.start()

```

```

        print(f"[ACTIVE CONNECTIONS] {threading.active_count() -
1}")

    else:
        print(f"[CONNECTION LIMIT REACHED] Maximum of
{self.max_connections} active connections. New connection attempt
rejected.")

        client_socket, addr = self.server_socket.accept() # Still
need to accept to close the connection
        try:
            # A "server busy" message to the client
            json_data = json.dumps({'accept': False}) # Convert
dict to JSON string
            compressed_data =
gzip.compress(json_data.encode('utf-8')) # Compress the JSON string
            length_bytes = struct.pack('>I', len(compressed_data))
# Pack length as a big-endian unsigned integer (4 bytes)
            client_socket.sendall(length_bytes)
            client_socket.sendall(compressed_data)
            client_socket.close()
            print(f"[DISCONNECTED] {addr} disconnected.")
        except (ConnectionResetError, BrokenPipeError,
socket.error):
            client_socket.close()
            print(f"[DISCONNECTED] {addr} disconnected.")

    def handle_client(self, client_socket, addr):
        """
        Handle communication with a connected client, first there are few
steps for data security - sending the public key for each new connected
client,
        then gets from the client its AES key and number of bits shifting
num (which is random for each client for more advanced protection).
        After this procedure there is endless lupe (until logout or closing
the app in the clients side) which have the repeated procedure;
        :param client_socket: The socket object representing the client
connection.
        :param addr: The address of the connected client.
        """
        connected=True
        try: # service 'Handshake'
            print(f"[NEW CONNECTION] {addr} connected.")
            # sending that server can accept new clients
            json_data = json.dumps({'accept': True}) # Convert dict to
JSON string
            compressed_data = gzip.compress(json_data.encode('utf-8')) #
Compress the JSON string
            length_bytes = struct.pack('>I', len(compressed_data)) # Pack
length as a big-endian unsigned integer (4 bytes)
            client_socket.sendall(length_bytes)
            client_socket.sendall(compressed_data)
            # sending the server's public key to the user
            json_data = json.dumps(self.public_key) # Convert the key to
JSON string

```

```

        compressed_data = gzip.compress(json_data.encode('utf-8')) #
Compress the JSON string
        length_bytes = struct.pack('>I', len(compressed_data)) # Pack
length as a big-endian unsigned integer (4 bytes)
        client_socket.sendall(length_bytes)
        client_socket.sendall(compressed_data)
        # receives the client's aes key and decrypt it to bytes
        length = client_socket.recv(4)
        length = struct.unpack('>I', length)[0]
        message = client_socket.recv(length)
        data = gzip.decompress(message) # Decompress and convert back
to string
        data = json.loads(data)
        encrypted_aes_key = data['encrypted_key']
        # decrypt the AES key
        encoded_aes_key =
self.encryption_ins.decrypt_message_RSA(encrypted_aes_key,
"private_key.txt")
        # decodes the AES key to bytes
        aes_key_bytes = base64.b64decode(encoded_aes_key)
        # decrypt the bits shifting num according to the server ip
        ip_parts_str = self.host.split('.') # splits the ip address of
the host for four
        ip_tuple_int = tuple(int(part) for part in ip_parts_str) #
convert them to int
        sum = 0
        for i in ip_tuple_int:
            sum += ((i * 7) + 13) * 2
        bytes_shifting_num = data['num'] - sum
    except (ConnectionResetError, BrokenPipeError, socket.error):
        connected=False
    if connected:
        while True:
            try:
                iv_bytes = client_socket.recv(16)
                length = client_socket.recv(4)
                length = struct.unpack('>I', length)[0]
                print(length)
                message = client_socket.recv(length)
                if message:
                    data = self.treat_message(message, aes_key_bytes,
iv_bytes, bytes_shifting_num)
                    if data[
                        'action'] == "close before logged in": # if
the client sent logout before it logged in, we can break the while loop,
the isn't any attached data, or any message the should be sent to it
                        break
                    response = self.process_request(data) # pass the
dict to get response
                    new_iv = self.encryption_ins.generate_aes_iv() #
generates enw iv for the new message
                    compressed_data = self.prepare_message(response,
aes_key_bytes, new_iv, bytes_shifting_num)

```

```

        client_socket.sendall(new_iv)
        # sends the data length in bytes
        length_bytes = struct.pack('>I',
                                    len(compressed_data)) #
Pack length as a big-endian unsigned integer (4 bytes)
        client_socket.sendall(length_bytes)
        client_socket.sendall(compressed_data)
        if data['action'] == "logout": # if the client
sent logout request, we can break the while loop
            break
    except:
        break
    client_socket.close() # closing the connection with the client's
socket
    print(f"[DISCONNECTED] {addr} disconnected.")
    # the thread ends
def process_request(self, data):
    """
    Process the incoming request and call the appropriate action.
    :param data: The data received from the client.
    :return: A dictionary containing the response to the client.
    """
    action = data['action']
    if action == 'register':
        return self.register_user(data)
    elif action == 'login':
        return self.login_user(data)
    elif action == 'send_message':
        return self.receive_message(data)
    elif action == 'retrieve_messages':
        return self.retrieve_messages(data)
    elif action == 'forgot_password':
        return self.forgot_password(data)
    elif action == 'reset_password':
        return self.reset_password(data)
    elif action == 'logout':
        return self.logout(data)
    elif action == "rsa public key":
        return {'status': 'success', 'public_key':
self.public_key.encode()}
    else:
        return {'status': 'error', 'message': 'Invalid action.'}
def logout(self, data):
    """
    sets message as unread, updates labels, importance and other
filters for the message in the db, updates the blocked users list,
the distribution list.
    :return if the logout was succesfull or not:
    """
    with self.db_lock:
        try:
            # Mark messages as unread

```

```

        self.cursor.execute('UPDATE messages SET read=0 WHERE
recipient=?', (data['mail'],))
        self.conn.commit()
        blocked_users_lst = data['blocked_users'] # updates the
blocked users
        result_string_blocked = "/".join(blocked_users_lst)
        blocked_users_lst = data['dist1'] # updates the dist1
users
        result_string_dist1 = "/".join(blocked_users_lst)
        blocked_users_lst = data['dist2'] # updates the dist2
users
        result_string_dist2 = "/".join(blocked_users_lst)
        blocked_users_lst = data['dist3'] # updates the dist3
users
        result_string_dist3 = "/".join(blocked_users_lst)
        labels_names_lst = data['labels_names'] # updates the
dist3 users
        result_string_labels_names = "/".join(labels_names_lst)
        self.cursor.execute(
            'UPDATE users SET blocked_users = ?,
distribution_list1=?, distribution_list2=?,
distribution_list3=?,labels_names=? WHERE email = ?',
            (result_string_blocked, result_string_dist1,
result_string_dist2, result_string_dist3,
            result_string_labels_names, data['mail'],))
        self.conn.commit()
        self.cursor.execute('UPDATE users SET color = ? WHERE email
= ?',
                                (data['color'], data['mail'],)) #
updates the last color
        self.conn.commit()
        messages = data['messages'] # updates the messages status
        for m in messages:
            self.cursor.execute(
                'UPDATE messages SET
important=?,readlater=?,deleted=?,labels=? WHERE title=? AND timestamp=?
AND sender=?',
                (m['important'], m['readlater'], m['deleted'],
m['labels'], m['title'], m['timestamp'],
                m['sender'],))
            self.conn.commit()
            # checking if should be banned
            self.cursor.execute(
                'SELECT violations FROM users WHERE email=?',
                (data['mail'],))
            violations = self.cursor.fetchall()
            if (violations[0][0] + data['violations']) > 5: # the user
should be banned
                tmp = 1
                self.cursor.execute('UPDATE users SET ban=? WHERE
email=?', (tmp, data['mail'],))
                self.conn.commit()
            else:

```

```

        self.cursor.execute('UPDATE users SET violations=?
WHERE email=?',
                                ((violations[0][0] +
data['violations']), data['mail'],))
        self.conn.commit()
        return {'status': 'success', 'message': "logout
successfully"}
    except:
        return {'status': 'error', 'message': "didn't logout
successfully due to server's data base error"}

    def forgot_password(self, data):
        """
        Handle the password recovery process by retrieving security
        questions for the given email.
        :param data: A dictionary containing the email for which to
        retrieve security questions.
        :return: A dictionary containing the status and message, and the
        security questions if found.
        """
        with self.db_lock:
            try:
                email = data['email']

                # Retrieve security questions from the database
                self.cursor.execute('SELECT security_q1, security_q2 FROM
users WHERE email=?', (email,))
                user = self.cursor.fetchone()
                if user:
                    security_q1, security_q2 = user

                    # Send security questions back to the client
                    response = {
                        'status': 'success',
                        'message': 'Security questions retrieved.',
                        'security_q1': security_q1,
                        'security_q2': security_q2
                    }
                    return response
                else:
                    return {'status': 'error', 'message': 'Email not
found.'}
            except sqlite3.OperationalError:
                return {'status': 'error',
                        'message': "There is problem in the service's Data
Base, please close the app, and try to again later"}

    def reset_password(self, data):
        """
        Reset the user's password by verifying the security answers and
        updating the password in the database.
        :param data: A dictionary containing the email, security answers,
        and the new password.
        :return: A dictionary containing the status and message indicating
        the result of the password reset.

```

```

'''
with self.db_lock:
    try:
        email = data['email']
        answer1 = data['security_a1']
        answer2 = data['security_a2']
        new_password = data['new_password']

        # Retrieve security answers from the database
        self.cursor.execute('SELECT security_a1, security_a2 FROM
users WHERE email=?', (email,))
        user = self.cursor.fetchone()
        if user: # we need to check again if the mail exists in
the db, because the user can chnagne the mail sunddelny in the middle of
the process of chnging passowrd.
            stored_a1, stored_a2 = user
            # Verify the answers
            if answer1 == stored_a1 and answer2 == stored_a2:
                # Update the password in the database
                self.cursor.execute('UPDATE users SET password=?
WHERE email=?', (new_password, email))
                self.conn.commit()
                return {'status': 'success', 'message': 'Password
reset successful.'}
            else:
                return {'status': 'error', 'message': 'Incorrect
security answers.'}
        else:
            return {'status': 'error', 'message': 'Email not
found.'}
    except sqlite3.OperationalError:
        return {'status': 'error',
                'message': "There is problem in the service's Data
Base, please close the app, and try to again later"}

def register_user(self, data):
    '''
    Register a new user in the database.
    :param data: The data containing user registration details.
    :return: A dictionary containing the registration status and
message.
    '''
    with self.db_lock:
        try:
            email = data['email']
            password = data['password']
            security_q1 = data['security_q1']
            security_a1 = data['security_a1']
            security_q2 = data['security_q2']
            security_a2 = data['security_a2']
            birthday_date = data['birthday_date']

            self.cursor.execute(

```

```

        'INSERT INTO users (email, password, security_q1,
security_a1, security_q2, security_a2,birthday_date) VALUES (?, ?, ?, ?,
?, ?, ?)',
        (email, password, security_q1, security_a1,
security_q2, security_a2, birthday_date))
        self.conn.commit()
        return {'status': 'success', 'message': 'Registration
successful.'}

    except sqlite3.IntegrityError: # becuase i defined email
column as unique it there can't be in this column the same string - the
same mail, so will catch sqlite integrity error
        return {'status': 'error', 'message': 'Email already
exists.'}

    except sqlite3.OperationalError:
        return {'status': 'error',
                'message': "There is problem in the service's Data
Base, please log out, and try to register later"}

    def login_user(self, data):
        '''
        Authenticate user and login in if ht password is verified posistive
wuth the hashed one in the db.
        :param data: The data containing user login details.
        :return: A dictionary containing the login status and some data.
        '''
        with self.db_lock:
            try:
                email = data['email']
                password = data['password']
                # rertives the hashed passowrd
                self.cursor.execute('SELECT password FROM users WHERE
email=? ', (email,))
                hashed_password = self.cursor.fetchone()
                # checks if the email exists and that the hashed password
and the regular one are the same
                if self.email_exists(email) and
self.verify_password(password, hashed_password):
                    # if the two passowrd are the same we can connect the
user
                    self.cursor.execute('SELECT * FROM users WHERE email=?
', (email,))
                    user = self.cursor.fetchone()
                    if user and user[8] >= 5: # the user banned
                        return {'status': 'error',
                                'message': 'You got banned due to violating
the service policy.\nIn order to cancel the ban you can pay fine of 100$'}
                    # the user not banned
                    elif user and self.is_birthday_today(user[10]) and not
self.twice_birthday_message(
                        user[0]): # the user have birthday today
                        # insret to the message table a birthday mail for
the user
                        strings = user[0].split("@")
                        self.cursor.execute(

```



```

        'INSERT INTO messages (sender, recipient,title,
message,image_data,file_data) VALUES (?, ?, ?, ?,?,?)',
        ("Service_Official@DaVinci.cl", user[0],
"Birthday",
        f"Dear {strings[0]}, Happy Birthday!\n\nThe Mail
Team wishes you a fantastic day and thanks you for using our service!",
        "WASNOT_ATTACHED", "WASNOT_ATTACHED",))
        return {'status': 'success', 'message': 'Login
successful.',
                'blocked_users': user[6], 'last_color':
user[7], 'dist1': user[11], 'dist2': user[12],
                'dist3': user[13], 'labels_names':
user[14]}

        elif user:
            return {'status': 'success', 'message': 'Login
successful.',
                    'blocked_users': user[6], 'last_color':
user[7], 'dist1': user[11], 'dist2': user[12],
                    'dist3': user[13], 'labels_names':
user[14]}

        else:
            # if the two passwords aren't the same we can't connect
the user
            return {'status': 'error', 'message': 'Invalid email or
password.'}

    except sqlite3.OperationalError:
        return {'status': 'error',
                'message': "There is problem in the service's Data
Base, please log out, and try to login later"}

    def receive_message(self, data):
        """
        Save the received message to the database.
        :param data: The data containing message details.
        :return: a dictionary containing the message sending status and
message.
        """
        with self.db_lock:
            try:
                recipient = data['recipient']
                if self.email_exists(recipient):
                    sender = data['sender']
                    title = data['title']
                    message = data['message']
                    file_data = data['file_data']
                    image_data = data['image_data']
                    self.cursor.execute(
                        'INSERT INTO messages (sender, recipient,title,
message,image_data,file_data) VALUES (?, ?, ?, ?,?,?)',
                        (sender, recipient, title, message, image_data,
file_data,))

                    self.conn.commit()
                    return {'status': 'success', 'message': 'Message sent
successfully.'}

```

```

        else:
            return {'status': 'error', 'message': 'Email does not
exists.'}
    except sqlite3.OperationalError:
        return {'status': 'error',
                'message': "There is problem in the service's Data
Base, please log out, and try to connect later"}
    def retrieve_messages(self, data):
        """
        Retrieve all the unread (read=0) messages for a specific recipient.
        :param data: The data containing recipient details.
        :return: a dictionary containing the status and the list of
messages, each message dictionary whom represents all the necessary
details.
        """
        with self.db_lock:
            try:
                recipient = data['recipient']
                self.cursor.execute(
                    'SELECT sender,title, message,
timestamp,important,readlater,deleted,file_data,image_data,labels FROM
messages WHERE recipient=? AND read=0',
                    (recipient,))
                messages = self.cursor.fetchall()
                # Mark messages as read
                self.cursor.execute('UPDATE messages SET read=1 WHERE
recipient=?', (recipient,))
                self.conn.commit()
                messages_list = [{'sender': m[0], 'title': m[1], 'message':
m[2], 'timestamp': m[3], 'important': m[4],
                                'readlater': m[5], 'deleted': m[6],
                                'image_data': m[8], 'file_data': m[7],
                                'labels': m[9]}
                                for m in
                                messages]
                return {'status': 'success', 'messages': messages_list}
            except sqlite3.OperationalError:
                return {'status': 'error',
                        'messages': "There is problem in the service's Data
Base, please log out, and try to connect later"}
    def twice_birthday_message(self, recipient):
        """
        checks if user have already got birthday message today
        :param recipient - the user's mail address:
        :return True if got, False if not:
        """
        # Get the current date in 'YYYY-MM-DD' format
        today = datetime.now().strftime('%Y-%m-%d')
        self.cursor.execute(
            """
            SELECT sender, timestamp
            FROM messages
            WHERE sender = 'Service_Official@DaVinci.cl'

```

```

        AND recipient=?
        AND DATE(timestamp) = ?
        """
        (recipient, today,)
    )
    message = self.cursor.fetchall()
    print(message)
    if message != []: # a birthday message was sent
        return True
    return False
def email_exists(self, email):
    """
    Check if a given email exists in the users table.
    :param email: The email to check.
    :return: True if the email exists, if not - False.
    """
    self.cursor.execute("SELECT 1 FROM users WHERE email = ?",
(email,))
    result = self.cursor.fetchone()
    return result is not None

if __name__ == "__main__":
    ServiceMainClass()
    MailServer()

```